2026 开源鸿蒙大学生创新大赛

完整技术报告

KernAdapt

面向混合负载的自适应内核资源管控 Agent

Adaptive Kernel Resource Control for Mixed Workloads

赛道一：系统与技术创新

OSKernel

中国科学院大学

队长：孙健康　指导教师：郑晨

研究方向：操作系统资源管理与调度

摘　要

混合负载共享计算节点时，应用性能不仅取决于获得的 CPU 时间，还取决于相互依赖的进程能否及时运行。通用调度器按任务分配资源，难以直接识别锁释放、工具调用返回等应用进展所依赖的执行关系；CPU 利用率和运行队列长度等聚合指标也不足以确定应当优先运行哪些进程、采用何种资源配置。如何识别这些依赖，并在公平性和资源约束下利用它们，是自适应内核资源管控需要解决的问题。

本文提出 KernAdapt 自适应内核资源管控系统。控制面根据运行状态与用户目标预测组合资源配置对吞吐、延迟及共存任务的影响，比较候选方案并评估选择风险；执行模块检查权限、作用范围和资源预算，通过实测结果决定保留调整或恢复原配置。模型提出资源方案，内核完成逐任务调度，二者通过结构化动作及执行记录交互。

针对两类具体依赖，本文设计了 LOCKCOOP 与 FlowCausal 。LOCKCOOP 维护等待者、具体锁、已确认持锁者及近期 CPU 的关联：LockBoost 在解锁前向持锁者提供有上限的时间片扩展，LockNest 在唤醒后根据同锁执行历史选择 CPU，分别缓解持锁者调度延迟和锁交接的局部性损失。FlowCausal 以控制组（cgroup）组织会话，通过工具调用标识和实际阻塞事件确认工具返回所依赖的进程，在会话份额内安排短时间片提前运行，并将实际 CPU 使用计入所属会话。

实验分别评估资源配置预测、决策执行和专用调度机制。KernAdapt 在 560 个实验组上的平均 regret 为 0.04357，相对因子化 GBDT 降低 30.95%；在独立的 600 次决策实验中执行 147 次调整，其中 114 次取得正收益。两组实验分别固定模型和执行策略。在 openEuler 24.03-LTS ARM64 的 256 线程 MariaDB OLTP 实验中，LOCKCOOP 相对同机 `scx_simple` 在节点 0 达到 5.30 倍吞吐，p95 延迟降低 84.8%；x86 与 RISC-V 对照分别达到 4.14 倍和 2.15 倍吞吐。在 4 个 CPU、8 个并发工作流、每种方案交错运行 3 轮的实验中，FlowCausal 相对原生 Linux 将平均完成时间降低 7.32%，全部工作流完成时间增加 2.65%。结果表明，应用依赖能够为内核资源配置与调度提供依据，但平均响应、整体完成时间和共存任务服务仍需分别评价。

关键词：混合负载；内核资源管控；Linux 调度；`sched_ext` ；eBPF；锁竞争；沙箱工作流；自适应系统

Abstract

When mixed workloads share a compute node, application performance depends both on allocated CPU time and on whether dependent processes run promptly. General-purpose schedulers distribute resources among tasks but do not directly identify the execution dependencies behind lock release or tool-call completion. Aggregate metrics such as CPU utilization and run-queue length are insufficient to determine which processes should run first or which resource configuration should be used.

This work presents KernAdapt, an adaptive kernel resource-control system. Its control plane uses runtime state and user goals to predict the effects of composite configurations on throughput, latency, and colocated tasks, compare candidates, and assess selection risk. The execution module checks permissions, scope, and resource budgets, then uses measured outcomes to retain a change or restore the previous configuration. Models propose resource plans; the kernel performs per-task scheduling.

Two specialized mechanisms address concrete dependencies. LOCKCOOP associates waiters, locks, verified holders, and recent CPUs. LockBoost extends a holder's timeslice within a bounded budget before unlock; LockNest uses per-lock execution history to place awakened tasks, addressing holder delay and handoff locality separately. FlowCausal organizes sessions with cgroups, identifies tool-return dependencies from tool-call identifiers and actual blocking events, and provides bounded early execution while charging CPU service to the owning session.

The evaluation measures configuration prediction, decision execution, and specialized scheduling separately. Across 560 experimental groups, KernAdapt achieves mean regret 0.04357, a 30.95% reduction over factorized GBDT. In a separate 600-decision experiment, it executes 147 adjustments, of which 114 yield positive utility. Each experiment uses a fixed model or policy. With 256 MariaDB OLTP threads on openEuler 24.03-LTS ARM64, LOCKCOOP achieves 5.30× throughput and 84.8% lower p95 latency than `scx_simple` on node 0; same-machine comparisons on x86 and RISC-V achieve 4.14× and 2.15× throughput. With eight concurrent workflows on four CPUs and three interleaved trials per method, FlowCausal reduces mean completion time by 7.32% while increasing makespan by 2.65% relative to native Linux. These results support using application dependencies in kernel resource decisions while evaluating mean response, overall completion, and colocated service separately.

Keywords: mixed workloads; kernel resource control; Linux scheduling; `sched_ext`; eBPF; lock contention; sandbox workflows; adaptive systems

符号与缩略语

符号或缩略语	含义
DSQ	`sched_ext` 的 Dispatch Queue，调度器用于组织待分发任务的队列。
PMU	Performance Monitoring Unit，处理器硬件性能监控单元。
PSI	Pressure Stall Information，Linux 资源压力停顿信息。
LLC	Last-Level Cache，末级缓存。
NUMA	Non-Uniform Memory Access，非一致性内存访问体系结构。
S_t	时刻 t 的进展结构、资源状态、目标与系统健康状态。
$\Pi_w(t)$	负载 w 在时刻 t 的进展结构。
$\mathcal{A}_t^{safe}$	当前满足能力、作用域、预算与恢复条件的动作集合。
C_m	KernAdapt 中机制 m 的控制描述。
I_{victim}	资源调整对共存负载造成的性能影响。

目 录

摘要 i

Abstract ii

符号与缩略语 iii

第一章 绪论 1

1.1 研究背景与问题 1

1.2 主要贡献 2

1.3 研究内容与章节安排 3

第二章 相关研究与技术基础 4

2.1 Linux CPU 调度与可编程接口 4

2.2 混部干扰与跨资源控制 4

2.3 锁、同步与竞争感知调度 5

2.4 Sandbox、代码智能体与工作流调度 5

2.5 GPU 与大模型推理调度 6

2.6 在线内核调优与智能控制 6

2.7 本章小结 7

第三章 混合负载的执行特征与内核资源管控问题 8

3.1 研究对象与系统边界 8

3.1.1 节点内混合负载 8

3.1.2 负载类别与进展单位 8

3.1.3 三个控制时间尺度 9

3.2 资源竞争与性能退化路径 9

3.2.1 CPU 排队与调度延迟 9

3.2.2 同步依赖与锁交接 10

3.2.3 缓存、NUMA 与内存带宽 10

3.2.4 沙箱阶段与文件系统路径 10

3.2.5 CPU–GPU 供给链 10

3.2.6 退化路径的耦合 11

3.3 粗粒度指标为何不足以选择机制 11

3.3.1 观测到根因是多对一映射 11

3.3.2 任务公平不等于应用进展 11

3.3.3 应用名称和静态标签不足 12

3.4 专用机制的适用边界 12
3.4.1 LockCoop 的适用条件 12
3.4.2 FlowCausal 的适用条件 12
3.4.3 其他负载的开放问题 12
3.5 内核观测空间 12
3.5.1 调度与生命周期事件 12
3.5.2 微架构与压力信号 13
3.5.3 应用与运行时语义 13
3.6 资源动作空间与冲突 14
3.6.1 可用执行接口 14
3.6.2 动作的三个边界 14
3.7 自适应资源管控问题定义 14
3.7.1 状态、动作与目标 14
3.7.2 选择正确性的判据 15
3.7.3 稳定性与代价 15
3.8 对自适应系统的设计要求 15
3.9 本章小结 16

第四章 面向异构专用机制的自适应资源管控方法 17
4.1 问题模型与基本假设 17
4.2 进展关系驱动的机制适用性 18
4.2.1 从资源现象到机制前提 18
4.2.2 动作、直接指标与端到端目标 18
4.2.3 默认策略的研究含义 18
4.3 不同频率下的观测与调度 18
4.3.1 快速数据面与慢速控制面 18
4.3.2 候选生成 19
4.3.3 受限验证与结果判定 19
4.3.4 反馈更新 19
4.4 安全性与稳定性约束 19
4.4.1 安全动作集合 19
4.4.2 切换稳定性 20
4.4.3 公平性与故障恢复 20
4.5 在线专家的作用与可信边界 21
4.6 机制描述与异构数据面接入 21
4.7 KernAdapt 系统实现 22
4.8 目标条件化的组合动作学习 22
4.8.1 动作效果、目标与约束分开表示 22
4.8.2 校准完整选择过程 23
4.9 E1 评测结果 23

4.10 本章小结 25

第五章 LockCoop ： 竞争锁交接前后的调度机制 26

5.1 引言 26
5.2 背景与动机 27
5.2.1 Linux 调度与竞争边界 27
5.2.2 可运行持锁者延迟、反复竞争与放置局部性 28
5.2.3 同一依赖的两个调度动作 28
5.3 LockCoop 设计 28
5.3.1 记录缺失的关系 29
5.3.2 有界持锁者推进 30
5.3.3 资源引导的同锁放置 30
5.3.4 组合与回退 32
5.4 实现 32
5.4.1 观测路径 32
5.4.2 调度钩子与有界状态 32
5.4.3 加载与失败语义 33
5.5 评估 33
5.5.1 实验设置 33
5.5.2 受控 Mutex 交接 34
5.5.3 策略刻画 35
5.5.4 端到端性能与机制证据 37
5.5.5 跨层对比与快速路径成本 40
5.5.6 运行区间与可移植性 41
5.6 相关工作 41
5.7 结论 42

第六章 FlowCausal：工具返回关系与会话调度 43

6.1 引言 43
6.2 背景 44
6.2.1 共享主机上的代码智能体执行 44
6.2.2 Linux 的任务与 cgroup 调度 44
6.2.3 工作流与应用接口 44
6.3 动机 45
6.3.1 调度器缺少的具体信息 45
6.3.2 公开代码库任务中的证据 45
6.3.3 问题定义与设计要求 45
6.4 HarnessReplayBench 46
6.4.1 来源任务与覆盖范围 46
6.4.2 重放记录与执行条件 46

6.4.3 避免依据候选方案选择任务 . . . 47
6.4.4 评价指标 . . . 47
6.4.5 重放一致性与可复现性 . . . 47
6.5 系统总览与设计依据 . . . 47
6.6 核心机制 . . . 48
6.6.1 记录不同工具调用产生的进程 . . . 48
6.6.2 确认真正的阻塞等待 . . . 49
6.6.3 按会话调度与有限提前运行 . . . 50
6.7 系统实现 . . . 51
6.7.1 调度器与状态组织 . . . 51
6.7.2 事件处理与调度路径 . . . 52
6.7.3 C 程序登记与异常恢复 . . . 52
6.7.4 CPU 服务核算与重新分派 . . . 52
6.8 实验评价 . . . 53
6.8.1 实验方法 . . . 53
6.8.2 研究问题一（a）：完整系统与调度基线 . . . 53
6.8.3 研究问题一（b）：会话调度的作用 . . . 54
6.8.4 研究问题二：工具返回关系的额外收益 . . . 55
6.8.5 研究问题三（a）：处理器数量与会话数量 . . . 55
6.8.6 研究问题三（b）：全部任务的覆盖检查 . . . 56
6.8.7 研究问题三（c）：成本、关系识别率与恢复 . . . 56
6.9 跨命令依赖的调度与评估 . . . 57
6.9.1 旧命令进程成为当前等待对象 . . . 57
6.9.2 固定工作量消融 . . . 57
6.9.3 八工作流对照 . . . 57
6.9.4 作用条件 . . . 58
6.10 讨论 . . . 58
6.11 相关工作 . . . 59
6.12 结论 . . . 59

第七章 专用机制的适用边界与跨场景分析 61
7.1 锁交接的两个调度机会 . . . 61
7.2 工具返回与会话公平的关系 . . . 61
7.3 证据对应与机制选择 . . . 61
7.4 适用范围 . . . 61

第八章 总结与展望 63
8.1 研究工作总结 . . . 63
8.2 主要研究结论 . . . 63
8.2.1 应用进展与任务公平具有不同语义 . . . 63

8.2.2 锁竞争需要同时处理时间与空间退化 64
8.2.3 工具返回关系决定会话内的调度次序 64
8.2.4 异构机制应统一选择原则而非数据面状态 64
8.3 研究局限 64
8.4 未来研究方向 64
8.4.1 缓存容量与内存带宽管控 64
8.4.2 异构推理的主机关键路径 65
8.4.3 容器、虚拟机与跨层协同 65
8.4.4 可验证在线专家 65
8.5 本章小结 65

研究成果说明 **77**

图目录

1.1 总体研究路线：从锁交接和工具返回关系形成专用机制，由 KernAdapt 组织适用性判断、受限执行与实验验证。组合动作学习及其固定模型评测见第四章。 2

3.1 从负载进展结构和资源瓶颈到管控动作的匹配关系。实线表示本文评估的机制路径，虚线表示由问题模型导出的扩展路径；执行关系不稳定或瓶颈不可归因时保持默认策略。 11

4.1 KernAdapt 自适应控制与资源执行结构 22
4.2 KernAdapt 的组合动作效果学习与选择流程。结果预测、目标相关排序和只能拒绝的可靠性检查分别承担不同职责；最终方案仍需通过独立执行检查。 23
4.3 E1 实验结果。左侧为完整动作留出的平均 regret；右侧为保留 Linux 和执行候选后的结果分布。两组实验分别使用固定的预测模型与执行策略。 24

5.1 普通调度器能够观察任务和拓扑，却缺少及时调度持锁者和放置被唤醒等待者所需的信息。 26
5.2 LOCKCOOP 记录一把锁与其等待者、持有者和执行位置之间的关系，并在解锁前和唤醒后采取动作。 29
5.3 有界的持锁者时间片扩展。 31
5.4 资源引导锚点的建立与使用。 31
5.5 Mutex 动作隔离。上：吞吐；下：一把热点 mutex、八把热点 mutex 与空临界区的 p99.9 锁等待。 34
5.6 受控 mutex 交接。上：吞吐；下：相对 Simple 归一化吞吐。虚线标记 128 个在线 CPU。 35
5.7 全机不绑定条件下的资源引导 LockNest。ARM 分解首次放置与锚点投影，x86 评估完整策略；误差线为 95% 置信区间。 36
5.8 有界持锁者推进控制：重复 OLTP、混合负载中受干扰任务的延迟、临界区公平性与短临界区保护实验。 36
5.9 MariaDB/Sysbench OLTP 吞吐（上）与 p95 延迟（下）。 37
5.10 CPU 干扰下的 x86 不绑定 OLTP。(a) 四个后台任务运行时的吞吐时间线；(b) 2–16 个后台任务下干扰阶段的 OLTP 吞吐，空心点为单次结果，误差棒为十次运行的 95% 置信区间；(c) LockCoop 相对 LockNest 的逐次批处理工作量，阴影标出 ±1% 区间。 37
5.11 固定 CPU 竞争下的应用吞吐量，相对 Simple 归一化（均值 ± 标准差，三轮）。 . . 38
5.12 x86 MariaDB 进程计数器：IPC、忙碌逻辑 CPU、迁移数与 L1/L3 缓存未命中率。 . 39
5.13 EEVDF 与 LockCoop 下 MariaDB 在 256 clients 时的 CPU 采样（三轮均值 ± 标准差）。各柱独立统计不同调用路径的周期占比，路径之间可能重叠。 39
5.14 使用 native pthread mutex 的 x86 OLTP 吞吐与 p99；误差线为 95% 置信区间。 40

6.1 调度器缺少的信息。Linux 能看到可运行进程和控制组，却不知道哪些后代进程结束后会解除当前阻塞等待。FlowCausal 根据内核事件重建这一关系。 43

6.2 FlowCausal 的系统结构和反馈过程。C 程序登记会话、工具调用和 cgroup 标识。内核事件更新大小固定的 BPF 状态表；sched_ext 回调根据这些状态选择进程和可用处理器；随后记录实际运行时间，供下一次调度使用。 48
6.3 根据内核事件的先后顺序确认工具返回关系。上方三行是内核事件，下方一行是 BPF 状态。进入等待时只产生候选项，随后观察到进程睡眠才确认关系；最先出现的结束事件会删除关系。若等待在进程睡眠前返回，则不会建立关系。 49
6.4 进程选择和处理器时间记录示例。普通调度按照虚拟时间选择会话 B，再选择 b_1。已经确认的等待关系可以改为让 b^* 提前运行一次，最长 2 毫秒。两种方式都按实际运行时间更新所属会话，然后继续普通调度。 50
6.5 “混合工作流组合”的完整结果。标签给出五轮绝对平均时间，细线表示样本标准差，数值越低越好。 54
6.6 工具返回关系相对仅会话调度版本的平均工作流完成时间收益，每种方法五轮。“构建等待组合”稳定改善，被等待子进程只运行约 8 毫秒的组合接近不变。 55
6.7 工具返回组合的规模变化。所有数值均为相对 Linux 的平均工作流完成时间降幅，细线为逐轮配对的 95% 置信区间。处理器规模测试使用四个会话，会话规模测试使用八个处理器。 56
6.8 跨命令依赖的固定工作量消融与八工作流对照。两类实验分别测量依赖转换的作用与完整工具执行的完成时间。 58

表目录

1.1 研究问题、研究内容与论文章节的对应关系 3

3.1 混合负载的进展单位与节点内执行特征 9
3.2 进展结构、候选动作及其适用边界 13
3.3 节点内资源管控的观测层次 14

4.1 节点内资源动作及其可恢复状态 20
4.2 两类专用机制与默认策略的控制描述 21
4.3 KernAdapt 固定模型评测的主要结果及统计范围 24

5.1 各类同步事件能够启用的调度动作。所有路径均通过独立的转换与拆除验证；性能评估聚焦于 mutex/futex 路径。 29
5.2 评估平台。 33
5.3 快速路径压力结果。固定速率运行目标 0.50 百万次 futex 操作/秒；不限速运行报告持续速率。 41

6.1 HarnessReplayBench 1.0 的任务库和测试组合。 46
6.2 内核事实、调度动作及其设计理由。 51
6.3 实验采用的实现范围。 51
6.4 混合工作流组合（平均值 ± 样本标准差，五轮）。 54
6.5 工具返回组合中的机制证据与开销。 57
6.6 八工作流完成时间，单位为秒；均值 ± 样本标准差，3 轮。 58

第一章　绪论

1.1　研究背景与问题

共享基础设施通过统一资源池提高硬件利用率，已经成为数据中心、科研计算平台和高性能计算节点的重要组织方式。数据库、存储服务、持续集成、代码构建、代码智能体沙箱和人工智能推理不再长期独占服务器，而是以不同优先级、生命周期和资源需求共置运行。Twine、Gödel 与 MAST 等系统分别展示了大规模统一资源管理、异构资源调度和跨地域训练调度的工程价值 [1–3]。集群调度器完成任务的节点选择之后，Linux 内核仍需决定每个线程何时运行、运行在哪个 CPU，以及多个负载如何共享缓存、NUMA、内存带宽和 I/O 等节点内资源。

然而，混合部署使原本相对独立的资源问题发生耦合，也使任务级资源指标与应用级进展之间出现明显差异。数据库事务可能因为持锁者长时间得不到 CPU 而整体阻塞；代码智能体会话可能在模型等待、文件检索、编译、测试和失败重试之间反复切换；GPU 推理服务即使具有空闲设备，也可能因为 CPU 端的数据准备线程、主机内存或数据传输未及时推进而形成等待数据的空闲时段。相似的 CPU 利用率、运行队列长度或 LLC miss rate 因而可能对应不同根因。仅根据“CPU 利用率升高”或“延迟上升”无法确定应该增加 CPU 份额、调整任务放置、限制内存带宽、推进锁持有者，还是继续使用默认策略。

针对上述普遍问题，已有研究从通用调度、专用机制和在线控制三个方向展开探索。Linux CFS/EEVDF 通过虚拟时间、截止期和负载均衡提供通用公平性；Lozi 等人揭示了负载均衡中的空闲核心浪费，Nest 和 Caladan 分别利用热核与微秒级资源控制改善局部性和干扰 [4–6]。在同步优化方面，LCAS、Scheduler-Cooperative Locks 和 Shuffling Locks 分别从锁密集任务识别、调度器与锁协同以及 NUMA 等待队列重排缓解锁竞争 [7–9]。在工作流与智能体系统方面，Decima 利用显式 DAG 优化作业完成时间，AgentCgroup 在工具调用边界核算并限制代码智能体资源 [10, 11]。随着 `sched_ext` 进入主线 Linux，SchedCP、Mixture-of-Schedulers 等研究又开始利用大模型或学习方法生成、选择和部署专用调度策略 [12, 13]。

但是，现有研究仍存在一个共同缺口：多数方法预先确定了优化对象或负载类别，却没有统一回答专用机制何时真正适用。通用调度器以可运行任务为基本对象，无法直接观察等待者依赖的持锁者或决定沙箱阶段完成的动态进程前沿；锁竞争机制通常只处理持锁者推进、锁公平性或 NUMA 交接中的一部分；DAG 调度要求依赖图预先可得，而沙箱进程图往往随工具执行逐步显现；在线调度器生成与路由降低了策略切换成本，却容易把应用名称或粗粒度资源现象直接解释为瓶颈。因而，可编程接口解决了“如何执行策略”，但尚未充分解决“当前瓶颈是什么”“为何应选择该机制”以及“错误选择后如何恢复”。

针对这些局限，本文从用户可见进展而不是应用名称出发，研究进展关系、资源退化路径与内核动作之间的联系。本文将多种专用机制的在线选择表述为部分可观测状态下的受约束决策问题，并设计 KernAdapt 自适应内核资源管控系统，说明机制适用条件、调度动作、直接指标和应用性能之间的关系。在数据面，本文提出 LOCKCOOP，将等待者–锁–持锁者–后继者及其 CPU/缓存一致性拓扑关系暴露给调度器，以 LockBoost 推进被依赖的持锁者，并以 LockNest 保持同锁交接在缓存一致性拓扑中的局部性；同时提出 FlowCausal，通过工具调用代际与实际阻塞等待确认工具返回关系，将有界提前运行与会话公平服务结合。在线专家只负责形成可检查的瓶颈假设与候选策略，逐任务调度和资源执行仍由确定性数据面与安全约束完成。

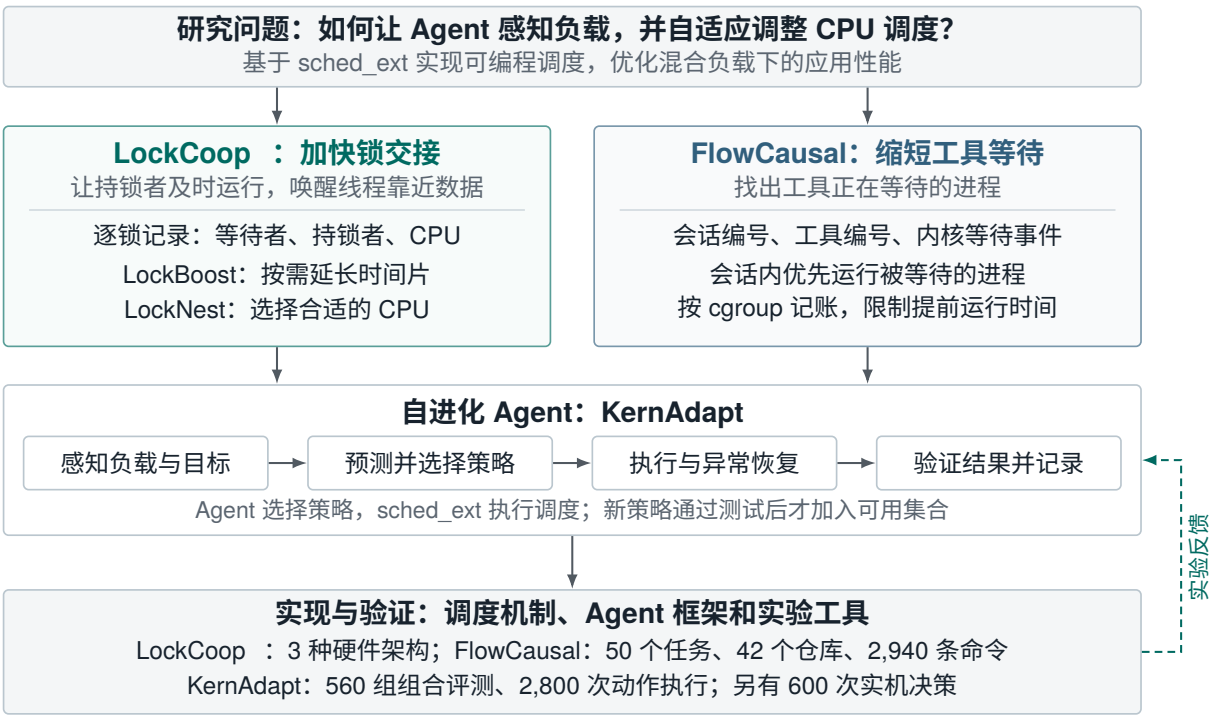


图 1.1: 总体研究路线：从锁交接和工具返回关系形成专用机制，由 KernAdapt 组织适用性判断、受限执行与实验验证。组合动作学习及其固定模型评测见第四章。

实验覆盖 openEuler 24.03-LTS ARM64、x86、RISC-V 和 VMware。256 线程 MariaDB OLTP 下，LOCKCOOP 相对 `scx_simple` 在 ARM64 节点 0 达到 5.30 倍吞吐、p95 降低 84.8%，在 x86 全机不绑定达到 4.14 倍吞吐、p95 降低 82.2%，在 RISC-V 达到 2.15 倍吞吐。CPU 干扰下，相对仅启用 LockNest，组合机制额外提高吞吐 9.6%–36.1%，批处理工作量差异不超过 0.53%。FlowCausal 在四 CPU、八工作流、每种方案交错运行三轮的对照中，平均完成时间降低 7.32%，全部完成时间增加 2.65%。独立的跨命令依赖原型在固定工作量消融中降低平均完成时间 15.79%，但未在该八工作流回放中带来额外收益。

1.2 主要贡献

本文的主要贡献概括如下。

1. **锁因果关系驱动的双阶段调度机制**。提出 LOCKCOOP，以进程作用域的逐锁因果状态连接 LockBoost 自适应有界持锁者推进和 LockNest 每锁锚点放置，在同一条锁依赖链上分别处理 unlock 前的持锁者调度延迟，并在 unlock 后保持同锁交接在缓存一致性拓扑中的局部性；通过拆分机制、直接指标和多平台应用负载验证两个动作的互补性。
2. **面向动态沙箱工作流的进展调度机制**。提出 FlowCausal，在没有预先完整 DAG 的条件下维护会话、工具调用与动态等待关系，根据短生命周期进程的创建、退出和实际阻塞确定受会话份额、预算与配额约束的调度动作，并通过有完整记录的回放分离模型等待与 CPU 调度作用。
3. **资源配置的效果预测与执行验证**。根据运行条件筛选候选方案，分别预测各项指标变化及用户目标下的收益，评估方案选择造成的预测偏差。独立执行层检查权限与参数，并处理异常恢复。KernAdapt 在 E1 实验中固定模型与策略，分别评价组合预测与执行效果，同时统计保持 Linux 默认调度的情况。

三项贡献围绕同一问题展开。LockBoost 和 LockNest 是 LOCKCOOP 内部处理锁释放前后两

表 1.1: 研究问题、研究内容与论文章节的对应关系

核心问题	研究内容	对应章节
混合负载为何需要不同机制	分析同步、工作流、批处理与异构推理的进展单位、资源退化路径、控制时间尺度以及内核观测与动作边界	第二、三章
如何选择异构专用机制	建立适用性约束、组合动作效果学习、目标相关排序与选择后风险控制，通过固定配置实验评价决策质量和执行收益	第四、七章
如何缓解锁竞争的调度退化	建立逐锁等待者–持锁者–后继者关系，分别设计持锁者推进和同锁交接局部性机制，并通过微基准与数据库实验归因	第五章
如何调度动态沙箱工作流	从工具调用来源和实际阻塞事件确认返回关系，在会话服务记账内提供有界提前运行，并通过回放及依赖变化消融评价效果	第六章

个阶段的动作；LOCKCOOP 与 FlowCausal 分别揭示两类不同的进展关系；KernAdapt 则在机制外部判断这些关系是否成立，并管理候选验证与恢复。因此，本文的研究对象不是若干并列调度器，而是从进展关系发现、专用机制设计到在线选择的完整资源管控过程。

1.3 研究内容与章节安排

围绕上述问题，本文形成四项相互衔接的研究内容，如表 1.1 所示。表中“核心问题”描述各部分试图回答的研究问题，“研究内容”给出相应的方法与证据，“对应章节”表明其在全文中的位置。

全文共八章。第一章介绍研究背景、研究缺口、本文方法、实验结果和主要贡献。第二章综述 Linux 调度、锁竞争、沙箱工作流、异构推理和在线内核调优研究。第三章分析混合负载的进展结构、资源退化路径、机制适用边界以及内核观测与动作空间。第四章研究异构专用机制的适用性判定、受约束选择、多时间尺度反馈和安全执行，并说明其在 KernAdapt 中的实现。第五章和第六章分别介绍 LOCKCOOP 与 FlowCausal 的设计、实现和实验结果。第七章综合分析两类机制的正负证据、适用边界和跨场景选择条件。第八章总结全文并讨论未来研究方向。

第二章　相关研究与技术基础

本章从调度对象、观测状态和执行动作三个维度梳理相关工作。相关研究跨越 Linux CPU 调度、同步原语、沙箱工作流和在线控制等多个层次，单纯按照实现接口分类难以揭示其研究对象和机制差异。即使同样采用 BPF 调度器，不同工作也可能分别关注内核可编程接口、锁竞争、数据库优先级或在线策略生成，因而需要结合所观测的执行关系、所施加的资源动作及其评价目标进行比较。

2.1　Linux CPU 调度与可编程接口

Linux 通用调度器长期围绕公平性、响应性和可扩展性演进。CFS 使用虚拟运行时间在任务之间近似提供比例公平，EEVDF 则在具有非负 lag 的 eligible task 中选择虚拟截止时间最早者，以改善延迟控制。通用策略的优势是无需应用知识即可覆盖大量工作负载，其代价是只能根据任务和运行队列状态作出保守选择。

Lozi 等人系统分析 Linux 调度器中的负载均衡缺陷，发现可运行任务等待时仍可能存在空闲 CPU，并将这类静默问题归因于复杂启发式之间的组合 [4]。Nest 指出 CFS 倾向扩散任务，会把新任务放到长期空闲、频率较低的核心或不必要地跨 socket 分散；其 warm-core 复用改善的是一般任务的频率和缓存热度，而不是特定锁的交接关系 [5]。Caladan 则展示微秒级 CPU、LLC 与内存带宽干扰需要比传统集群控制更快的观测和资源动作 [6]。

ghOSt 将部分调度决策委托到用户空间，为应用特定调度提供灵活控制 [14]；Arachne 从应用运行时管理核心和用户级线程 [15]；Enoki 通过安全语言、记录重放和热升级降低调度器研发成本 [16]。`sched_ext` 进一步在主线 Linux 中提供 BPF struct_ops 接口，使调度器能够实现 `select_cpu()`、`enqueue()`、`dispatch()` 等回调，并在调度器异常时回退到内核安全路径。这些工作解决了策略放在哪里实现、如何动态加载和如何调试的问题，但不自动提供负载的锁关系或工作流阶段。

kSTEP 的确定性测试表明，调度器错误往往不会触发崩溃，而会表现为公平性、局部性或利用率的静默退化 [17]。因此，专用调度器的评价不能局限于平均性能，还需要考察确定性微测试、状态生命周期、长期公平和异常回退。对于在线控制系统，这一要求进一步延伸到机制切换过程的可复现性与可追溯性。

2.2　混部干扰与跨资源控制

混部节点同时受到 CPU 排队、缓存容量、缓存一致性、内存带宽、NUMA 远端访问和 I/O 等因素影响。这些现象常被聚合指标掩盖，但其根因和动作不同。

- **缓存容量干扰**发生在多个工作集占用同一 LLC 并相互驱逐时，常通过 LLC occupancy、MPKI 和工作集变化观测，可通过 resctrl/CAT 或任务放置控制。
- **内存带宽竞争**发生在多个执行单元共同压满内存控制器或互连时，需要 uncore PMU、MBM 或带宽采样，控制动作包括并行宽度、NUMA 放置和带宽限制。
- **缓存一致性竞争**发生在多个 CPU 反复写同一缓存行时，HITM、remote HITM 和交接距离比一般 LLC miss 更直接；LockNest 处理的主要是这一现象。
- **局部性损失**可能由任务迁移或错误拓扑抽象造成。vSched 说明云虚拟机中的 vCPU 容量与拓

扑若不准确，guest 调度器会作出错误放置 [18]。

在集群与服务混部方面，LibPreemptible 利用用户态中断提供微秒级抢占 [19]；Fluid 以数据集抽象和弹性缓存改善云上训练的数据亲和性 [20]；内存分层研究使用访问热度和迁移机制管理异构内存 [21]。这些工作说明“资源管控”不能被简化为 CPU scheduler selection。KernAdapt 因而把 `sched_ext`、cgroup、resctrl、NUMA 与运行时接口建模为不同执行器，而不是假设一个调度器可以解决全部干扰。

2.3 锁、同步与竞争感知调度

锁竞争把同步语义和 CPU 调度连接起来。持锁者在临界区内被延迟运行时，等待者无法推进；锁释放后，后继者若被放置到远端 CPU，锁字和被保护数据需要跨缓存层次迁移。现有工作分别从调度器、锁算法和应用运行时处理这一问题。

LCAS 根据任务锁等待比例识别 lock-intensive task，并把它们聚集到 special set of cores，以降低多核可扩展性崩溃 [7]。其决策对象是任务总体锁强度和 SSC 大小，并不维护某一把锁的 owner 与 successor。Scheduler-Cooperative Locks 讨论锁使用模式对比例共享调度目标的颠覆，并通过 lock opportunity 使同步原语配合调度目标 [8]。CFL 则把锁占用视为跨 cgroup 可分配资源，关注共享锁的公平使用 [22]。

Shuffling lock 通过重排等待队列改善 NUMA 局部性和 park/wake 行为 [9]。SynCord 使用 eBPF 和 live patching 定制同步策略，说明应用特定同步机制可以在运行时部署 [23]。数据库领域的 contention-aware lock scheduling 改变锁授予顺序以降低事务延迟 [24]；UFS 在 PostgreSQL 混合负载中使用应用 hint 处理优先级反转，并以 `sched_ext` 隔离时延敏感任务与后台任务 [25]。

现有方法分别刻画任务级锁强度、锁授予公平性或等待队列局部性，但较少在调度器中显式维护 waiter–lock–owner–successor 的逐锁关系，也未同时连接 unlock 前的持锁者推进与 unlock 后的后继者放置。由此形成的研究空白在于：能否以同一锁因果状态贯通两个调度阶段，并在状态失效或公平预算耗尽时恢复通用调度。

2.4 Sandbox、代码智能体与工作流调度

传统 DAG 调度器假设阶段和依赖由上层系统显式给出。Decima 直接以作业 DAG、阶段并行度和资源分配为输入，学习优化作业完成时间 [10]。代码构建、测试和代码智能体也具有阶段结构，但 shell、构建工具和测试框架会动态派生进程，任务开始前通常没有完整 DAG。

Firecracker 展示 microVM 能够在多租户环境提供强隔离、较低内存开销和快速启动 [26]。但是隔离边界并不等于进展边界：给整个 sandbox 分配相同 CPU 权重，无法区分当前阻塞会话的测试进程和可以延后的后台打包任务。在 microVM 场景，主机调度器主要看到 VMM/vCPU 线程，甚至无法直接看到 guest 内部进程；这要求运行时或 guest 提供最小状态接口。

SWE-bench 从真实 GitHub issue 构造需要修改仓库并通过测试的软件工程任务 [27], SWE-agent 说明 agent–computer interface 会显著影响任务执行 [28]，Terminal-Bench 将评测扩展到构建、配置和终端操作 [29]。这些基准实验表明，代码智能体的执行不止于一次大模型推理，还包括

$$\text{reasoning} \rightarrow \text{tool} \rightarrow \text{process tree} \rightarrow \text{state} \rightarrow \text{next turn} \tag{2.1}$$

构成的连续执行过程。CPU 调度影响工具执行时间，进而影响固定时间预算内能够完成的工具调用次数。

AgentCgroup 测量 coding-agent 工具调用的 CPU 和内存突发，并使用 tool-call 子 cgroup、eBPF、`sched_ext` 与 memcg hook 实施优先级保护、throttle 和 freeze[11]；其结果还表明，内存可能先于 CPU 限制并发密度。Crab 使用 agent turn 语义判断状态是否需要 checkpoint，并在模型等待期间重叠恢复相关工作 [30]。Cortex 与 SAGA 则面向 agentic serving 的 GPU 资源池化、KV cache 和 workflow fairness[31, 32]。与上述工作相比，FlowCausal 的调度对象不是工具调用的静态资源容器或模型服务请求，而是主机 CPU 上随阶段动态变化的进程前沿。

FlowCausal 聚焦剩余的、更具体问题：在不知道完整 DAG 且不信任资源需求提示的条件下，能否从会话、阶段、进程生命周期和调度事件识别当前推进工作流的进程集合；这些进程的执行是否受到可干预的 CPU 排队、协同尾部或局部性损失；有界调度动作能否在多会话环境改善完成时间且保持公平。

2.5 GPU 与大模型推理调度

GPU 资源管理至少包含集群、设备和请求三个层次。Gandiva、Tiresias、Themis、AntMan 和 Pollux 分别研究训练作业的时间片、未知作业长度、finish-time fairness、GPU 资源动态伸缩和 goodput 联合优化 [33–37]。REEF 与 XSched 处理 GPU/XPU command 或 kernel 的微秒级抢占和并发 [38, 39]。Linux CPU scheduler 不能替代设备命令队列和硬件抢占。

在 LLM serving 中，Orca 的 iteration-level scheduling、vLLM 的 PagedAttention、Sarathi-Serve 的 chunked prefill、DistServe 的 prefill/decode 解耦以及 Llumnix 的请求与 KV 状态迁移，已经覆盖请求组织、KV 内存和阶段调度 [40–44]。PowerInfer 进一步研究 CPU–GPU hybrid inference 的神经元放置 [45]。

上述研究表明，Latency、Throughput 和 Batch 描述的是优化目标或请求组织方式，而非彼此独立的内核调度范式。只有当分词、采样、算子启动、主机内存访问、数据传输或 CPU 工作线程的执行限制了推理进度时，调整 CPU 调度才可能改善端到端性能。因此，异构推理在本文中用于界定节点内资源管控的作用边界，而不被预设为一种已经成立的专用机制。

2.6 在线内核调优与智能控制

在线内核优化可以分为编译期配置搜索、运行时参数调节、调度器生成和策略路由。AutoOS 使用 LLM 搜索 Linux Kconfig，解决依赖复杂且需要反复构建启动的配置空间 [46]。Xkernel 将性能关键常量转化为可在运行内核中安全更新的 knob，并处理更新副作用 [47]。二者表明“自动调内核配置”或“在线修改参数”都已有直接近邻。

SchedCP 将工作负载分析、调度策略库和执行检查模块组织为面向大模型智能体的控制平面，并用智能体生成和部署 `sched_ext` 调度器 [12]。Mixture-of-Schedulers 使用学习模型识别工作负载，再通过时间加权投票选择专用调度器 [13]。Expert in Residence 研究常驻专家式操作系统调优 [48]，KnowOS 用结构化知识约束 LLM 的内核配置生成和版本漂移 [49]。

现有在线调优研究主要沿配置搜索、参数更新、调度器生成和专家路由四条路径展开，其共同特点是把策略生成或选择作为核心问题。相比之下，节点内混合负载还要求控制系统回答三个相互关联的问题：候选机制的执行关系是否真实存在，候选动作能否在资源与公平预算内实施，以及动作失效后能否恢复到可验证的安全状态。KernAdapt 因而以机制前提和直接指标而非工作负载标签组织选择依据，将 `sched_ext` 与 cgroup、resctrl、NUMA 等接口视为不同执行器，并把受控验证、失败归因和恢复纳入同一控制过程。

2.7 本章小结

综上，现有研究已经在通用 CPU 调度、内核可编程接口、同步优化、沙箱隔离、工具调用资源管理、GPU 调度和在线内核调优等方面形成了较为完整的技术体系。然而，这些工作大多针对单一资源、单一执行关系或预设负载类别设计机制，尚缺乏面向节点内混合负载的统一问题刻画。尤其是，相似资源指标可能对应不同执行瓶颈，专用机制的收益取决于其所针对的进展关系，而阶段变化又会使静态策略失效。因此，从负载进展出发刻画资源退化路径、机制适用条件和内核干预边界，是实现多种专用机制在线选择与安全执行的基础。

第三章　混合负载的执行特征与内核资源管控问题

前两章说明了混合部署和内核可编程接口的发展背景。本章进一步回答全文首先必须回答的问题：为什么节点上观察到相似的 CPU 利用率、运行队列或缓存未命中，不同负载却需要不同的调度与资源动作？关键原因在于，资源计数器只描述任务如何消耗资源，不能直接说明哪些任务决定应用进展。数据库事务可能被一个未运行的持锁者阻塞，沙箱会话可能被一个尾部编译器或测试进程限制，GPU 推理也可能因主机数据准备线程或数据传输未及时完成而出现空闲。若不先识别这些执行关系，资源动作就缺少明确的作用对象。

本章不提出一个新的固定调度器，也不把某组计数器包装成通用分类器，而是从负载进展、资源退化路径、机制适用边界以及内核可观测/可控制范围四个方面建立全文的问题模型。该模型用于解释为什么 LOCKCOOP 与 FlowCausal 需要不同状态，也用于约束第四章 KernAdapt 可以选择和验证的动作范围。

3.1　研究对象与系统边界

3.1.1　节点内混合负载

本文研究集群或用户已经把负载放到计算节点之后的节点内资源管控。Twine、Gödel 等集群系统在作业、容器和资源池层完成容量分配与放置 [1, 2]；节点内仍由 Linux 决定线程何时运行、运行在哪个 CPU，以及 cgroup、NUMA、缓存、内存带宽和 I/O 资源如何分配。本文不重新设计集群排队与跨节点放置，而关注多个已经获准运行的负载如何在同一内核上共享资源。

图 1.1 给出本文采用的研究路线。集群编排完成节点选择之后，用户态控制层汇集观测、提出并验证节点内资源方案，内核中的 BPF 探针与 `sched_ext` 接口维护逐任务状态并执行调度。本文在这一边界内分别研究锁交接、工具返回关系以及候选方案的选择与恢复；具体部署可以采用进程、容器或虚拟机，而不依赖某一编排平台。

设节点上的负载集合为 $\mathcal{W} = \{w_1, \ldots, w_n\}$。每个负载 w 在时刻 t 具有一组活动任务 $\mathcal{T}_w(t)$，但真正决定其短期进展的任务通常只是其中一个动态子集 $\mathcal{P}_w(t) \subseteq \mathcal{T}_w(t)$。通用调度器直接管理的是 $\mathcal{T}_w(t)$ 中的可运行任务；用户关心的却是事务完成、阶段越过屏障、会话返回结果或设备收到下一批工作。二者之间的差异构成本文的基本研究对象。

为了避免把“进展”抽象成单一吞吐量，本文用四元组描述一个负载实例：

$$W_w(t) = \langle \mathcal{T}_w(t), \Pi_w(t), \mathcal{R}_w(t), Y_w(t) \rangle, \tag{3.1}$$

其中 $\Pi_w(t)$ 表示当前进展结构，例如锁依赖或阶段前沿；$\mathcal{R}_w(t)$ 表示该结构经过的 CPU、缓存、内存和设备资源路径；$Y_w(t)$ 表示吞吐、尾延迟、阶段完成时间或公平性等目标。资源管控需要同时知道“谁决定进展”和“哪段资源路径阻止它进展”，而不能只知道任务属于哪个进程或容器。

3.1.2　负载类别与进展单位

表 3.1 给出本文涉及的主要负载类别。分类依据不是应用名称，而是用户可见完成事件和决定该事件的执行关系。

表 3.1: 混合负载的进展单位与节点内执行特征

负载类别	用户可见进展	典型执行结构	主要节点资源路径
数据库与同步密集服务	事务、请求或操作完成	等待者–锁–持锁者–后继者	CPU 服务、锁交接、缓存一致性、NUMA
代码智能体与 CI 沙箱	工具结果、阶段或会话完成	会话–阶段–进程组–关键前沿	CPU 排队、并行宽度、文件系统、阶段交接
构建与批处理	作业或流水线阶段完成	动态任务图、同步屏障、工作线程组	CPU、LLC、内存带宽、I/O
CPU 或混合推理	token、请求或批次完成	worker、barrier、算子与内存访问链	CPU、LLC、NUMA、内存带宽
GPU 主导推理	迭代、批次或请求完成	请求队列–feeder–传输–GPU 执行	主机 CPU、页锁定内存、互连与 GPU

同一应用在不同配置下可以跨越类别。例如，CPU-only 推理主要受 worker 并行、缓存和内存带宽影响，采用 GPU offload 后又会增加主机准备与设备传输路径。数据库也并非必然由锁主导，低并发查询可能受 I/O 或执行计划限制。因此表 3.1 只定义分析起点，不作为调度器选择标签。

3.1.3 三个控制时间尺度

混合负载的状态变化跨越不同时间尺度。微秒级事件包括 wakeup、锁交接、任务迁移和短临界区；毫秒至秒级事件包括沙箱阶段变化、请求突发和 worker 数调整；分钟及更长时间尺度包括 cgroup 配额、缓存分区、模型装载和容量规划。Caladan 说明微秒级干扰需要快速数据面 [6]，而集群 autoscaling 或资源规划则可以使用更长观测窗口 [50]。

控制路径不能用同一周期处理所有动作。逐 wakeup 的 CPU 选择必须留在内核快速路径，阶段识别和机制选择可以在用户态控制路径运行，缓存分区或运行时重配置则需要更长的稳定窗口。时间尺度错配会产生两类错误：控制太慢时错过短暂的进展关系，控制太快时又把噪声放大为策略振荡。

3.2 资源竞争与性能退化路径

3.2.1 CPU 排队与调度延迟

Linux EEVDF 以可运行任务的虚拟时间和截止期组织公平服务，但它不知道任务完成对上层工作流的边际价值。Lozi 等人的研究表明，唤醒放置和负载均衡错误可以造成可运行任务等待而核心空闲 [4]；Nest 则说明近期活跃核心的缓存与频率状态会改变放置收益 [5]。这些问题表明，CPU 竞争不仅由总需求是否超过容量决定，还与任务何时变为可运行、被放到哪里以及其服务是否解除上层阻塞有关。

对负载 w，令 $q_i(t)$ 为任务 i 的 runnable wait，$s_i(t)$ 为实际 CPU service。仅最小化所有任务平均 $q_i(t)$ 不一定最小化 $Y_w(t)$：如果一个阶段必须等待最后一个 worker，尾部任务的等待比已经领先的 worker 更重要；如果多个等待者依赖一个持锁者，持锁者的一段连续服务可能同时释放多个阻塞任务。因此，任务级公平是必要的长期约束，却不是所有短期进展目标的充分表示。

3.2.2 同步依赖与锁交接

锁竞争使部分任务从运行队列消失：等待者阻塞后不再是普通 runnable task，调度器看到的只是仍可运行的持锁者和其他无关任务。当持锁者在 unlock 前被延迟时，锁占用区间被调度延迟人为放大；释放后若后继者被放到远端 CPU，锁字和被保护数据又需要跨 LLC 或 NUMA 边界迁移。Scheduler-Cooperative Locks、Shuffling locks 和 CFL 分别从锁语义、公平性和 NUMA 交接处理相关问题 [8, 9, 22]。

这条退化路径包含时间和空间两个阶段：unlock 前的 owner delay 决定等待者何时被释放，unlock 后的 handoff locality 决定下一次临界区以何种缓存状态执行。只增加持锁者优先级不能保证后继放置，只做普通缓存聚集也不知道哪些任务共享同一把锁。第五章据此把二者分别实现为 LockBoost 与 LockNest，再由 LOCKCOOP 组合。

3.2.3 缓存、NUMA 与内存带宽

LLC miss 是多种原因的叠加结果：工作集容量超过缓存、多个租户相互驱逐、顺序流式访问、远端 NUMA 取数以及锁保护缓存行的一致性迁移都可能增加 miss。相同 miss rate 对应的动作可能完全不同：容量干扰可能需要缓存分区或降低并行宽度，远端页面可能需要任务/内存共置，锁交接则需要依据锁身份放置后继者。Fluid 和异构内存管理研究同样说明，数据位置与访问热度需要结合运行时语义分析 [20, 21]。

内存带宽也不是简单的“越多越好”。一个流式批处理可以占满带宽却保持稳定吞吐，另一个共享 LLC 的时延负载可能因相同带宽压力发生尾延迟放大。PMU 和 resctrl MBM 能描述流量，PSI 能描述部分停顿，但它们不能独立判断哪一个负载应被限速、迁移或保留带宽。动作前仍需明确受影响的进展对象和共存负载。

3.2.4 沙箱阶段与文件系统路径

代码智能体和 CI 负载在模型等待、代码库准备、编译、测试、打包与重试之间切换。Firecracker 提供隔离边界 [26]，AgentCgroup 对工具调用核算资源 [11]，Crab 利用智能体轮次安排检查点与恢复 [30]；这些工作分别解决隔离、归属和生命周期问题，但主机调度器仍只看到沙箱内部产生的普通进程。

阶段进展可能由尾部编译器、链接器、测试进程或工作树准备决定。此时静态提高整个沙箱的 cgroup 权重会同时加速工作流正在等待其完成的任务和非关键后台工作，并可能伤害其他租户。第六章因此把“当前推动会话越过下一边界的进程集合”定义为动态前沿，而不是把沙箱身份本身当作高优先级。

3.2.5 CPU–GPU 供给链

GPU 服务系统主要在设备侧管理 batch、KV cache、prefill/decode 与迁移。Orca、vLLM、Sarathi-Serve、DistServe 和 Llumnix 分别优化迭代调度、PagedAttention、连续批处理、阶段解耦与实例迁移 [40–44]；REEF 与 XSched 研究设备级抢占 [38, 39]。这些机制说明 GPU 主导推理的核心问题通常不在 Linux CPU 调度器。

CPU 调度只有在主机侧稳定处于关键路径时才可能有意义，例如分词、请求组织、CPU 端的数据准备、页锁定内存分配或数据传输使 GPU 等待数据。反过来，若设备算子或 KV 容量已经饱和，提高 CPU 优先级不会改善完成时间，只会挤占其他负载。本文因此把推理作为待识别的扩展场景，而不把它写成已经完成的第三个调度机制。

3.2.6 退化路径的耦合

真实退化通常跨越多种资源。持锁者被抢占会增长等待集合，更多等待者又增加唤醒和缓存交接；沙箱并行宽度过大会增加运行队列、LLC 驱逐和 I/O 竞争；CPU 端的数据准备线程延迟可能造成 GPU 空闲，但盲目增加数据准备线程的并行度又可能放大内存带宽和锁竞争。因此，“当前 CPU 很忙”并不能唯一确定根因，资源动作之间也不能假设彼此独立。

3.3 粗粒度指标为何不足以选择机制

3.3.1 观测到根因是多对一映射

令 Z_t 表示真实但未完全可见的执行结构和瓶颈，内核观测为

$$O_t = g(Z_t, F_t) + \epsilon_t, \tag{3.2}$$

其中 F_t 是硬件、内核和负载配置，ϵ_t 是采样与时间对齐误差。不同的 Z_t 可以产生相近的 O_t：高运行队列既可能来自可并行计算，也可能来自大量被反复唤醒的锁等待者；高 LLC miss 既可能来自容量压力，也可能来自远端锁交接；低 GPU 利用率既可能来自 CPU 端的数据准备线程，也可能来自请求不足。式 (3.2) 是一个多对一映射，不能仅靠阈值反推出唯一动作。

图 3.1 展示本文采用的分析顺序。资源现象首先与负载的进展结构对齐，随后定位可干预的瓶颈路径。同步密集服务和沙箱工作流分别对应 LOCKCOOP 与 FLOWCAUSAL，CPU–GPU 推理则用于说明主机侧调度的作用边界；无法形成稳定关系时，控制系统保持默认策略。

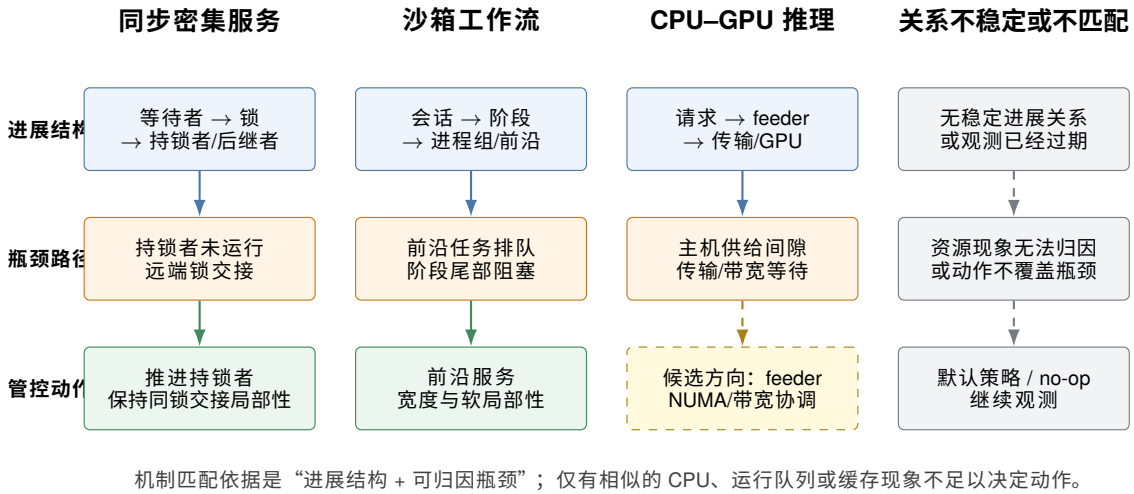


图 3.1: 从负载进展结构和资源瓶颈到管控动作的匹配关系。实线表示本文评估的机制路径，虚线表示由问题模型导出的扩展路径；执行关系不稳定或瓶颈不可归因时保持默认策略。

3.3.2 任务公平不等于应用进展

通用调度器必须避免饥饿并维持长期份额，但应用进展经常由少数任务的完成事件决定。锁持有者、阶段尾部进程和 feeder 都可能只占很少 CPU，却阻塞更多工作。相反，一个消耗大量 CPU 的后台压缩或日志进程未必位于当前用户可见关键路径。把所有 runnable task 的短期服务等同于工作流公平，会掩盖这种非对称关系。

专用机制可以在有限窗口内偏离任务级顺序，但不能取消长期公平约束。正确做法是分离两个层次：外层为 cgroup、session 或租户维持份额和最大等待，内层只在预算内重排能够推进当前结构的任务。这样，“优先推进”表示减少结构性阻塞，而不是为某个应用建立无限优先级。

3.3.3 应用名称和静态标签不足

按进程名或工作负载标签选择调度器容易在已知基准实验上得到表面命中，却无法处理配置变化。mysqld 可能由锁、I/O 或查询执行限制；sandbox 可能正在模型等待，也可能正在运行大量并行测试；CPU-only 与 GPU-offload 推理也具有不同资源路径。SchedCP 和 Mixture-of-Schedulers 表明工作负载分析与专家策略路由正成为研究方向 [12, 13]，但标签识别仍不能替代机制前提检查。

本文要求选择依据至少包含三部分：可确认的进展关系、与该关系时间对齐的资源停顿，以及候选动作能够覆盖的执行点。缺少其中任一部分，只能支持进一步观测或小范围验证，不能直接支持全局部署。

3.4 专用机制的适用边界

3.4.1 LockCoop 的适用条件

LOCKCOOP 的必要前提是存在稳定且足够强的 waiter–lock–owner 关系，并且 owner delay 或远端 handoff 对端到端进展具有可测影响。第五章的数据库和锁微基准满足该前提，因此可以分别验证持锁者推进与同锁放置。这里的适用性来自锁关系和直接指标，而不是“数据库”这个标签。

3.4.2 FlowCausal 的适用条件

FlowCausal 面向动态出现但可以由生命周期和阶段事件近似识别的会话前沿。其必要前提不是任务运行在 sandbox，而是存在能够解释阶段推进的 session–stage–cohort–frontier 状态，且工作流正在等待其完成的任务确实经历 CPU 可见等待。模型等待、远程服务等待或设备执行占主导时，Linux CPU 调度器无法直接缩短这段时间。

3.4.3 其他负载的开放问题

批处理、CPU 推理和 GPU 主导推理具有不同的进展结构，可能分别受任务图尾部、内存带宽、工作线程同步、主机数据准备线程或设备队列限制，因而需要独立的状态表征与资源动作。本文的问题模型为这些方向提供统一分析方法：先建立进展关系和瓶颈假设，再定义可验证动作与直接指标，最后评价端到端收益与资源约束。

表 3.2 总结不同进展结构的机制前提、候选动作与约束条件。

3.5 内核观测空间

3.5.1 调度与生命周期事件

`sched_ext`、tracepoint、kprobe 和 BPF map 能够记录 wakeup、enqueue、dispatch、running、stopping、fork、exec 与 exit，并将任务放置和运行时间连接起来 [16, 51]。这些事件适合回答“任务何时可运行、等待多久、在哪里运行”，也是 LockCoop 和 FlowCausal 数据面的共同基础。

表 3.2: 进展结构、候选动作及其适用边界

场景	支持机制的必要关系	可验证动作与直接指标	约束条件
同步密集服务	稳定 owner/waiter、热点锁、交接位置	owner 推进与 handoff locality；owner wait、IPC、远端交接	锁事件稀疏，动作未改变直接指标，主瓶颈为 I/O/计算
沙箱工作流	session/stage/cohort、可识别前沿、CPU 可见等待	前沿服务、宽度与软局部性；frontier wait、stage tail	模型/远程等待主导，前沿置信度低，后台工作不影响完成
CPU/混合推理	worker/barrier、内存访问与算子路径	尚待提出；需先验证 barrier、NUMA 或带宽指标	GPU 或模型算子饱和，CPU 动作不改变设备空洞
GPU 主导推理	请求–数据准备–传输–设备执行	数据准备线程、NUMA 与带宽分配；GPU 空闲时间	请求不足、KV/设备容量或设备队列主导
关系尚未识别	进展关系仍需观测或状态已经过期	默认策略、持续观测	专用动作需要明确作用对象与有效状态

但是，事件流本身不携带完整应用语义。一个 wakeup 可能来自锁释放、管道、定时器或 I/O；一个新进程可能是关键编译器，也可能是清理脚本。内核观测必须通过锁地址、父子关系、cgroup、session 注册或运行时阶段事件与上层对象对齐。仅凭 comm、PID 或一次调用栈做长期决策容易在版本变化后失效。

3.5.2 微架构与压力信号

PMU 可以提供 instructions、cycles、IPC、cache miss 和部分一致性事件，PSI 描述 CPU、内存与 I/O 压力造成的停顿，cgroup 与 resctrl 则提供资源归属和带宽/缓存监测。这些信号用于判断资源路径是否与进展停滞对齐，而不是独立产生机制选择。

采样还需要明确作用域和时间窗口。节点级 LLC miss 不能直接归因到某个 session；进程级 IPC 也可能混合多个阶段；PSI 的累计窗口可能晚于微秒级锁交接。每项观测至少应携带值、来源、时间戳、窗口和作用对象：

$$\hat{o} = \langle value, source, time, window, scope, validity \rangle. \tag{3.3}$$

缺失必须表示为 unavailable，而不能填成 0，因为“没有锁等待者”和“锁探针没有加载”会导向相反结论。

3.5.3 应用与运行时语义

可靠的阶段边界、请求 ID、会话根进程或 feeder 状态可以显著减少推断歧义，但不能要求所有遗留应用修改代码。本文采用分层来源：优先使用应用或 harness 显式事件；其次使用 cgroup、进程树和生命周期关系；最后才使用进程名或统计分类作为低置信提示。来源不同应限制动作幅度和验证范围，而不是被统一成一个没有来源信息的工作负载标签。

表 3.3 总结四类观测的能力与盲区。

表 3.3: 节点内资源管控的观测层次

观测层	代表信息	能回答的问题	主要盲区
调度事件	wakeup、enqueue、dispatch、迁移、运行时间	任务何时等待、获得多少 CPU、运行在哪里	不知道 wakeup 的应用原因
执行结构	锁 owner/waiter、进程树、session、stage、frontier	谁阻塞谁、谁可能决定当前进展	状态可能缺失、过期或识别错误
资源状态	PMU、PSI、cgroup、resctrl、NUMA、I/O	哪段资源路径出现压力或低效率	多种根因共享相似计数器
目标与健康	TPS、p95、完成时间、fairness、watchdog	动作是否值得保留、系统是否需要退出	端到端变化可能无法归因

3.6 资源动作空间与冲突

3.6.1 可用执行接口

节点内动作可以分为五类：`sched_ext` 改变任务选择、时间片和 CPU 放置；cgroup v2 改变 CPU、历史记录和 I/O 份额；resctrl 控制 LLC 容量和内存带宽；NUMA/affinity 改变任务与内存位置；应用运行时改变工作线程数、批大小、队列或数据准备参数。ghOSt 和 Arachne 等工作也说明用户态调度和运行时协同可以扩展内核默认抽象 [14, 15]。

接口越丰富，动作冲突越突出。两个全局 `sched_ext` 调度器不能同时控制同一 CPU 集合；task affinity 可能与 cpuset 或 NUMA policy 产生空交集；增加 worker width 可能与 CPU quota 相冲突；缓存分区和内存带宽限制也会改变调度器观测到的 IPC。自适应系统因此不能把执行器看成互不相关的工具列表，而要记录动作所有权、作用域、持续时间和恢复状态。

3.6.2 动作的三个边界

每个动作至少需要三个明确边界。第一是**对象边界**，即作用于任务、锁后继者、session、cgroup 还是整个节点；第二是**时间边界**，即一次 wakeup、一个阶段、一个验证窗口还是长期配置；第三是**资源边界**，即允许消耗多少 CPU、缓存、带宽以及对共存负载造成多大影响。缺少对象边界会把局部机制变成全局提权，缺少时间边界会留下过期状态，缺少资源边界则无法讨论混部公平性。

动作还必须具有已知安全状态。对 `sched_ext` 是卸载扩展调度器并恢复默认类，对 cgroup/resctrl 是恢复修改前值，对 affinity/NUMA 是恢复原 mask 和 policy，对运行时参数则是恢复注册的默认配置。若动作无法在执行前给出可检查的恢复路径，它不应进入在线控制。

3.7 自适应资源管控问题定义

3.7.1 状态、动作与目标

结合前述分析，时刻 t 的控制状态写为

$$X_t = \langle \hat{\Pi}_t, \hat{R}_t, \hat{O}_t, Y_t, F_t, H_t \rangle, \tag{3.4}$$

其中 $\hat{\Pi}_t$ 是估计的进展结构，$\hat{R}_t$ 是与其对齐的资源路径，$\hat{O}_t$ 是原始与派生观测，Y_t 是目标与公平约束，F_t 是环境配置，H_t 是观测器和执行器健康状态。估计符号强调进展结构可能不完整，不能把推断结果当成内核事实。

动作集合 $\mathcal{A}$ 包含专用调度、cgroup、缓存/带宽、NUMA/affinity、运行时参数以及默认动作 a_0。在任一时刻，真正可执行的是满足能力、作用域、预算和恢复条件的子集

$$\mathcal{A}_t^{safe} = \{a \in \mathcal{A} \mid capability(a, F_t) \wedge scope(a, X_t) \wedge budget(a, Y_t) \wedge recoverable(a, H_t)\}. \quad (3.5)$$

当没有专用动作满足条件时，a_0 表示继续使用已知安全策略。no-op 是一个需要被评价的正确决策，而不是系统没有输出。

3.7.2 选择正确性的判据

一个候选动作成立需要连续回答三个问题：当前是否存在它声称处理的进展关系；动作是否先改变与该关系直接相关的机制指标；该变化是否在约束内改善端到端目标。若端到端结果变化但直接指标不变，结果不能归因于该机制；若直接指标变化但端到端目标不变，则说明该路径不是当前主瓶颈或收益被其他路径抵消。

因此，自适应系统的选择质量需要同时考察机制匹配、动作执行和状态转换。静态机制矩阵用于度量稳态条件下的选择准确性，动态阶段实验用于度量状态变化时的检测延迟、切换开销与控制稳定性。

3.7.3 稳定性与代价

混合负载阶段变化会使候选条件反复出现和消失。控制器需要迟滞、最小驻留时间、过期时间和最大切换频率，确保观测噪声不会导致策略振荡。与此同时，观测、状态分析和验证本身会消耗 CPU、内存和日志带宽。一个选择准确但开销接近收益的系统同样没有部署价值。

安全约束优先于性能目标。候选只有在最大 runnable wait、长期份额、共存负载的性能下降、watchdog 和系统健康条件内才可以保留；性能优化不能通过让后台任务饥饿或把风险转移给共存租户实现。

3.8 对自适应系统的设计要求

前述问题分析导出以下系统要求。

1. **进展结构与资源信号分离**。系统需要分别记录谁决定进展和该对象为何停滞，避免用一个资源阈值代替根因。
2. **专用机制保持独立**。不同机制保留自己的状态、快速路径和直接指标；控制系统只通过稳定接口管理它们，不能把所有语义压成统一任务优先级。
3. **选择依据可检查**。候选必须声明所需观测、作用对象、参数范围、主要指标和不适用条件，进程名或自然语言解释不能单独取得执行权。
4. **动作有界且可恢复**。执行前明确 scope、预算、持续时间和恢复状态，观测过期、性能退化或控制器故障时能够退出。
5. **控制路径与数据路径分离**。逐事件决策留在内核或本地运行时，复杂状态分析、在线专家和历史检索不进入调度热路径。
6. **验证结果参与适配**。系统依据机制前提、直接指标和端到端指标更新候选条件与收益估计，使后续选择建立在可重放证据上。
7. **同时评价目标负载与共存负载**。吞吐和完成时间之外，还要报告公平性、最大等待、共存负载的性能下降、切换与回退开销。

第四章将这些要求具体化为 KernAdapt 的机制适配接口、状态记录、策略描述、在线选择、执行检查、效果评价和恢复流程。策略约束用于规定系统可以执行的动作及其条件。

3.9 本章小结

本章从节点内混合负载的进展单位出发，分析了 CPU 排队、锁交接、缓存与内存带宽、沙箱阶段以及 CPU–GPU 供给链的不同退化路径。分析表明，资源计数器到执行根因构成多对一映射，任务级公平、应用名称或单一调度策略均不足以表达全部应用进展。LOCKCOOP 与 FlowCausal 分别刻画锁因果结构和沙箱前沿结构，批处理与推理等场景则用于说明进展关系和资源动作的扩展空间。在此基础上，本章给出了观测空间、动作边界和自适应控制的正确性条件，为后续系统设计与机制实现提供理论依据。

第四章　面向异构专用机制的自适应资源管控方法

第三章表明，节点内资源现象与性能瓶颈之间不存在一一对应关系。运行队列增长既可能来自普通 CPU 过载，也可能来自锁持有者未及时运行或工作流前沿受阻；缓存未命中升高既可能表示容量竞争，也可能表示锁交接导致的一致性迁移。因而，自适应资源管控的关键并非在若干调度器之间进行名称匹配，而是判断某一专用机制所依赖的进展关系是否存在、该关系是否构成当前瓶颈，以及机制动作能否在安全边界内改变用户可见进展。

本章研究如何根据运行状态和用户目标选择资源配置，并给出 KernAdapt 的实现方法。系统先检查机制是否适用，再比较候选方案，限定运行范围和时长，最后根据实测指标决定保留调整或恢复原配置。模型负责提出方案，执行模块负责权限与参数检查。第五章和第六章分别介绍锁竞争和工具工作流的调度机制。

4.1　问题模型与基本假设

设节点内可用机制集合为

$$\mathcal{M} = \{m_0, m_1, \ldots, m_K\}, \tag{4.1}$$

其中 m_0 是满足长期公平和系统健康要求的默认策略，其余机制针对特定进展结构。机制 m 在时刻 t 的可观测状态记为 X_t^m，包含进展事件、资源计数器、作用对象、状态新鲜度和数据面健康状态。负载的真实瓶颈 Z_t 无法被直接完整观测，只能由 $X_t = \bigcup_m X_t^m$ 推断。因此，机制选择本质上是一个带有部分可观测性、切换代价和安全约束的在线决策问题。

对目标负载 w，定义机制 m 在验证窗口 $[t, t+H]$ 内的净效用为

$$U_w(m,t) = G_w(m,t) - \lambda_v I_{victim}(m,t) - \lambda_o O(m,t) - \lambda_s C_{switch}(m,t), \tag{4.2}$$

其中 G_w 表示目标负载的吞吐、尾延迟或完成时间改善，I_{victim} 表示共存负载受到的干扰，O 表示观测和控制开销，C_{switch} 表示机制加载、状态重建与热状态扰动带来的切换代价。不同量纲的指标在具体部署中由服务目标转换为相对效用，λ_v、λ_o 和 λ_s 分别表达受干扰负载、控制开销和切换成本约束。

系统在每个决策时刻求解

$$m_t^* = \arg\max_{m \in \mathcal{M}_{safe}(X_t)} \mathbb{E}[U_w(m,t) \mid X_t], \tag{4.3}$$

其中 $\mathcal{M}_{safe}(X_t)$ 是当前状态下满足能力、作用域、预算与恢复条件的机制集合。式 (4.3) 与一般性能优化的差异在于，默认策略 m_0 始终是合法候选；当专用机制的前提证据不足、预期收益不能覆盖切换代价或安全集合为空时，保持 m_0 即为正确决策。

上述模型建立在三个基本假设上。第一，专用机制必须给出能够在执行前检查的适用条件，而不能仅声明适用的应用名称。第二，机制必须暴露至少一个比端到端指标更接近其调度动作的直接指标，以区分“动作未生效”和“动作生效但并非主要瓶颈”。第三，任何在线动作都具有有限作用域和可执行恢复路径；无法满足该条件的机制不进入在线候选集合。

4.2 进展关系驱动的机制适用性

4.2.1 从资源现象到机制前提

对机制 m，以谓词 $P_m(X_t)$ 表示其适用条件。该谓词由四部分组成：进展关系 $P_m^{progress}$、可干预瓶颈 $P_m^{bottleneck}$、环境能力 $P_m^{capability}$ 和观测有效性 $P_m^{freshness}$，即

$$P_m(X_t) = P_m^{progress} \wedge P_m^{bottleneck} \wedge P_m^{capability} \wedge P_m^{freshness}. \tag{4.4}$$

$P_m^{progress}$ 回答哪些任务或执行实体决定当前进展。例如，LOCKCOOP 需要可识别的等待者–锁–持锁者关系，FlowCausal 需要可识别的会话–阶段–进程组–前沿关系。$P_m^{bottleneck}$ 进一步判断该关系是否受到机制能够改变的资源路径限制：观察到锁关系并不意味着持锁者调度延迟是主瓶颈，观察到沙箱进程树也不意味着工作流所依赖的任务正在等待 CPU。$P_m^{capability}$ 检查内核版本、拓扑、权限和执行接口，$P_m^{freshness}$ 则排除已经过期或来源失效的状态。

这种分解避免把相关性直接解释为机制因果性。以数据库为例，进程名、CPU 利用率和 LLC miss 均不足以单独触发锁感知调度；只有热点锁关系、持锁者延迟或远端交接与端到端退化在同一窗口内同时成立时，锁调度才进入候选。对沙箱负载同样如此，运行于 container 或 microVM 只是隔离属性，不能替代前沿状态和 CPU 排队证据。

4.2.2 动作、直接指标与端到端目标

机制 m 的有效性按以下步骤评价：

$$P_m(X_t) \xrightarrow{a_m} \Delta D_m \rightarrow \Delta Y_w, \tag{4.5}$$

其中 a_m 是实际施加的调度或资源动作，D_m 是与该动作直接相关的机制指标，Y_w 是用户可见目标。对 LockBoost，D_m 可以是竞争持锁者的 runnable wait 和连续运行时间；对 LockNest，D_m 可以是同锁交接距离、迁移和一致性事件；对 FlowCausal，D_m 可以是工作流所依赖任务的等待时间、阶段尾部延迟和共享服务比例。

机制验证依次检查动作命中、直接指标和端到端目标。只有 a_m 命中预期对象、ΔD_m 沿设计方向变化、ΔY_w 稳定改善，并且受干扰负载与控制开销未越界时，候选才具备扩大作用域的依据。

4.2.3 默认策略的研究含义

默认策略不是候选集合中的空缺项，而是观测尚不充分、专用机制条件尚未成立或机制切换正在进行时的稳定状态。将默认策略显式纳入动作集合，可以使专用机制的启用建立在完整前提和验证结果上，并为阶段转换提供一致的恢复目标。

4.3 不同频率下的观测与调度

4.3.1 快速数据面与慢速控制面

资源管控事件跨越不同时间尺度。任务唤醒、锁交接和 CPU 选择发生在微秒尺度，不能等待用户态推理；阶段变化、机制适用性判断和验证窗口位于毫秒至秒尺度；长期经验更新和容量策略则处于更慢尺度。KernAdapt 因而将控制过程划分为三个层次：数据面执行逐任务调度和计数，确定性控制层检查候选与维护租约，在线专家在较长窗口上综合异构证据。

这种划分的目的不是简单地区分用户态与内核态，而是使每类状态由能够及时维护它的层次拥有。锁地址、等待者计数和调度队列状态留在 BPF 数据面；阶段摘要、环境信息和验证窗口由用户态控制层维护；跨运行的已验证案例用于在线专家形成候选。慢速层可以调整机制及其有界参数，但不能介入单次 wakeup 或在运行时生成未经验证的内核快速路径代码。

4.3.2 候选生成

候选生成首先对观测进行时间对齐和缺失检查，再按照式 (4.4) 过滤不满足前提的机制。对剩余候选，系统根据历史同类环境中的收益分布、当前服务目标和切换代价估计式 (4.2)。确定性规则、统计模型或在线专家均可承担排序过程，但输出统一为

$$d_t = \langle m, \theta, scope, H, confidence, evidence \rangle, \tag{4.6}$$

其中 θ 为有界参数，$scope$ 为 CPU、cgroup 或会话作用域，H 为验证窗口，$confidence$ 表示候选置信度，$evidence$ 引用形成判断的观测。候选只表达尚待实验确认的假设，不因生成器给出较高置信度而直接成为全局策略。

4.3.3 受限验证与结果判定

对候选 d_t，系统先保存执行前状态，再在有限作用域内运行。验证同时观察 D_m、Y_w、受干扰负载和系统健康状态。设相对默认策略的目标改善下置信界为 L_G，受干扰负载退化上置信界为 U_I，控制预算分别为 ϵ_G 和 ϵ_I。候选被接受至少需要满足

$$P_m(X_t) = 1, \qquad L_G > \epsilon_G, \qquad U_I < \epsilon_I, \tag{4.7}$$

并确认机制直接指标沿预期方向变化。若安全谓词在窗口内失效，系统立即回退；若数据缺失或方差使区间无法判定，则结果为不可归因，而不是被写入成功或失败经验。

受限验证不能完全消除在线环境中的反事实困难。对可重放负载，应使用相同输入、环境准备和随机化顺序比较候选与基线；对连续服务，可采用会话级 canary、交错窗口或影子选择，并显式记录热状态与趋势。事后最优机制只作为选择质量上界，不能向在线选择器泄露窗口结束后的状态。

4.3.4 反馈更新

一次验证形成

$$Q = \langle F, X_t, d_t, A_t, D_t, Y_t, V_t \rangle, \tag{4.8}$$

其中 F 为环境信息，A_t 为实际执行动作，V_t 为接受、拒绝、回退或不可归因结论。历史经验按“环境–前提–动作–结果–归因”组织，而不是固化为“应用名称–调度器”映射。满足验证条件的运行用于更新收益分布，其余运行用于校正适用条件、观测完整性和实验过程。

4.4 安全性与稳定性约束

4.4.1 安全动作集合

式 (4.3) 中的 $\mathcal{M}_{safe}(X_t)$ 由四类约束共同确定：机制及参数必须预先注册；作用对象不得超出授权的 CPU、cgroup 或会话；资源动作不得违反长期份额和共存负载预算；执行前必须存在可验

证的恢复状态。对调度器加载、CPU mask、cgroup、resctrl 和运行时参数等不同动作，恢复条件分别指向其执行前状态，而不能只以“控制进程退出”代替资源恢复。

表 4.1 给出统一动作空间及其安全状态。该表描述的是资源作用点，不意味着所有动作可以任意组合。同一 CPU 集合不能同时加载相互排斥的全局 sched_ext 调度器；affinity 与 NUMA policy 的交集必须非空；改变 CPU 份额与并行宽度时还需检查是否对同一瓶颈重复施加控制。

表 4.1: 节点内资源动作及其可恢复状态

作用层次	候选动作	安全状态与主要约束
sched_ext 调度	调度器切换、有界 BPF 参数、CPU 或会话作用域	恢复默认调度类；检查加载状态、CPU 集合和公平预算
cgroup v2	CPU 份额与上限、内存高低水位、I/O 权重	恢复执行前参数；保持父子层级和资源总量约束
缓存与带宽	resctrl CAT/MBA、带宽监测与分组	移回默认资源组；避免无效 way mask 和组间冲突
NUMA 与亲和性	CPU mask、内存策略和软局部性提示	恢复原 mask 与 policy；保证合法拓扑交集
应用运行时	工作线程数、队列、数据准备或阶段参数	恢复运行时默认值；不越过应用正确性边界

4.4.2 切换稳定性

阶段边界和观测噪声可能使多个机制的适用谓词反复变化。系统对机制进入与退出使用不同阈值，并设置最小驻留时间 T_{res}、最小切换间隔 T_{switch} 与最大租约 T_{lease}。T_{res} 用于覆盖机制加载和热状态重建开销，T_{switch} 抑制短周期振荡，T_{lease} 则保证控制器失联后专用动作不会无限保留。租约续期需要重新检查前提与系统健康，而不是仅由定时器自动延长。

在动作作用域不超过 S_{max}、持续时间不超过 T_{lease}，且 watchdog 与数据面执行路径相互独立的条件下，单次错误候选的直接影响被限制在 $S_{max} \times T_{lease}$ 所定义的时空范围内。该结论不保证候选一定提高性能，但给出了在线探索可以接受的必要边界。对无法限定作用域的全局调度切换，应先使用可重放负载或离线验证，而不能把 canary 概念形式化地套用到不可隔离的动作上。

4.4.3 公平性与故障恢复

专用机制可以在短窗口内改变执行顺序，但长期资源份额仍需满足外层 cgroup 或会话预算。验证过程同时检查最大 runnable wait、后台进展、Jain 公平指数和共存负载 p95/p99 延迟的增幅。目标负载改善而共存负载超过预算时，候选仍应被拒绝或回退。

故障恢复区分观测失效、候选生成器失联、执行器错误和数据面异常。前两类故障不应影响已经运行的数据面；执行器错误需要依据执行前快照撤销部分动作；sched_ext 数据面异常由内核安全退出路径恢复默认调度。控制器重启后将持久化记录与实际系统状态进行比较，不能假定上一次执行或回退已经原子完成。

4.5 在线专家的作用与可信边界

混合负载状态具有来源异构、证据不完整和替代解释较多等特点。在线专家适合完成三类工作：从系统事件、机制摘要和历史案例中提出瓶颈假设；比较多个适用机制及其作用条件；生成能够由确定性层检查的候选与解释。其价值在于降低新负载分析和跨来源证据组织的成本，而不是代替调度器快速路径或资源权限系统。

为区分模型能力与控制框架本身的收益，在线专家必须与透明规则选择器共享相同观测、候选集合和安全约束。评价时分别比较规则选择器、在线专家、全程固定机制和事后最优机制，并报告正确专用机制率、正确保持默认策略率、错误驻留时间和相对事后最优的效用差距。若在线专家仅复述应用名称，或其候选不能引用式 (4.4) 所需证据，则该决策不具备执行资格。

专家输出不直接写入 BPF map、cgroup 或系统文件。自然语言理由也不参与权限判断；执行层只接受已经注册的机制标识、参数范围、作用域和恢复动作。模型版本、输入证据与候选结果被保留用于重放，从而判断改进来自模型推理、状态观测还是安全层拒绝。该边界使模型错误表现为候选被拒绝或有限窗口回退，而不是未经约束的内核状态修改。

4.6 机制描述与异构数据面接入

上述方法要求各专用机制保留自身状态，同时向控制层提供稳定的适用性与验证信息。为此，KernAdapt 将机制 m 的控制描述写为

$$C_m = \langle I_m, P_m, A_m, D_m, Y_m, B_m, R_m \rangle, \tag{4.9}$$

其中 I_m 标识机制版本和环境能力，P_m 是式 (4.4) 的适用条件，A_m 是允许动作与参数，D_m 和 Y_m 分别给出直接指标与端到端目标，B_m 是资源和公平预算，R_m 是恢复状态。系统实现中将该结构称为 PolicyContract；它是机制接入自适应控制层的机器可检查描述，而不是独立于选择方法之外的新调度理论。

表 4.2: 两类专用机制与默认策略的控制描述

机制	适用关系与瓶颈	动作与直接指标	拒绝或回退条件
LOCKCOOP	等待者–锁–持锁者–后继者；持锁者延迟或远端交接	持锁者推进、同锁放置；owner wait、handoff locality	状态过期、热点消失、公平或受干扰负载预算超限
FlowCausal	会话–工具调用–实际等待的子进程；返回依赖受到 CPU 竞争	有界提前运行与会话内放置；等待时长、实际 CPU 服务	等待关系失效、预算耗尽、会话配额或并行上限
默认策略	无专用进展关系要求	通用公平调度；系统健康和长期进展	作为拒绝、回退及证据不足时的安全状态

数据面只需提供观测摘要、执行前检查、动作施加/撤销和结果摘要，不需要向控制层暴露不稳定的内部 map 布局。LOCKCOOP 仍由锁地址、owner/waiter 与锚点状态驱动，FlowCausal 则维护会话与工具调用编号、候选及已确认等待关系、会话与进程虚拟时间。统一描述的对象是机制边界和评价语义，而不是把不同进展结构压缩成同一种调度状态。

4.7 KernAdapt 系统实现

图 4.1 给出上述方法在 KernAdapt 中的实现结构。数据采集层汇集调度事件、机制计数器、PMU/PSI、cgroup 状态和应用进展事件；状态层按照会话与验证窗口组织带有时间、单位、来源和环境信息的记录；决策层生成并检查候选；执行层管理 `sched_ext` 、cgroup 及其他资源动作。逐任务调度始终由数据面机制完成，用户态控制过程只在较长时间尺度上改变机制与有界参数。

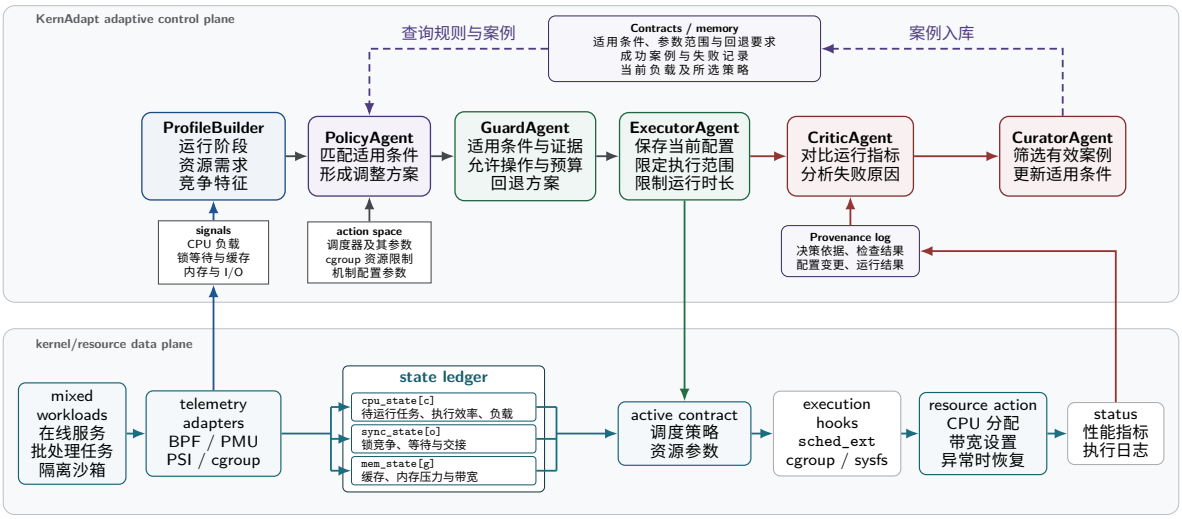


图 4.1: KernAdapt 自适应控制与资源执行结构

系统把一次决策组织为由观测、候选、约束检查、执行、测量和结论构成的状态序列。每个派生状态保留输入摘要与环境信息，使机制选择能够与实际加载动作、直接指标和端到端结果对应。原始观测采用追加式记录，策略执行不能覆盖其依据；分析方法或阈值变化后，可以在不重新执行资源动作的情况下重放候选生成与安全检查。

异构观测中的缺失状态被显式表示。例如，“等待者数量为零”和“锁探针未加载”必须产生不同状态；同样，“前沿没有 CPU 等待”和“会话阶段事件过期”不能被同一个数值零代替。该处理避免观测器故障被解释为有效系统状态，也保证历史经验只由观测完整的运行更新。

4.8 目标条件化的组合动作学习

在前述机制约束和可信执行边界之上，后续研究进一步回答一个具体问题：当调度器、CPU 亲和性和资源参数可以组成多个合法方案时，如何利用当前观测和用户目标选择完整方案，并控制从多个预测中挑选最优者所带来的误差放大？这一问题需要学习组合动作的实际效果，不能由按应用名称选择单个调度器替代。

4.8.1 动作效果、目标与约束分开表示

KernAdapt 将一次候选表示为结构化 ActionPlan，同时保留动作参数、适用范围和恢复要求。相同动作面对吞吐优先、尾延迟优先或后台服务预算时可能具有不同价值。因此，模型首先预测多项物理结果，再由用户目标确定效用；验证指标的预测和安全裕量由结果路径计算，排序模型只能决定候选优先级，不能自行生成满足约束的证据。

系统结合多指标结果模型、配对效用模型和局部检索模型。因子化组合模型使用类型化动作字段、目标和专家分歧判断各模型在当前候选上的适用性，不以完整动作编号、工作负载名称或配置

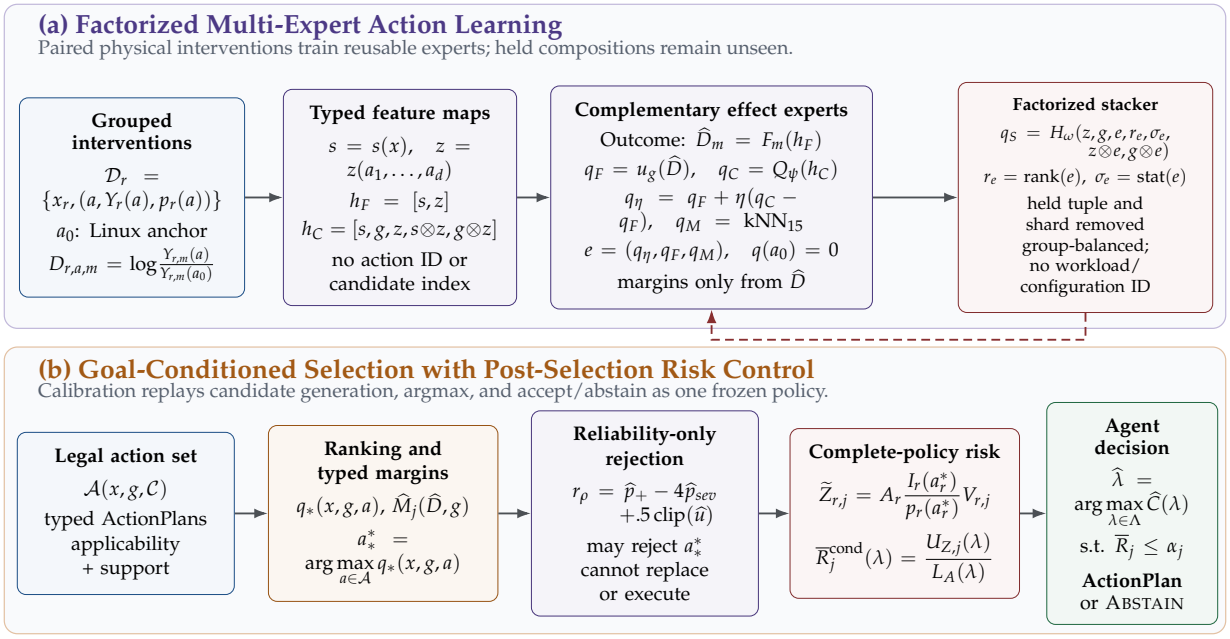


图 4.2: KernAdapt 的组合动作效果学习与选择流程。结果预测、目标相关排序和只能拒绝的可靠性检查分别承担不同职责；最终方案仍需通过独立执行检查。

身份代替组合推断。完整动作留出时，该动作的标签和采集分片从对应训练折中移除；状态支持检查在实际执行结果出现之前完成。

4.8.2 校准完整选择过程

从候选集中选出预测收益最高的动作，会使该动作的预测误差带有选择偏差。逐候选的边际置信区间因此不能直接作为最终动作的安全保证。后续方法将候选生成、评分、筛选、拒绝和 Linux 回退组成一个完整策略，以随机化的物理实验组为统计单位评价约束违规；可靠性检查只能拒绝已经选中的动作，不能另选一个未经相同校准的动作。

学习策略输出 ActionPlan 或保留 Linux。Guard 检查参数、权限与资源预算；Executor 保存并恢复执行前状态；Canary 监测运行异常；租约和独立 watchdog 限制动作持续时间。模型权重及阈值在留出评测中保持不变，记录反馈不等于在测试过程中继续训练。

4.9 E1 评测结果

组合实验包含 560 个实验组、2,800 次动作执行，评价同一主机任务分布内未完整出现过的动作组合。实际运行实验采用固定的检索策略和可靠性检查，包含 30 轮、每轮 20 个场景，共 600 次决策。两组实验分别测量组合预测质量与实际执行效果，模型和参数在评测期间保持不变。

这些结果对应 E1 主机及既有任务分布，验证了组合预测与资源配置选择的效果。regret 衡量方案选择质量，应用性能另由吞吐、延迟等实测指标评价。600 次决策中，453 次保留 Linux；147 次实际执行包含 33 次非正收益。实验未出现严重退化，对应的风险上界为 1.55%。所有决策都在取得该次运行的实际结果之前完成，结束后核对 Linux 调度和 cgroup 设置的恢复情况。

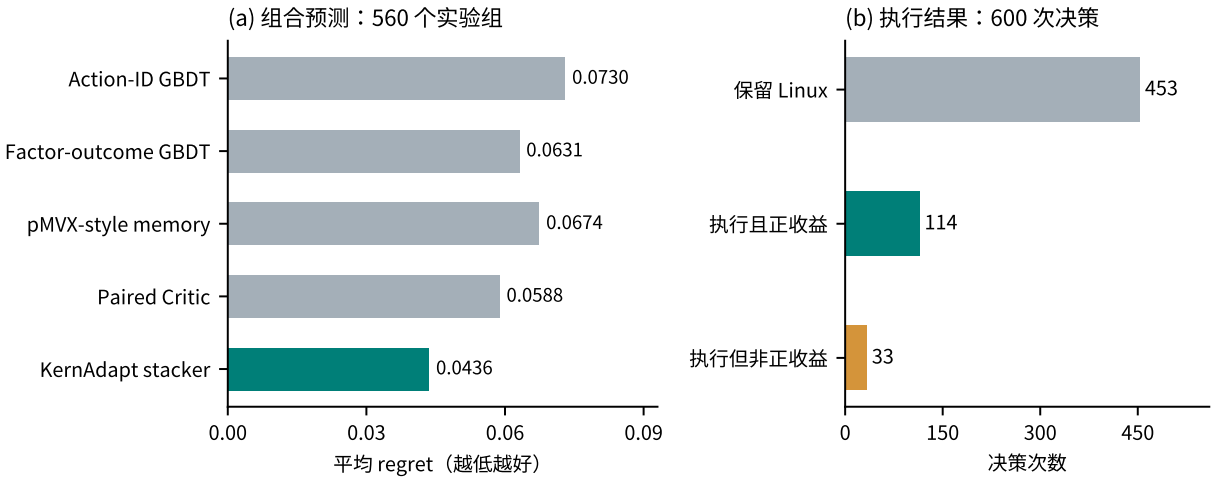


图 4.3: E1 实验结果。左侧为完整动作留出的平均 regret；右侧为保留 Linux 和执行候选后的结果分布。两组实验分别使用固定的预测模型与执行策略。

表 4.3: KernAdapt 固定模型评测的主要结果及统计范围

指标	结果	统计范围
组合模型平均 regret	0.04357	560 个物理组；越低越好
相对强因子 GBDT 的 regret 降幅	30.95%	对照为 0.06310；配对差值 95% 区间为 [0.01616, 0.02307]
相对近邻记忆的 regret 降幅	35.36%	对照为 0.06740；配对差值 95% 区间为 [0.02006, 0.02786]
实际执行动作的比例	147/600，24.50%	单侧 90% 下界为 22.23%
已执行动作的正收益比例	114/147，77.55%	不把保留 Linux 计为正收益动作
全部决策的平均相对效用	0.01398	保留 Linux 计零；按轮次分组重采样的 95% 区间为 [0.01094, 0.01694]
任一约束违规	2/147	精确单侧 90% 上界为 3.58%
严重退化	0/147	精确单侧 90% 上界为 1.55%

4.10 本章小结

本章给出了从运行状态选择资源配置的方法。系统根据等待关系、资源压力和环境能力筛选候选，再用实测指标判断方案效果。内核负责逐任务调度，用户态负责方案选择、试运行和结果分析；执行范围、资源预算、超时和恢复检查限制错误决策的影响。KernAdapt 实现上述控制过程，LockCoop 与 FlowCausal 分别提供锁竞争和工具等待场景下的调度机制。

第五章　LockCoop ： 竞争锁交接前后的调度机制

5.1　引言

多线程服务器使用工作线程处理并发请求，并通过锁保护共享数据。当线程竞争 CPU 时间时，持锁者可能在释放锁之前被切换出去。这不仅会延误持锁者，也会延误其他等待同一把锁的请求。锁释放后，将被唤醒的等待任务放到远端 CPU 上执行，可能增加访问该锁及其保护数据的开销 [52]。因此，CPU 调度既影响请求等待多久，也影响同步操作消耗多少 CPU 时间。在数据库服务器中，这些开销可能降低事务吞吐量并增加尾延迟。

Linux 默认调度器会考虑可运行任务和 CPU 拓扑，但通常不追踪哪些任务等待同一把锁、哪个任务持有该锁，以及该锁的持有者最近在哪个 CPU 上运行（图 5.1）。因此，它可能在未考虑其他任务正在等待该锁的情况下切换持锁者，也可能在未利用同一把锁执行历史的情况下，为被唤醒的等待任务选择 CPU。

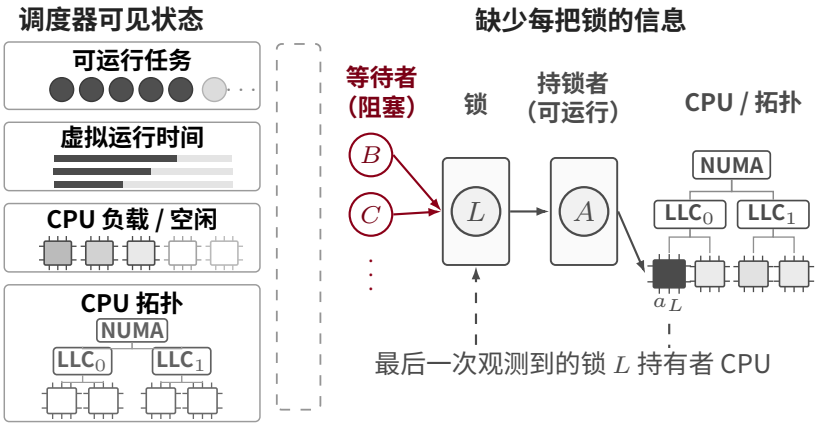


图 5.1: 普通调度器能够观察任务和拓扑，却缺少及时调度持锁者和放置被唤醒等待者所需的信息。

先前工作在不同决策位置处理上述开销。在锁实现内部，队列锁、Cohort、shuffling、CNA 与并发限制控制后继顺序或准入 [52–56]；在调度器中，LCAS、CFL 与通用缓存感知策略作用于聚合竞争或任务级复用，无法区分每把锁的执行历史 [57–59]；通过显式接口，SCL 与 SynCord 传递同步语义 [60, 61]。这些方法仍无法让操作系统调度器保留一把锁与其等待者、持有者和执行位置之间的关系。这些信息不能从持锁者服务一直保留到唤醒后的任务放置。因此，一个仍未解决的问题是：调度器能否从已有竞争路径恢复这些信息，并在不修改服务器软件的前提下作用于两个调度机会。

本文提出 LOCKCOOP ，一个 Linux `sched_ext` 调度器。它从运行时锁事件中重建这些信息，将其保存在有界的调度器状态中，并在解锁前后使用这些状态。我们把这种方法称为锁因果调度。LOCKCOOP 使用一条共享路径观测锁事件并保存状态，两个机制根据这些状态采取动作。LockCausal 记录等待每把观测到的锁的任务。在能够验证持锁者身份时，它还记录当前持锁者。此外，它记录最近一次观测到持锁者运行的 CPU。这些记录用于后续调度决策。持锁者字段指出哪个可运行任务能够释放被阻塞的等待者。CPU/拓扑字段记录同一把锁的持有者上一次在哪个 CPU 上运行。LockBoost 在解锁前采取动作，有限度地延长经过验证且受到延迟的持锁者的时间片。LockNest 在任务唤醒后采取动作，利用记录的 CPU 历史将被唤醒的等待者放置到该位置附近；被选中的 CPU

称为锚点。稳定的资源域将锚点限制在与工作负载相关的 CPU 中。完整的 LOCKCOOP 调度器同时启用两个机制。为了测量各自的效果，我们也分别启用每个机制。

对于每个观测到的事件，LOCKCOOP 记录现有证据能够支持持锁者时间片延长、后继放置或二者。记录状态会在陈旧后失效，并且状态规模与额外 CPU 服务均受到限制。没有可用证据时，LOCKCOOP 沿用普通虚拟时间调度。

受控交接分别隔离两个调度动作，端到端实验则展示后继放置何时占主导，以及持锁者时间片延长何时提供额外收益。在一台具有 64 个物理核心、128 个逻辑 CPU 的 x86 服务器上，运行 256 个线程的 MariaDB OLTP 时，LOCKCOOP 的吞吐量达到 `scx_simple` 的 4.14 倍，并将 p95 延迟降低 82.2%。在纯 OLTP 中，后继放置贡献了几乎全部收益；在 x86 CPU 干扰下，持锁者时间片延长进一步将 OLTP 吞吐量提高 9.6%–36.1%，同时不减少批处理工作量。

本文做出三点贡献：

1. 我们提出锁因果调度，为每把观测到的锁保留其等待者、持有者和执行位置之间的关系，并将其保存为有界的调度器状态。调度器在解锁前使用该状态推进持锁者，在唤醒后使用该状态放置被唤醒的等待者。
2. 我们在不修改服务器软件的前提下，在 Linux `sched_ext` 中设计并实现 LockBoost 与 LockNest，即 LOCKCOOP 内部的两个调度机制。LockBoost 基于经过验证的持锁者身份与在线反馈控制有界的时间片延长；LockNest 将稳定资源域与为每把观测到的锁记录的 CPU 执行历史结合起来。
3. 我们在 ARM64、x86、RISC-V 与 VMware guest 上验证了端到端吞吐和尾延迟收益。消融实验表明，放置在纯 OLTP 中占主导，而在 CPU 干扰下，有界持锁者时间片延长能够提供额外收益，同时不减少批处理工作量。受控交接实验与处理器计数器进一步将这些运行区间与持锁者延迟和缓存局部性联系起来。

5.2 背景与动机

5.2.1 Linux 调度与竞争边界

Linux 使用 EEVDF 依据虚拟截止期与 lag 在合格的可运行任务中选择 [62]。它提供通用的公平性与延迟纪律，但一个可运行的持锁者与一个不相干的可运行任务是普通竞争者：调度器并不知道完成其中一个可能解除大量请求的阻塞。空闲 CPU 选择与负载均衡同样只看到任务亲和性与 CPU 拓扑，而不知道哪个唤醒者、等待者和执行位置属于同一把锁。

Linux 6.12 合入的 `sched_ext` 为 CPU 选择、enqueue、dispatch、running 与 stopping 提供 verifier 强制检查的 BPF 回调 [63]。BPF map 也可以由同步路径上的探针更新。这就在锁事件与下一次调度回调之间建立了直接的内核内通路：观测记录一把锁与其等待者、持有者和执行位置之间的关系，而 `select_cpu()` 与 `tick()` 使用这些信息，无需内核补丁或用户态策略往返。这里的机会不只是可编程性，还包括把短暂锁事件的身份保留到调度器需要采取动作的时刻。

目标负载中的大多数 mutex 获取始于用户态。一个无竞争的 pthread mutex 走完快速路径，对内核有意不可见。一旦快速路径失败，等待者会在普通、共享、requeue 或优先级继承（PI）futex 路径中阻塞 [64]。之后的唤醒使等待者变得可运行，调度器再决定其执行时间与 CPU。内核 mutex 与读写信号量（rwsem）具有类似的慢路径。因此竞争天然形成了一条观测边界：无竞争操作保持不受影响，而能从调度动作获益的事件则穿过一条内核可见的路径。

然而默认接口在这一边界丢失了结构。一次唤醒只说明任务可以运行。它没有指出哪个同步对象引起了唤醒、该任务是否会在下一次获得这把锁、哪个持锁者当前阻塞了它，也没有指出这把锁

的持有者最近在哪里运行。第 3 节的逐锁记录正是用于重建这种结构。面向关键请求的 I/O 优先级机制也曾利用用户态/内核边界在阻塞路径上传递锁所有权 [65]。

5.2.2 可运行持锁者延迟、反复竞争与放置局部性

持锁者延迟会拉长锁占用。锁占用跨越从获取到释放的区间，而临界区只描述该区间内的应用工作。如果持锁者变得可运行、却排在不相干工作之后，调度延迟就会叠加到每个依赖等待者的阻塞时间上。当许多任务等待同一把锁时，一点点延迟的 CPU 服务就会变成巨大的聚合停顿。Lock-holder preemption 是这种更广义调度延迟的重要情形之一 [60, 66]。已有方案分别把运行时锁状态暴露给调度器、在 vCPU 抢占下调整自旋策略、用 eBPF 检测被抢占的临界区，或由应用显式请求临界区时间片扩展 [67–70]。

当许多等待者围绕同一把锁反复唤醒、获取失败并再次阻塞时，持锁者延迟会被进一步放大。虽然许多线程参与其中，但只有当前持锁者与少量后续任务能够推进串行化的请求路径。把所有可运行参与者当作独立负载，会在反复竞争上耗费 CPU，同时延迟真正改变锁状态的持锁者。这种现象推动了锁感知调度、竞争控制与数据库级准入机制的研究 [71–73]。

及时释放并不单独决定下一轮临界区的成本。后继任务通常会触碰前一位持锁者用过的锁字与被保护数据。相邻 CPU 之间的交接可以复用私有或共享缓存路径；跨 LLC、die 或 socket 边界的交接则需要额外的缓存一致性事务与缓存填充。因此相关拓扑是完整的缓存一致性层次，而非二元的 local/remote NUMA 标签。锁层设计通过层次化队列、调度感知的局部交接、按拓扑路由的次序或临界区委托来利用这一层次 [74–77]。

远端等待者所占比例越高，拓扑无感的选择就越可能沿昂贵的一致性路径转移所有权。反复交接会进一步放大成本，因为锁缓存行与被保护状态不断跟随后继任务迁移。LockNest 记住每把锁近期在哪里运行，并把这段历史限制在其资源域内；聚合负载均衡无法做出这种逐锁区分。

5.2.3 同一依赖的两个调度动作

持锁者推进与同锁放置解决同一条依赖的不同部分。推进持锁者改变 unlock 何时发生，却不选择缓存局部的后继任务；放置后继任务缩短下一次交接，却无法帮助一个被正确放置、但仍排在 CPU 干扰之后的持锁者。更好的放置能通过缩短临界区来减少后续的持锁者延迟，因此在一个给定负载中，一个动作可能消耗掉大部分机会。在混合 CPU 压力下，两种情况可能同时出现：交接流受益于局部性，而其当前持锁者仍需要及时服务。

两个动作的相对价值取决于负载，因此需要在受控锁竞争、端到端 OLTP 与 CPU 干扰下分别及组合评估它们。

设计目标是调度器机制，而非替换锁。LOCKCOOP 保持 pthread/futex 协议、数据库 latch、内存放置与应用二进制不变。它观测自然穿过内核的竞争路径，把该依赖重建为有界状态，并在下一个决策点提供相应的调度动作。由此得到的抽象可以与锁层准入、队列排序与内存放置共存，因为它控制的是另一种资源：为被观测依赖提供的 CPU 时间与 CPU 位置。

5.3 LockCoop 设计

图 5.2 概括系统记录的关系，以及解锁前后的两个调度动作。

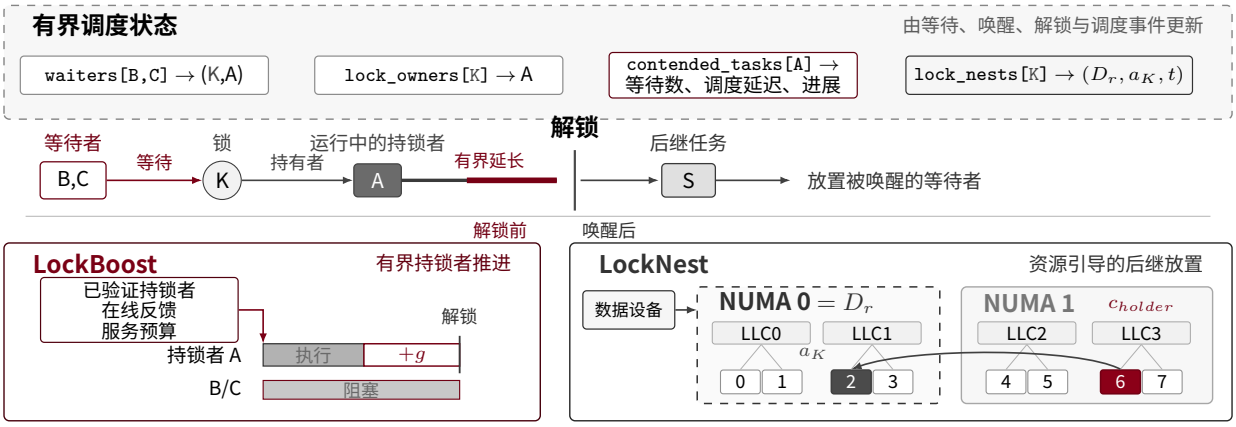


图 5.2: LOCKCOOP 记录一把锁与其等待者、持有者和执行位置之间的关系，并在解锁前和唤醒后采取动作。

表 5.1: 各类同步事件能够启用的调度动作。所有路径均通过独立的转换与拆除验证；性能评估聚焦于 mutex/futex 路径。

路径族	身份 scope	持锁者证据	启用的动作
内核 mutex/rwsem	内核对象	直接 owner/writer	推进 + 放置
私有 futex/pthread mutex	地址空间 + 地址	已验证 owner word	推进 + 放置
共享/PI futex	规范 futex key	PI owner 或交接	PI：两者；共享：放置
Futex requeue	源/目标 key	可选	放置
Pthread rwlock	地址空间 + 地址	已验证 writer / reader 集合	Writer：有界时间片延长
标注 spin/CAS	声明的 scope 与 key	可选 owner TID	由证据选择动作

5.3.1 记录缺失的关系

观测与动作发生在不同上下文。锁探针能够识别等待者和持锁者，却不能为未来的唤醒选择 CPU；调度钩子能够在唤醒时行动，但单独的唤醒已经无法说明由哪把锁引起。一条有界记录只把缺失的身份跨越这段时间传递到调度点，而不要求服务器软件传播调度元数据。

两个调度动作仍必须指向同一把锁。聚合竞争信息无法判断哪个可运行任务正在延迟哪组等待者，而虚拟地址本身也不是进程间全局唯一的身份。因此，对于一次竞争事件，LOCKCOOP 记录 $R = (w, K, o, c, t)$：等待者 w、锁键 K、可选持锁者 o、观测 CPU c 与时间戳 t。这些记录把该依赖编码为调度器使用的有界状态。带作用域的锁键 K 区分锁类型与所有权域，将同一把锁的不同别名映射到同一个键，同时区分彼此无关的锁实例。

即使不知道持锁者，一次唤醒也可为后继放置提供证据；时间片扩展则必须先验证持锁者。“已验证”表示同步路径已经确认持锁者身份；第 5.4.1 节说明锁键构造与各路径的持锁者检查。因此，每个事件都注明它允许扩展、放置或两者。带有持锁者 TID 且通过验证的 mutex、PI、rwsem writer 与标注路径允许两个动作。Pthread rwlock 探针保留 reader/writer 语义，只有已验证的 writer 才允许扩展。共享 futex 与 requeue 等不含持锁者的交接证据只允许放置。持锁者证据与时间信息同锁键分开保存，使后续事件能够增强或过期证据而不改变锁身份。

观测发生在高频同步路径上，因此状态查找与回收必须保持有界。记录状态使用有界 map，它们只保存两个动作所需的字段；每个调度钩子只进行直接且有界的查找。核心依赖状态按生命周期分开：等待者状态只消费一次，持锁者状态在 unlock 时结束，而锚点历史需要跨越一次交接继续保留。使用同一清理规则，要么会保留陈旧的持锁者，要么会过早丢弃仍有价值的局部性历史。`waiters` 把 TID 映射到锁键与可选持锁者；`lock_owners` 把锁键映射到当前持锁者；`contended_tasks` 保存

持锁者的等待任务数量、持锁者证据、可运行延迟与扩展记账；`lock_nests` 在容量 1,024 的 LRU 中保存近期同锁锚点、来源与时间信息。逐锁和逐 cgroup map 保存 LockBoost 的控制与预算状态（第 5.4.2 节）。map 丢失只移除一项优化，永远不会移除一个可运行任务。

身份与生命周期遵循不同规则。Acquire 替换持锁者状态;unlock 移除对应持锁者并刷新交接锚点;syscall 退出或 `select_cpu()` 原子地读取并清除等待者状态。锚点条目仅作为近期放置历史保留。因此, 在同一地址空间内于相同虚拟地址重新初始化 mutex 可能复用键, 但旧时间戳无法建立已验证持锁者, 过期后即被拒绝。这在不把分配器代际逻辑放到观测热路径上的前提下，限制了陈旧放置的生命周期。

5.3.2 有界持锁者推进

锁竞争可能在持锁者已经进入运行状态后才变得可见。重排调度队列无法加速这样的持锁者；剩余的机会是保持它朝 unlock 推进的执行进展。因此，LockBoost 仅在已验证持锁者既被竞争、又被延迟时行动。这里的“推进持锁者”指延长正在运行持锁者的有界剩余时间片，而非在调度队列中提升它或改变其优先级。仅有等待任务数量无法判断持锁者是否被延迟，也无法判断近期服务是否释放了等待任务。因此，控制器对等待任务数量、持锁者从可运行到实际运行的调度延迟、以及扩展后的进展使用权重为 $1/8$ 的指数加权移动平均。这三类信号回答不同问题：等待任务数量估计有多少被阻塞任务可能受益，调度延迟判断持锁者是否确实被推迟，而扩展后等待任务的完成情况用于判断额外 CPU 时间是否缩短了等待。任何单一信号都不充分：快速临界区也可能积累许多等待任务，持锁者延迟也可能来自增加 CPU 时间无法修复的原因。

等待者阈值 T 初始为 4，并始终限制在 $[2, 16]$。每个控制窗口更新一次：当持锁者延迟超过 500 μs、至少有 T 个等待任务，并且额外服务后释放了等待任务或等待任务数量达到 $2T$ 时，阈值减 1，使控制器更易触发；若前一次动作被预算或长临界区守卫拒绝，或等待任务数量与延迟都较低，则阈值加 1；其他情况保持不变。反馈只改变扩展的触发难度，持锁者验证、服务预算与扩展上限仍是硬边界。候选持锁者只能获得短扩展；至少有 T 个等待任务的已验证持锁者获得常规有界扩展；较多等待任务与较高延迟可选择更强档位。unlock 间隔低于 62.5 μs 时抑制扩展，因为持锁者已经在快速推进。

单一预算无法同时约束任务、依赖链与工作负载三个范围内的干扰。任务 bucket 防止一个持锁者长期独占 CPU。锁 bucket 把限制延续到同一热点锁的后续持锁者。cgroup bucket 则约束一个工作负载中多把锁的总体影响。同时从三级 bucket 扣除额度，可防止持锁者更替或锁数量增加绕过局部限制。对每个 bucket i，经过的时间以速率 r_i 补充 token，最多达到 B_i；大小为 g 的扩展要求三个 bucket 都有足够余额：

$$\begin{aligned} b_i &\leftarrow \min(B_i, b_i + r_i \Delta t), \\ \text{允许扩展} &\Leftrightarrow \min_i b_i \geq g. \end{aligned} \tag{5.1}$$

获准的扩展从三个 bucket 同时扣除。自适应配置使用 100-ms 窗口、20/40/40% 的任务/锁/cgroup 预算、1 与 2-ms 的常规/强扩展、4-ms 累计时间片上限、以及 5-ms 连续执行守卫。unlock 撤销活跃状态；无等待者进展的连续执行进入冷却。无限时间片仅作为显式实验消融保留。

图 5.3 总结有界持锁者推进控制器及其撤销路径。

5.3.3 资源引导的同锁放置

资源域稳定，但粒度太粗，不能保留单独一把锁的 CPU 和缓存历史。最近一位持锁者的 CPU 更精确，却可能属于偶然的资源域。LockNest 因此为每把锁记录两个位置：资源域 D_r，以及最近

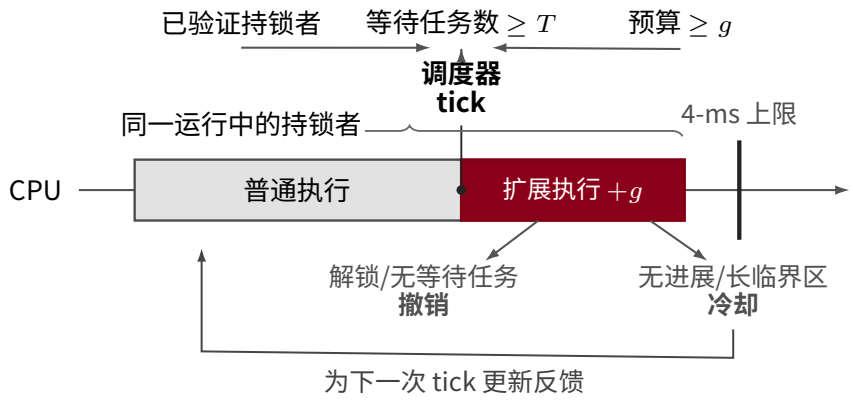


图 5.3: 有界的持锁者时间片扩展。

一位持锁者运行过的 CPU a_K，本文称 a_K 为该锁的锚点。加载器利用内核拓扑，将所配置数据设备（保存工作负载数据的块设备）所在 NUMA 节点的 CPU 定义为 D_r；若无法解析资源域，LockNest 仍为每把锁使用普通锚点路径。但在第一次交接之前，这样的锚点尚不存在。因此，每个新执行的用户任务会在资源域中放置一次。该一次性放置在逐锁锚点尚未建立、也无需等待时间窗口时，为后续锁执行提供域内的初始 CPU。稳态唤醒使用投影到同一域的锚点。两个阶段处理锚点生命周期中的不同位置：首次放置在锚点尚不存在时决定第一次锁执行的位置，投影则在持锁者更替后维持该资源域。只有首次放置无法阻止后续漂移；只有投影则必须在偶然的初始放置已经发生后再纠正它。其中，$\Pi_{D_r}(c)$ 将 CPU c 映射到 D_r 内对应的拓扑槽位，并同时满足在线与亲和性约束：

$$a_K \leftarrow \Pi_{D_r}(c_{holder}), \qquad c_{next} = a_K \text{ 若新鲜且被允许.} \tag{5.2}$$

投影保留同一把锁的持有者最近在缓存拓扑中的运行位置，也防止一个偶然的首位持有者把锁固定到不相干的 NUMA 域。离线 CPU、cpuset 违规、过期状态或被拒绝的锚点都回到普通 CPU 选择。

图 5.4 展示稳定资源域和为每把锁记录的 CPU 历史如何生成放置任务所用的锚点。

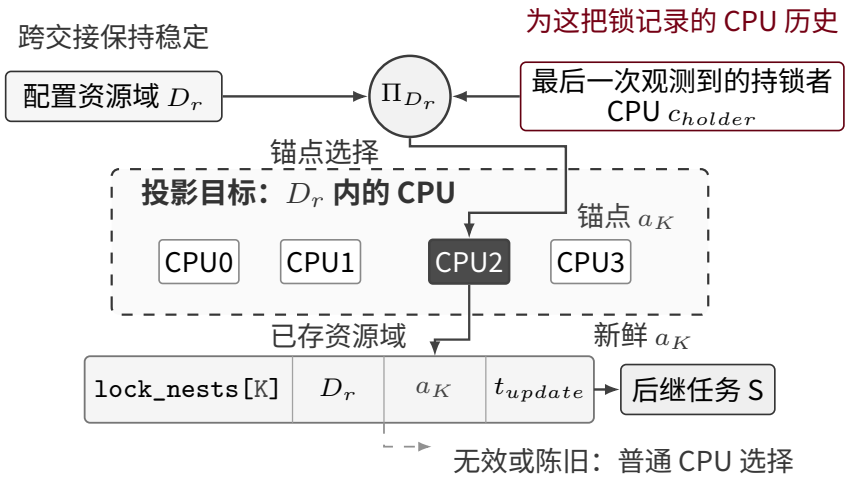


图 5.4: 资源引导锚点的建立与使用。

放置使用记录的执行历史，而不是当前等待者数量。唤醒事件确定等待者对应的锁后，有效锚点无需扫描或估计队列即可提供所需的 CPU 历史。等待者数量变化时，该锚点仍然有效。因此，选定策略使用 2-s 时间范围内的所有锚点；陈旧或非法锚点回退到普通 CPU 选择。第 5.5.3 节评估了更严格的接纳与生命周期配置，但都没有改善重复 OLTP 性能。

5.3.4 组合与回退

两个动作发生在 unlock 的两侧，并且需要不同强度的证据。组合调度器共享观测结果，但在相应钩子独立决定每个动作。统一的触发条件会丢弃仍然有用的部分证据：只有 wake 的路径可以在不知道持锁者时支持后继放置，而已验证且被延迟的持锁者即使没有有效锚点也可能需要时间片扩展。分别判断触发条件使每条依赖只触发其证据能够支持的动作。`tick()` 限制被竞争持锁者的剩余时间片；`select_cpu()` 读取并清除等待者状态并应用有效锚点。持锁者证据缺失会禁用时间片扩展，但不禁用放置；锚点状态陈旧会禁用放置，但不撤销有效的时间片扩展。若两个动作都不适用，任务使用与锁无感知 `sched_ext` 基线相同的虚拟时间路径。允许的 CPU mask 始终是允许放置任务的范围，且所有 BPF 程序在 attach 前都通过内核 verifier。

5.4 实现

5.4.1 观测路径

不同同步路径暴露不同的身份与所有权语义，而调度回调需要一个固定接口。LOCKCOOP 因而由 BPF 锁事件程序、`sched_ext` 调度回调与一个小型用户态加载器组成。内核 mutex/rwsem 路径使用 kprobe 与 tracepoint；futex 路径捕获私有、共享、PI、wake 与 requeue 语义；uprobe 或标注 ABI 覆盖 pthread rwlock 与可选用户态同步。所有事件更新同一组逐锁记录，因此调度回调只读取锁身份和允许的动作，无需处理不同路径的解析细节。

身份归一化遵循各路径的语义：内核对象使用内核地址与全局作用域；私有用户锁把虚拟地址与地址空间身份组合起来；共享与 PI futex 使用 `get_futex_key()` 返回的规范身份。对于评估采用的 glibc/NPTL 环境，加载器根据 libc 布局获得 pthread-mutex owner-word 偏移，并在启动时使用普通、递归和错误检查 mutex 完成验证。候选 TID 只有在 task 查找与同地址空间检查之后才成为已验证持锁者。owner miss 仍形成放置证据，但不能授权时间片扩展。PI 状态提供内核建立的持锁者，而 requeue 记录一个有界的源到目标重定向。Syscall 退出清理与原子等待者 claim 防止两条路径消费同一事件。

内核 mutex 入口/返回记录成功的所有权，慢路径在创建等待者依赖前解析当前持锁者。unlock 移除匹配的持锁者状态、刷新强锚点、并标记任何扩展以撤销。rwsem 观测保留一个 writer 持锁者与一个 reader 计数，而非把 reader 强行塞进单持锁者表示。pthread-rwlock uprobe 同样区分读获取、写获取、竞争与释放；只有已验证的 writer 所有权才启用面向持锁者的时间片延长。可选 spin/CAS ABI 通过同一事件接口导出 wait/acquire/release 事件与可选 owner TID。

5.4.2 调度钩子与有界状态

唤醒与 tick 都是延迟敏感的调度路径，因此策略决策避免拓扑遍历和无界搜索。`select_cpu()` 执行首放置、或消费等待者并测试锚点。`enqueue()` 记录持锁者可运行时间并按虚拟时间插入普通任务。持锁者开始执行后，`running()` 关闭其可运行延迟样本，并可赋予一次初始的短有界扩展；后续 `tick()` 可将剩余时间片延长到所选上限。两个钩子都不会将排队任务提前或重排调度队列。`stopping()` 对运行时间记账。每个调度决策使用常量次数的 map 操作；有界循环只扫描固定大小的拓扑数组。

加载器解析所配置数据设备的 NUMA 节点、填充 CPU/拓扑 map、配置控制器边界、attach 探针、然后注册调度器。detach 把任务全部交还给 EEVDF。状态 map 是建议性且有界的：分配失败、LRU 淘汰、不支持的事件或陈旧状态只增加诊断计数并选择回退路径。

owner、waiter 与 contention map 各持有 10,240 个条目；anchor 与 per-lock boost map 各 1,024；cgroup 预算状态 4,096。CPU 到节点、CPU 到 slot、节点到 CPU 数组由加载器一次性填充，因此锚点投影在唤醒时不遍历拓扑。每任务 BPF storage 携带一位首放置 token，并由 `sched_process_exec` 重置。

5.4.3 加载与失败语义

独立测试覆盖表 5.1 中的每类同步路径，检查私有映射和共享映射下的锁身份、持锁者验证及状态清理。其他测试覆盖地址复用、futex 等待队列转移、读者与写者动作的区分，以及 LRU 淘汰。这些测试均未出现验证器失败、map 更新失败、任务停滞或无法继续执行的情况。

5.5 评估

我们评估随着工作线程数量增加、等待锁的情况变得频繁，LockCoop 能否维持吞吐并限制尾延迟增长。受控 pthread-mutex 实验首先隔离持锁者时间片延长与后继任务放置，随后用策略实验评估所选边界与在线控制。MariaDB/Sysbench 将客户端数量从 64 增加到 256，用于测量并发数达到或超过 CPU 数量时的端到端行为。同时运行的 CPU 密集型后台任务用于判断持锁者时间片延长何时能在放置之上产生额外收益。随后，我们在固定 CPU 竞争下测试 MariaDB、PostgreSQL、SQLite 和 RocksDB，以考察不同应用中的效果。最后，我们在多种处理器架构上重复 OLTP，并与锁层和内存层机制比较，以评估可移植性与机制层次差异。

5.5.1 实验设置

表 5.2: 评估平台。

平台	处理器与被评估 CPU 范围	内存/存储	软件
ARM64 裸机	2× Ampere Altra Max M128-30；256 核，无 SMT；节点 0 或不绑定	1.0 TiB；Intel NVMe	openEuler 24.03；Linux 6.17；MariaDB 10.6.27
x86 裸机	2× Xeon Platinum 8375C；64 核/128 线程；节点 0 或不绑定	256 GiB；Intel NVMe	Ubuntu 24.04；Linux 6.17；MariaDB 10.11.14
RISC-V 裸机	Milk-V Pioneer；64 个 C920 核；4 个 NUMA 节点；不绑定	128 GiB；NVMe	RevyOS；Linux 6.17.13；MariaDB 11.8.6
VMware guest	32 vCPU；单 vNUMA 节点；VMware 暴露的 EPYC 9654 CPUID	128 GB；120 GB PVSCSI	Ubuntu 26.04；Linux 7.0；MariaDB 11.8.6

主负载是 Sysbench OLTP read/write，包含 16 张表、每表一百万行。性能实验重点评估会使用 mutex 和 futex 路径的负载。Linux default 与所有 `sched_ext` 用例统一使用 `performance` governor。我们对比 stock EEVDF、上游锁无感知 `scx_simple`（图中记为 Simple）、依据已发表 LCAS、CFL 与 Nest 策略完成的 `sched_ext` 移植、两个动作消融 LockBoost/LockNest，以及组合 LockCoop 。这些移植实现执行相应论文中的策略决策，并在全部正式测量中使用同一组配置：LCAS 在 1 秒窗口内采用 10/6% 的竞争滞回阈值；CFL 采用 4 个 waiter 的阈值、200 ms 衰减时间和 1 ms owner credit；Nest 采用 $r_{\max}/r_{\text{impatient}} = 5/2$、2 ms 移除时间和 20 ms 时间片 [57–59]。“固定”仅指这些外部配置参数；各策略的在线滞回、衰减、credit 与放置决策仍然正常运行。除非另有说明，每个 OLTP 配置重复十次，图中报告均值与 95% 置信区间。

Sysbench 以共置的闭式 OLTP 工作负载驱动 MariaDB。所报告的吞吐衡量端到端完成事务数。“不绑定”表示未施加外部 CPU 或内存亲和性；包括 LockCoop 资源域选择在内的调度器内部放置策略仍然生效。正式 OLTP 点使用 10-s warmup 加 60-s measurement。

应用范围实验在 x86 机器上以全机不绑定方式运行，在 128 和 256 个工作线程下比较 EEVDF、`scx_simple` 和 LockCoop 。负载包括使用 Sysbench OLTP read/write 的 MariaDB 和 PostgreSQL、采用 WAL 模式并执行 90% 读取和 10% 更新的 SQLite，以及运行 YCSB Workload A 的 RocksDB。PostgreSQL 使用 `synchronous_commit=off`。每个前台负载同时运行一个 4 线程 Fluidanimate 实例和一个 10 线程 Streamcluster 实例。每组配置预热 30 秒、测量 120 秒，并按随机顺序重复三轮。

5.5.2 受控 Mutex 交接

动作隔离。 我们首先变化锁数量与临界区工作：一把 1-μs 临界区 mutex、八把 1-μs 临界区 mutex、以及一把空临界区 mutex。图 5.5 报告到 256 线程的吞吐与 p99.9 获取等待。

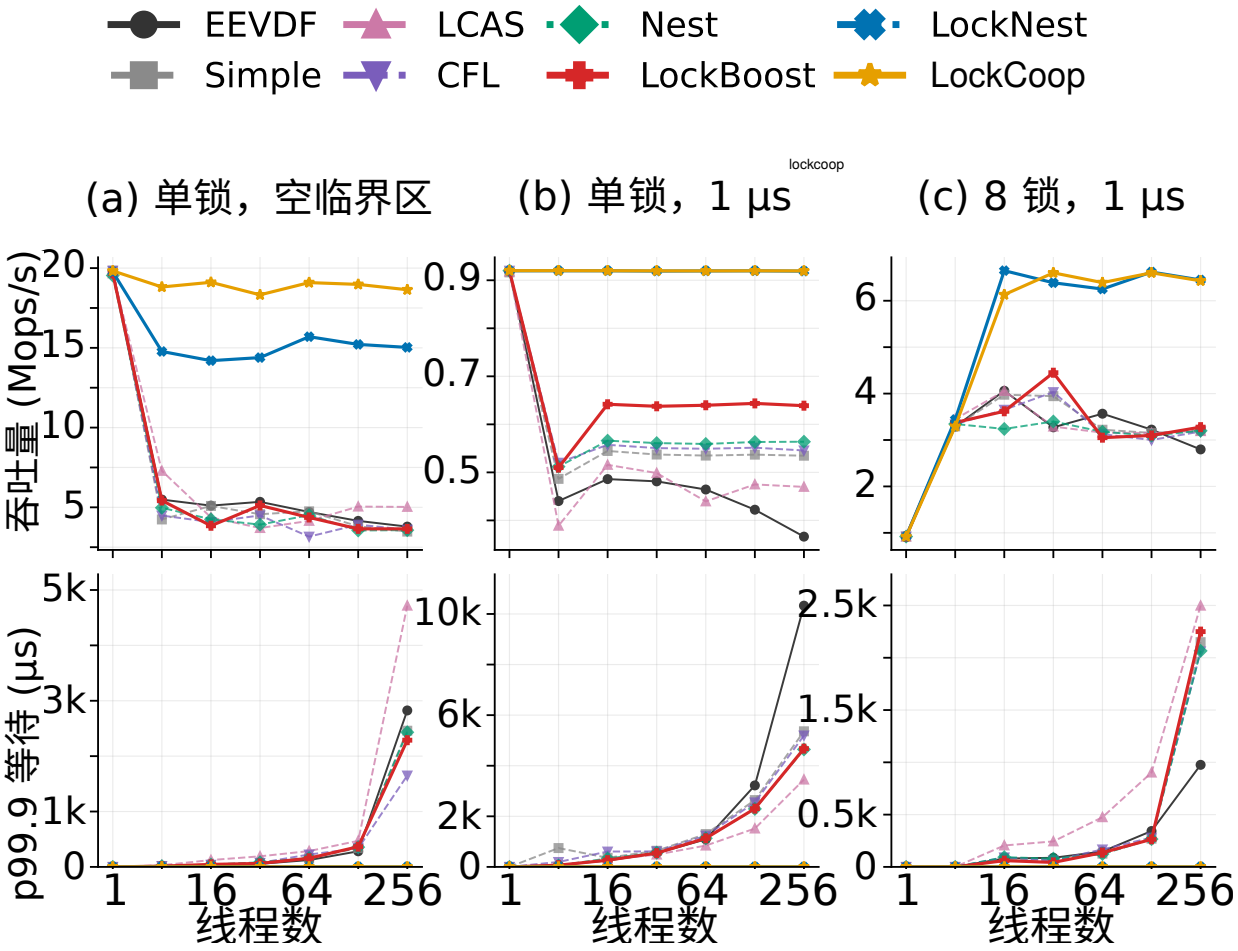


图 5.5: Mutex 动作隔离。上：吞吐；下：一把热点 mutex、八把热点 mutex 与空临界区的 p99.9 锁等待。

结果清楚地区分了两个调度机会。256 线程时，空临界区单锁场景下 LockCoop 达到 18.64 Mops/s，是 `scx_simple` 的 5.36 倍，同时把 p99.9 等待保持在测量下界；八把独立 mutex 时，LockNest 达到 EEVDF 的 2.30 倍。因此，一条串行依赖链突出持锁者扩展的作用，多条独立依赖链则保留逐锁放置收益。

线程域与进程域。 线程私有内核在多线程间共享一个静态 pthread mutex；进程共享内核在共享映射中使用 `PTHREAD_PROCESS_SHARED` 自适应 mutex。每个临界区递增一个计数器。我们在 x86 上

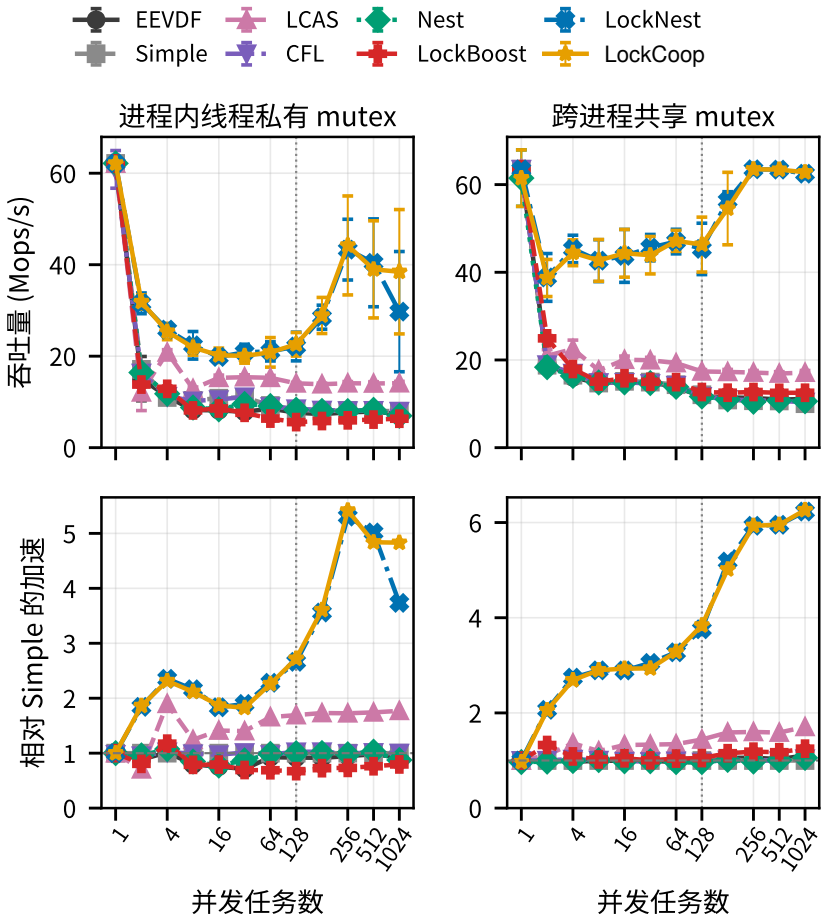


图 5.6: 受控 mutex 交接。上：吞吐；下：相对 Simple 归一化吞吐。虚线标记 128 个在线 CPU。

不绑定地 sweep 1–1024 个任务；图 5.6报告十次运行均值。

两种 mutex 作用域在超售下都保持收益。1024 线程时，LockCoop 是 LockNest 的 1.29 倍；1024 进程时，它是 `scx_simple` 的 6.26 倍。独立 LockBoost 达到 `scx_simple` 的 1.25 倍，说明持锁者扩展确有帮助，但这些短临界区仍主要受益于后继放置。

调度器计数区分有用进展与无差别扩展。进程共享内核中，24.3% 的采样 LockBoost 持锁者处于可运行状态，而线程私有内核中为 0.64%。组合保留 89.1% 的局部性命中率，同时把扩展降到每百万次操作 13.34 次：稳定放置使需要 LockBoost 推进的延迟持锁者变少。

5.5.3 策略刻画

图 5.7 把资源引导放置分成两部分：每个任务执行一次首次放置，以及把每个锚点稳定地投影到 D_r，实验不施加外部 CPU 或内存绑定。ARM 分别开关两个动作，x86 则在第二种体系结构上评估完整策略。所配置的数据设备在 ARM 上解析到节点 0，在 x86 上解析到节点 1。

ARM 上 256 线程时，完整策略达到首次观测锚定的 2.69 倍，并将多次运行之间的变异系数从 18.19% 降到 4.41%。首次放置或锚点投影单独启用时都无法达到这种稳定性：前者为后续锚点播种资源域，后者把之后的同锁执行限制在该域内。x86 上，锚点投影贡献了主要收益；加入首次放置后，整个并发区间内与仅投影相差不超过 3.4%，并在 256 线程达到 6,550.1 TPS。因此，同一策略无需平台专属锚点规则即可保持 x86 的扩展趋势。

实验性的等待者压力阈值 $Q = 8$–128 会减少逐锁放置，但没有一种配置在正式重复中始终超

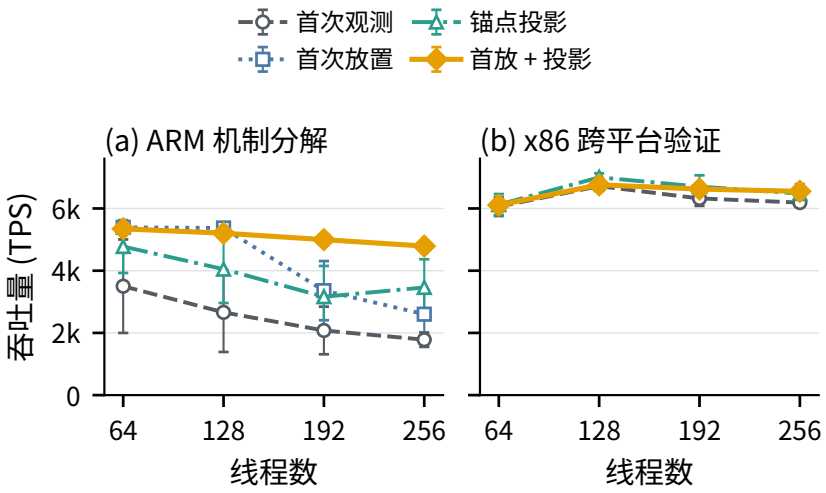


图 5.7: 全机不绑定条件下的资源引导 LockNest。ARM 分解首次放置与锚点投影，x86 评估完整策略；误差线为 95% 置信区间。

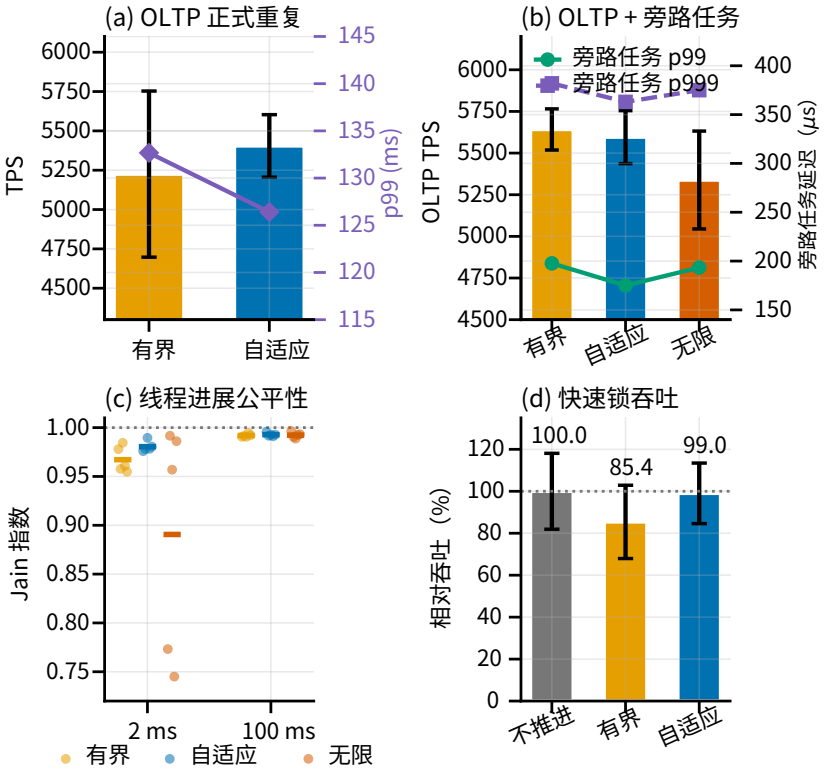


图 5.8: 有界持锁者推进控制：重复 OLTP、混合负载中受干扰任务的延迟、临界区公平性与短临界区保护实验。

过直接使用所有有效锚点。1–64 热点锁压力测试无 map 失败或零进展线程，因此选定策略在常见唤醒路径上不计算队列长度。

图 5.8 把自适应控制与静态有界、无界扩展分开。自适应控制在重复 OLTP 中优于静态有界控制，并在同时运行 cyclictest 时把其 p99/p999 延迟限制在 175/363 μs。2-ms 临界区下其 Jain 指数为 0.9805，而无界消融为 0.8906。在零工作快速锁上，自适应控制与不启用扩展相差不超过 0.99%，而静态有界控制损失 14.6%。因此，在线证据决定有限扩展何时有用，而非简单地加长时间片。

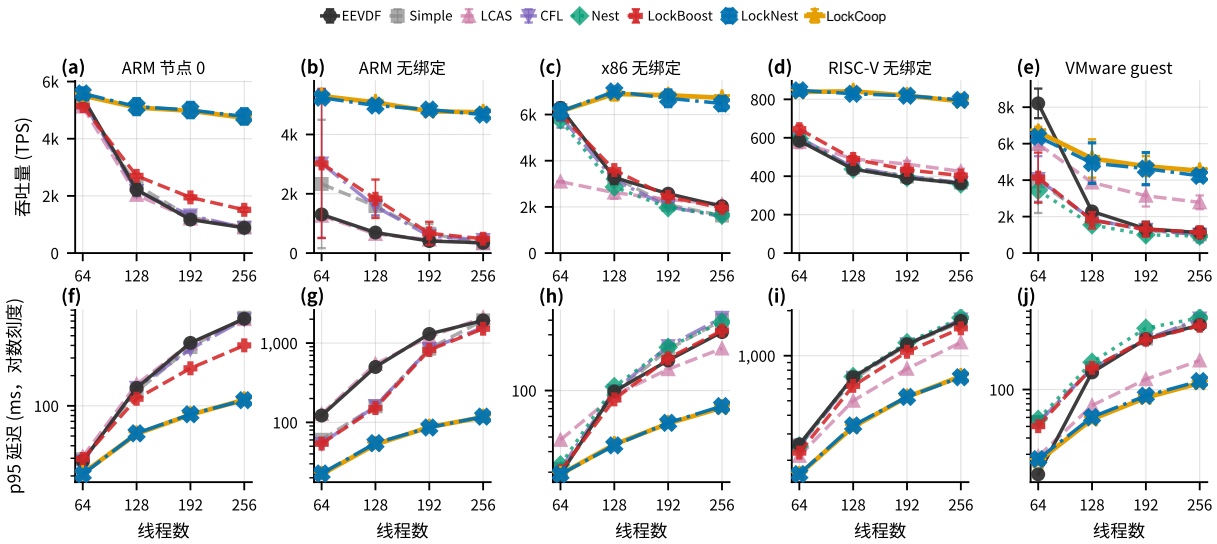


图 5.9: MariaDB/Sysbench OLTP 吞吐（上）与 p95 延迟（下）。

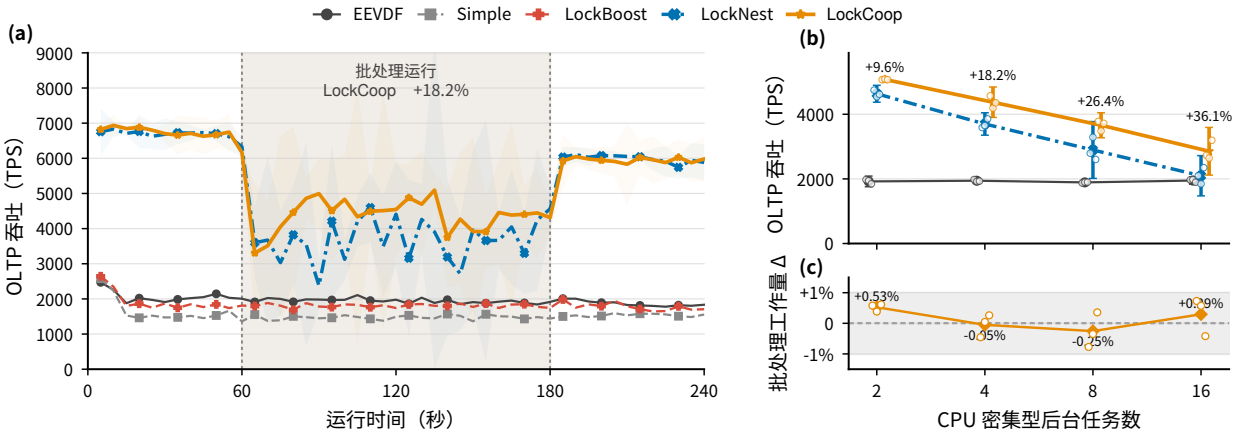


图 5.10: CPU 干扰下的 x86 不绑定 OLTP。(a) 四个后台任务运行时的吞吐时间线；(b) 2–16 个后台任务下干扰阶段的 OLTP 吞吐，空心点为单次结果，误差棒为十次运行的 95% 置信区间；(c) LockCoop 相对 LockNest 的逐次批处理工作量，阴影标出 ±1% 区间。

在一次持续 20 秒的 MariaDB 诊断实验中，只有 1.9% 的持锁者调度延迟样本对应时间片扩展。几乎所有扩展都在解锁时撤销。在另一项快速锁诊断实验中，自适应控制检测到持锁者已能快速推进后，不再发放扩展。

参数研究覆盖 $T = 2$–16、250–2,000-μs 扩展、1–10-ms 上限与 5–20% 任务预算。没有任何单一合成最优点决定选定配置：重复 OLTP、混合负载中受干扰任务的延迟、长临界区公平性与短临界区保护共同划定运行区域。控制器随后在这些边界内在线选择动作。

5.5.4 端到端性能与机制证据

图 5.9 在五个执行范围内展示一致的高吞吐结果。256 线程时，`scx_lockcoop` 相对 `scx_simple` 提升 2.15–13.9 倍吞吐、降低 56.6–94.0% 的 p95。代表性的 ARM64 节点 0 结果为 5.30 倍吞吐与 84.8% 的 p95 降幅，x86 不绑定则达到 4.14 倍与 82.2%。在 64 核 RISC-V 上，`scx_lockcoop` 达到 2.15 倍吞吐并将 p95 降低 56.6%。不绑定实验仍保持高吞吐，32-vCPU VMware guest 达到 `scx_simple` 的 4.42 倍。

在相同节点 0 测试点上，scx_lockcoop 分别比 ARM64 和 x86 上更强的 LCAS/CFL 结果高 5.16 倍和 2.61 倍。聚合竞争引导在部分场景中有效，但缺少逐锁交接身份时无法进入相同的高吞吐区间。

ARM64 节点 0 的 64 线程下，各机制因竞争较弱而收敛；随着等待任务数量增长，LockCoop 的优势才逐渐出现。尽管三种裸机 ISA 的核心、SMT 与拓扑组织不同，同样的定性转变都会出现。

图 5.10 在受控 CPU 干扰下评估持锁者推进何时能够在同锁放置之上产生增量。

纯 OLTP 由局部性主导：256 线程时，LockNest 与 LockCoop 在 ARM64 节点 0 上相差 1% 以内、在 x86 节点 0 上相差 0.8% 以内，而独立 LockBoost 仍位于低吞吐区间。这一消融表明，此时放置已经捕获几乎全部收益。为在同一条端到端路径上暴露持锁者延迟，我们保持 256 线程 x86 不绑定前台不变，并引入 2–16 个固定时长的 CPU 密集型后台任务。每次 240 秒运行依次包含 60 秒纯前台、120 秒并发批处理和 60 秒恢复阶段。

LockCoop 在干扰区间之外与 LockNest 基本一致，并在干扰结束后恢复。干扰期间，其平均吞吐优势随后台任务从 2 个增加到 16 个，由 9.6% 增长到 36.1%，其中四个后台任务时为 18.2%。两个调度器下的批处理工作量差异不超过 0.53%，后台任务的 CPU 时间也保持一致。独立 LockBoost 曲线仍位于低吞吐区间；此时有界持锁者推进通过保护已被 LockNest 局部化的依赖链产生增量。

接下来，我们评估 CPU 竞争下的结果是否只存在于 MariaDB。图 5.11 在相同的后台计算负载下比较四种存储系统。

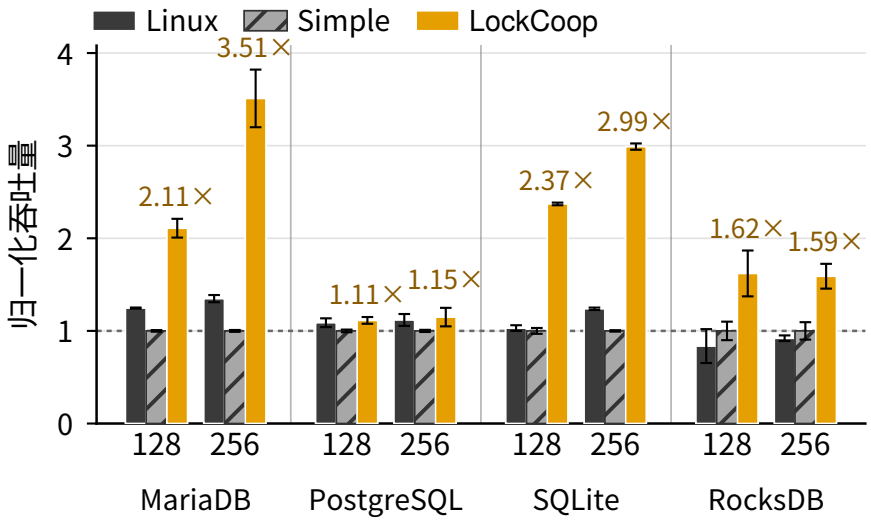


图 5.11: 固定 CPU 竞争下的应用吞吐量，相对 Simple 归一化（均值 ± 标准差，三轮）。

在 128 和 256 个工作线程下，LockCoop 在四种应用中的吞吐量均高于 Simple，达到其 1.11–3.51 倍。

图 5.12 评估后继放置如何改变 MariaDB 的处理器执行行为。256 线程时，LockCoop 把 IPC 从 0.63 提高到 0.99、把迁移减少 62%，同时保持相近的数据库忙碌 CPU 数；LockNest 独立达到 IPC 1.00。LockCoop 的缓存未命中率保持在 6–7%，也低于通用拓扑 Nest 的 20.3%。结合 2×2 分解（图 5.7），相近的忙碌 CPU 数量排除了额外 CPU 容量。相对 Simple，更低的迁移率、更高的 IPC 和更少的缓存未命中表明，LockCoop 有效复用了为每把锁记录的 CPU 历史。

为确定高竞争区间中 MariaDB 的 CPU 时间主要消耗在哪里，图 5.13 展示 x86 全机不绑定的 256-client 测试点上 EEVDF 与 LockCoop 的 MariaDB 进程级 CPU 采样。

在这组 MariaDB 负载中，吞吐量提高的同时，每笔事务的 CPU 开销也有所下降。在 EEVDF 下，MariaDB 约一半的采样周期消耗在内核队列自旋锁慢路径，主要来自 futex 等待和唤醒。LockCoop

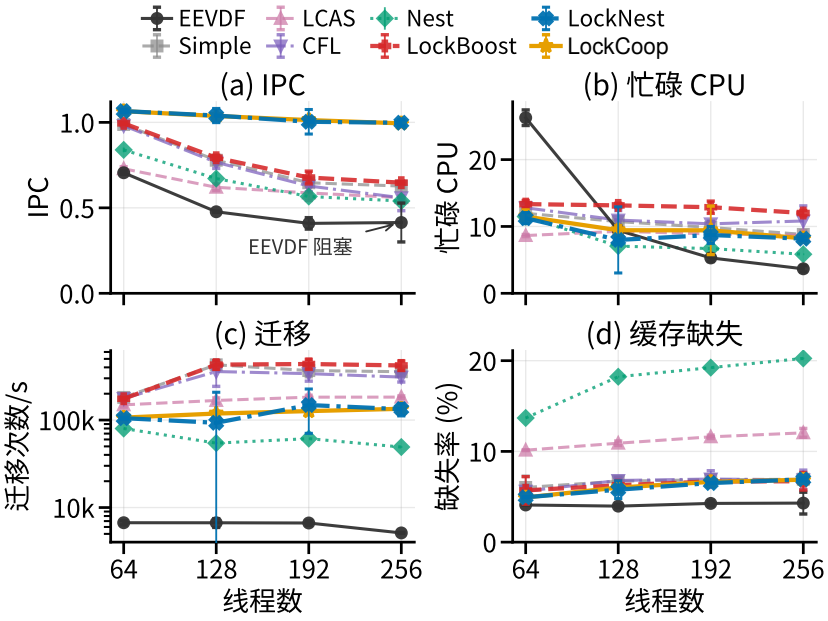


图 5.12: x86 MariaDB 进程计数器：IPC、忙碌逻辑 CPU、迁移数与 L1/L3 缓存未命中率。

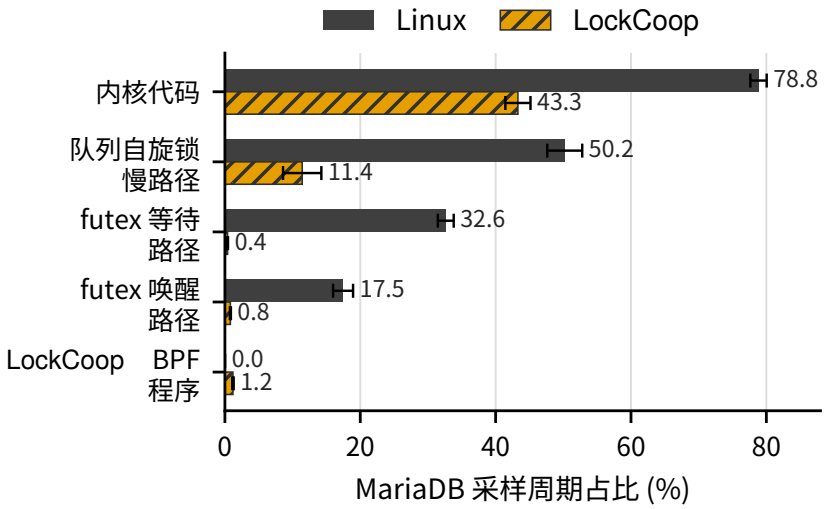


图 5.13: EEVDF 与 LockCoop 下 MariaDB 在 256 clients 时的 CPU 采样（三轮均值 ± 标准差）。各柱独立统计不同调用路径的周期占比，路径之间可能重叠。

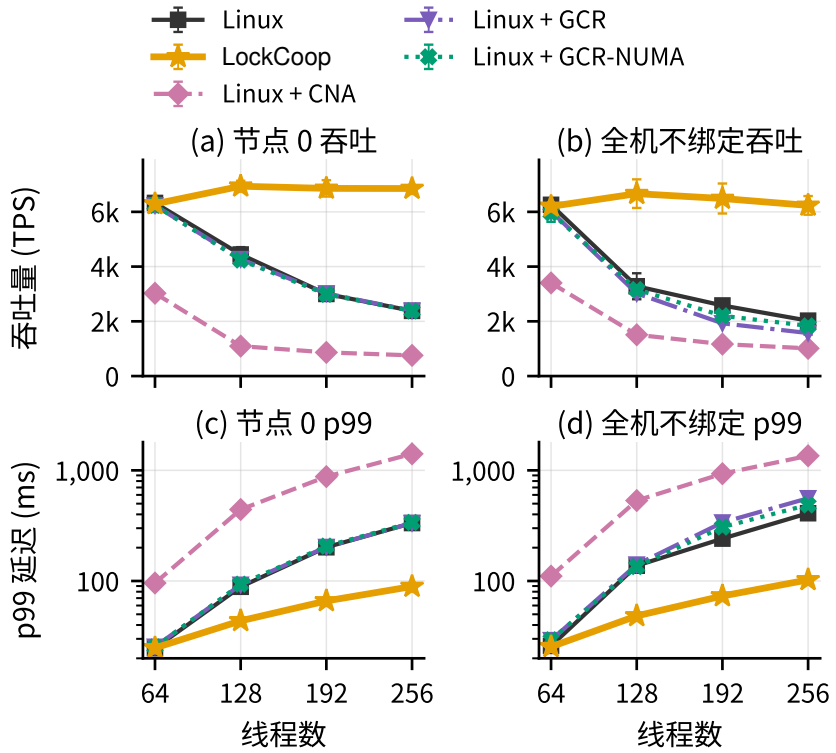


图 5.14: 使用 native pthread mutex 的 x86 OLTP 吞吐与 p99；误差线为 95% 置信区间。

将这一占比降至 11.4%。六组配对计数器实验也显示，每笔完成事务所需的 CPU 周期更少。

5.5.5 跨层对比与快速路径成本

图 5.14 比较 LockCoop 与 CNA 的 NUMA 感知队列交接 [56]、GCR 的 active-set 准入及其 NUMA 扩展 [52]、以及 Linux AutoNUMA 放置 [78]。AutoNUMA 使用上游内核实现。在 EEVDF 下，我们的 pthread 兼容 CNA 移植实现已发表的本地 successor 与 secondary queue 交接策略，并采用自旋 2,048 次后 park 的 pthread 集成方式；GCR 与 GCR-NUMA 移植实现已发表的 active-set 准入策略，active 上限为 4、join 阈值为 2、批准周期为 16,384 次获取。所有 pthread 后端均通过 mutex 与条件变量语义测试。配套验证运行开启机制路径计数，确认 CNA 本地/远程交接、GCR 主动/被动准入及 GCR-NUMA 优选节点轮转实际执行；正式性能运行关闭这些计数器。LockCoop 保留 native pthread mutex，所有用例使用相同的 x86 放置范围与频率策略。

256 线程时，LockCoop 在节点 0 上达到 GCR 的 2.87 倍，在不绑定范围内达到 native pthread 的 3.08 倍；其节点 0 p99 为 89.16 ms，而 GCR 为 333.92 ms。GCR-NUMA 仅在不绑定时比 GCR 提高 1.17 倍，CNA 在两个范围内都低于 native pthread，分别为其 0.32 倍和 0.50 倍。在所测配置中，仅采用 NUMA 感知的锁获取顺序，并未达到 LockCoop 的收益。切换 AutoNUMA 使吞吐最多变化 4.72%、p99 最多变化 3.54%；LockCoop 在两种设置下都保持高吞吐。这些机制分别作用于锁队列顺序、准入并发、页面放置和逐锁 CPU 调度。

固定速率下，独立的定时器任务在完整配置下记录 59.98/74.65 μs 的 p99/p99.9 延迟。

端到端观测开销。 为隔离观测与调度动作，我们保留全部锁事件采集和状态维护，同时关闭 LockBoost 与 LockNest，并采用与 `scx_simple` 相同的虚拟时间策略。在十次 60 秒实验中，仅观测配置在 64 客户端下相对 `scx_simple` 的吞吐变化为 $+1.1\% \pm 1.2\%$（配对 95% 置信区间），表明其

表 5.3: 快速路径压力结果。固定速率运行目标 0.50 百万次 futex 操作/秒；不限速运行报告持续速率。

Profile / key 集	速率 (Mops/s)	诊断
Scheduler-only / fixed	0.50	0.36 CPU-eq.
Full / fixed	0.50	1.16 CPU-eq.
Full / 单 key	4.41	0 失败/停滞
Full / 8,192 key	6.77	0 失败/停滞

在非过载区间没有可测量的端到端性能退化。在 256 客户端下，其吞吐提高 $19.4\% \pm 1.2\%$。由于该过载运行点对微小的执行时序扰动较为敏感，我们仅将这一正向变化视为无退化结果，而不将其解释为观测机制带来的收益。完整 LockCoop 的提升无法仅由观测过程解释。表 5.3 进一步在固定事件速率下隔离了观测路径的处理成本。

5.5.6 运行区间与可移植性

在纯 MariaDB OLTP 和临界区最短的 mutex 测试中，后继放置贡献了几乎全部吞吐量收益。在 CPU 干扰和进程共享 mutex 测试中，持锁者时间片扩展也能带来收益。LockCoop 分别检查每种动作所需的证据，而不是为整个负载选定一种模式。

由于 D_r 不必对应一个 socket，且 a_K 记录域内 CPU，同一规则可以在节点受限时利用私有缓存与 LLC 局部性、在不绑定时利用 NUMA 局部性、并在 guest 内利用虚拟 CPU 局部性。不同平台改变的是拓扑与软件版本，而不是这条调度规则。

5.6 相关工作

锁感知调度与同步。 CA-Scheduler 使用锁状态调整执行，LCAS 把锁密集任务路由到专门核心，CFL 把锁占用作为资源调度 [57, 58, 67]。Scheduler-conscious synchronization、SCL、SynCord、I-Spinlock 与 FlexGuard 通过锁、运行时或 vCPU 状态协调调度；RSEQ 允许经过插桩的用户态代码请求有界的临界区扩展 [60, 61, 66, 68–70]。面向请求的 I/O 优先级机制也会传递用户锁持有者以继承 I/O 关键性 [65]。LockCoop 从现有路径推导每把锁的等待者与持锁者关系，并让锁身份贯穿持锁者服务与后继放置。数据库调度器与锁位点控制减少到达热点锁的工作，或选择专门锁实现 [73, 79–82]；这些决策与两个调度动作互补。

锁算法与竞争控制。 MCS、Cohort、HMCS、shuffling、CNA、CST 与 RON 在锁协议内部组织等待者或锁转移，以控制竞争和局部性 [53–56, 74–76, 83]。TCLOCKS 把临界区委托到特定核心执行，使被保护数据保持就近 [77]。GCR 与 HTLL 调节活跃竞争者，数据库控制则限制进入热点锁的工作 [52, 73, 79, 80, 84]。LockCoop 保持准入与锁语义不变，调度被观测的持锁者与被唤醒任务。这些方法说明锁交接局部性能够减少一致性流量；LockNest 在锁外利用逐锁身份应用这一原则。

NUMA 与缓存感知调度。 Nest 是最接近的调度器层放置 baseline；它使用聚合复用信号，而不是某把锁自身的执行历史 [59]。缓存感知聚类同样使用聚合信号，NUMA 放置与 AutoNUMA 则把任务与内存共置 [78, 85, 86]。近期 Linux 工作把共享地址空间的线程聚集到偏好的 LLC [87]；其它调度器使用应用、交互或虚拟拓扑信号 [88–91]。LockCoop 则识别每个同步信号对应的锁，并把该锁的 CPU 历史从私有缓存投影到 NUMA 域。

可编程调度器。 ghOSt 把策略委托给用户态，相关系统则为专用运行时提供机制 [92–98]。

sched_ext 提供本文使用的内核 BPF 钩子 [63]；本文贡献是逐锁关系与两个有界动作，而不是调度器可编程性本身。

5.7 结论

LockCoop 记录每把锁的等待者、持有者与执行位置，在解锁前推进被延迟的持锁者，并在唤醒后保持后继任务的局部性。两个动作都能独立发挥作用，并在 CPU 干扰下相互补充；缺少支持证据时，任务仍使用普通调度。

第六章　FlowCausal：工具返回关系与会话调度

6.1　引言

代码智能体正在把代码库维护变成主机上的系统负载。本文把智能体运行时发起的一次命令、构建、测试或代码库操作称为一次“工具调用”，把一个智能体连续发起的多次工具调用称为一个“会话”。一个会话会在容器或微虚拟机中启动命令解释器、语言运行时、编译器、测试框架、软件包管理器和辅助服务。在同一主机集中运行多个会话能够提高利用率，却也使完成时间取决于主机如何在多棵不断变化的进程树之间分配处理器时间。本文把当前工具调用直接启动的进程称为“工具进程”。在实验中，每个重放的软件工程任务占用一个会话，因此“工作流完成时间”就是该会话从启动到结束的时间。

这里的调度目标不同于单个任务完成。用户看到的是当前工具调用何时返回以及智能体何时能够继续，Linux 看到的则是可运行进程。阻塞在 `wait` 中的工具进程可能等待测试进程，而这些进程又与先前工具调用遗留的计算进程竞争处理器。两者可以具有相同的控制组（cgroup）和优先级。因此，按任务公平分配处理器时间仍可能推迟当前工具调用必须等待的进程。图 6.1 给出了这种差异：在一条公开 DVC 记录中，52 个能够解除当前等待的进程与先前工具调用产生的计算同时运行。

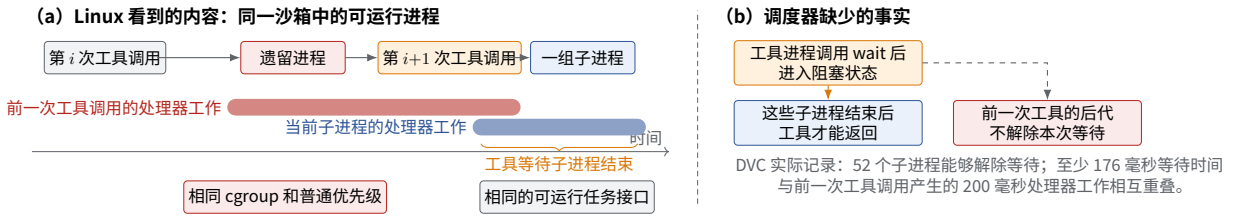


图 6.1: 调度器缺少的信息。Linux 能看到可运行进程和控制组，却不知道哪些后代进程结束后会解除当前阻塞等待。FlowCausal 根据内核事件重建这一关系。

现有主流机制提供了相邻能力。Linux 控制组在容器之间分配处理器时间；AgentCgroup 将资源核算与智能体工具调用对应 [99]；工作流调度器处理显式作业或依赖图 [100–102]；可编程调度框架提供策略接口 [103, 104]。这些机制分别工作在任务、控制组、工具或者已声明工作流的边界上。

但它们都没有告诉处理器调度器：当前有哪些存活子进程必须结束，一个已经阻塞的工具调用才能返回。父子关系也不足以回答这个问题，因为父进程可以拥有不等待的异步子进程，等待函数也可能立即返回。在用户态周期性扫描 `/proc` 不仅会晚于短进程，还无法把进入等待与紧随其后的调度状态作为同一个事实处理。本文把已阻塞工具进程与能够解除其等待的存活子进程之间的关系称为“工具返回关系”。精确问题因而是：如何确认当前工具调用的这条关系，在条件变化后及时删除，并且不破坏多个会话之间的处理器时间分配。

本文提出 FlowCausal，并将两个决策分开。外层按照加权虚拟运行时间在会话之间分配处理器时间，避免进程较多的会话仅凭任务数量获得更多资源；内层允许工具返回关系中的一个进程在会话内普通顺序之前获得一个短时间片，所用时间仍计入所属会话。小型 C 程序只登记会话和工具调用编号；eBPF 在进程创建时传递这些编号，根据内核事件确认阻塞等待，并在等待、进程或

工具调用变化后删除关系。在线系统不轮询进程，也不依赖命令名称。

本文使用由 42 个公开代码库中 50 个智能体任务构成的 HarnessReplayBench 评价系统。两组四会话实验对每种方法交错运行五轮，并与 Linux、使用同权控制组的 Linux 和上游 `scx_simple` 比较。FlowCausal 相对 Linux 将平均工作流完成时间缩短 4.2%–5.2%，相对 `scx_simple` 缩短 27.9%–31.9%，并相对 Linux 将最后一个工作流的完成时间缩短 1.7%–2.6%。使用同一份代码的消融实验表明，当构建工作决定工具返回时，依据工具返回关系安排进程次序可进一步缩短 4.88%。系统识别了工具返回关系中 99.04% 的进程，内核事件零丢失。

本文贡献如下：

- **会话级处理器调度**。把智能体会话作为长期处理器时间的分配单位，把会话中的进程作为短期执行单位。两级设计避免进程数量决定会话获得的处理器时间，同时保留普通加权调度。
- **工具返回关系**。把运行时提供的工具调用编号与内核观察到的进程来源和阻塞事件结合起来，形成能够确认并及时删除的关系；相关进程提前使用的时间完整计入所属会话。
- **可重复的实验依据**。把 50 个公开智能体任务整理为可组合调度测试，通过完整系统比较、使用同一实现的消融实验、全部任务覆盖和异常恢复测试，分别验证会话调度、工具返回关系以及实现成本。

6.2 背景

6.2.1 共享主机上的代码智能体执行

代码智能体运行时会把每次工具调用转换为一个或多个操作系统进程。命令解释器可能启动 Python 入口，测试框架再创建多个进程并调用编译器。沙箱提供隔离，却不会替代主机处理器调度器。在混合运行的机器上，多个会话中的可运行后代最终进入同一组主机运行队列。

运行时和内核提供互补的视角。运行时能够在会话和工具调用开始时分配编号，却无法预先知道完整进程树；Linux 能观察创建、执行、睡眠、唤醒和退出，却不知道这些事件属于哪次工具调用。面向智能体负载的主机调度器因而需要把运行时提供的编号与内核观察到的进程状态结合起来。

容器和微虚拟机保留了这一主机调度问题，同时使它更容易复现。Firecracker 提供隔离边界 [105]；AgentCgroup 将资源核算与工具调用对应 [99]；Crab 根据智能体执行轮次安排检查点保存和恢复 [106]。这些工作说明系统软件可以取得会话和工具调用编号。

6.2.2 Linux 的任务与 cgroup 调度

Linux 公平调度按照实体权重和虚拟运行时间分配处理器服务。cgroup 调度能够汇总多个进程并给整个组设置权重，处理器亲和性则限定进程可以在哪些处理器上运行。`sched_ext` 进一步允许 BPF 策略在不修改核心调度器的情况下选择队列、时间片和处理器 [104]。

这些机制能够使用可运行状态、已消耗的处理器时间、权重、处理器范围和控制组归属。它们说明一个实体已经获得多少服务以及可以在哪里运行，却不会给复用沙箱中的某个进程标明所属工具调用。同一个控制组可以同时包含当前工具调用、先前工具调用留下的工作进程以及长期辅助服务。

6.2.3 工作流与应用接口

集群工作流调度器在运行前接收作业、依赖图或资源请求 [100–102]。应用信息参与内核策略的研究也表明，单个进程之上的信息能够改进同步处理 [107]。但智能体工具的进程树在调度后才

逐步形成，部分进程会在调用结束后继续存活，而且父进程可能只等待其中一部分。

因此，本文只用会话和工具调用编号标明哪些工作属于同一次调用，具体的进程关系由内核事件产生；资源权重和处理器范围仍由普通系统管理接口控制。

6.3 动机

6.3.1 调度器缺少的具体信息

仅靠父子关系、等待函数或控制组归属，都不能确定工具返回所需的进程。这里存在三种必须排除的歧义。

创建子进程不等于等待子进程。一个进程可以拥有许多仍存活的子进程，却不等待其中任何一个。若把全部后代提前执行，异步服务和先前工作也会得到同样处理。

进入等待函数不一定真的阻塞。程序会使用 `WNOHANG` 查询状态，回收已经退出的子进程，或者进行重试。只观察等待函数入口会产生错误关系。

一个控制组可以包含不同工具调用产生的进程。复用沙箱有利于保持缓存和辅助服务，但也会保留不再决定当前工具调用何时返回的进程。控制组成员关系无法区分这些进程。

这三种歧义共同限定了工具返回关系：它属于某次工具调用，只在父进程确实阻塞后成立，指出哪些存活子进程可以解除等待，并在条件变化后立即失效。

6.3.2 公开代码库任务中的证据

我们在查看任何 FlowCausal 结果之前检索基线调度记录。对每次工具调用，分析程序寻找三个条件：实际阻塞等待、能够解除该等待的仍存活子进程，以及先前工具调用产生且同时消耗处理器时间的后代。DVC 任务 `iterative__dvc-1494` 的第 172 次工具调用包含全部条件。当前等待的实测下界为 176 毫秒，先前工具调用产生的进程至少重叠运行 200 毫秒，能够解除当前等待的子进程共有 52 个。这些数值是下界，因为只统计调度事件能够确认的区间。

该现象准确说明了调度机会：当能够解除等待的子进程仍在争用处理器时，让其中一个进程更早运行，可以使工具调用及其会话更早完成；当子进程已经接近结束时，可调整的空间自然消失。这为实验提供了对照条件，而不是假定每次创建子进程都应带来收益。

6.3.3 问题定义与设计要求

本文研究的问题是：“主机调度器能否重建决定代码智能体工具调用何时返回的存活进程关系，利用它缩短会话完成时间，同时准确核算并发会话消耗的处理器时间?”这一问题把宽泛的混合运行场景收窄到工具执行期间可以获得的信息和可以采取的调度动作。

问题分析给出五项要求。**依据行为而非名称：**进程分类应使用创建和等待事件，不能依赖命令字符串。**确认实际阻塞：**只有等待进程真的睡眠后，关系才能影响调度。**保持会话间公平：**普通任务必须按会话调度，提前执行消耗的时间必须计入所属会话。**限制过期状态和调度调整：**进程退出、进程号复用、工具切换或结束消息遗漏都不能留下长期优先级。**运行期间只使用 C/eBPF：**用户态只报告低频事实，持续变化的关系和在线调度决定由内核事件驱动。

表 6.1: HarnessReplayBench 1.0 的任务库和测试组合。

项目	数量	在实验中的作用
来源任务	50	公开代码库任务和工具记录
代码库	42	覆盖不同构建与测试环境
工具命令	2,940	真实检索、构建、测试与服务命令
可运行任务	50	能够稳定结束并核对返回状态
单任务组合	50	独立重放和特征分析
多任务组合	34	受控竞争和规模变化

6.4 HarnessReplayBench

HarnessReplayBench 用于在调度器变化时保持工具执行不变。它是在公认的公开软件工程智能体任务之上新建的系统基准测试，并不替代补丁正确率排行榜。它的作用是把主机侧进程执行转化为可组合、可重复的测试输入。本文把一个来源任务及其命令保存为一份“重放记录”；把选择一份或多份记录、指定会话和控制组、启动时间以及可用处理器的清单称为“测试组合”。这样，任务本身与多个任务共享机器的方式彼此独立。

6.4.1 来源任务与覆盖范围

任务库包含 50 个来源任务、42 个代码库和 2,940 条工具命令。其中 46 个任务来自 SWE-rebench，3 个来自 SWE-bench Lite，1 个来自公开的 SWE-agent 执行记录。原始执行包括 12 个 OpenHands 任务、8 个 SWE-agent 任务和 30 个 mini-SWE-agent 任务。这些来源提供公开代码库版本、可执行容器和真实工具序列，而不是人工编写的处理器循环。

50 个任务都能在版本固定的软件环境中结束，返回状态符合预期，内核事件丢失为零。对每个任务，我们还在命令和可用处理器相同的条件下，记录其在 `scx_simple` 下的进程创建、退出和处理器执行情况。三份导入记录只修复运行环境，不改变计算内容：Dask 使用适合标准输入程序的多进程启动方式，Sanic 保存工作进程输出，DVC 从同一任务和镜像重新建立工作区。三者现在都能正常结束，返回状态正确且没有丢失内核事件。

这些命令覆盖代码库查看、一般命令、编辑、测试、软件包管理和构建，并可能继续启动编译器、工作进程池或本地服务。经过验证的单任务执行从 39 个进程到 115,080 个进程不等，耗时从不足一秒到超过一千秒。因此，测试组合可以根据进程结构和执行长度选择任务，而不需要修改任务本身。

6.4.2 重放记录与执行条件

每份重放记录包括容器镜像、代码库状态、工作目录、环境、命令、启动时间、预期返回类型、超时时间和清理方法。命令仍在原来的软件环境中执行，编译器和测试框架不会被休眠或空循环代替。运行程序保存四类记录：运行时报告的工具调用起止信息、C/eBPF 记录的进程创建和调度事件、命令返回状态、资源使用情况。只有每条命令的返回状态属于重放记录允许的类型、清理成功且内核事件丢失数为零时，该次运行结果才会进入统计。

发布包提供 50 个单任务组合和 34 个多任务组合。清单明确列出任务、会话和控制组，指定同时启动或错峰启动，并固定允许使用的处理器。任务库与测试组合相互独立，后续研究可以增加新组合而无须改写任务记录。

导入程序先把公开任务绑定到代码库版本和镜像，再整理每条命令的工作目录、环境、执行方式和预期返回类型。随后在 `scx_simple` 下单独执行任务，确认命令序列能够结束并记录进程与处理器事件。测试组合再把不变的任务编号与启动时间和处理器设置结合。单任务记录产生的离线标注只用于实验结束后的分析；FlowCausal 运行时只能使用实际部署中也能取得的运行时消息和内核事件。

6.4.3 避免依据候选方案选择任务

主要实验使用两个互补的四会话组合。“混合工作流组合”同时运行 Dask、DVC、Numba 和 SQLFluff，用来检查按会话调度在并发启动时的作用。“工具返回组合”完全依据 `scx_simple` 的记录选择，选择条件正是第 6.3节给出的实际阻塞、能够解除等待的存活子进程，以及先前工具调用产生的进程同时使用处理器。选择过程不读取任何 FlowCausal 耗时结果。

另外两个组合用来判断机制是否因所述原因生效。“构建等待组合”包含仍在继续运行的构建进程；“短子进程组合”中，能够解除等待的子进程只运行 7.75 毫秒且只有一个，理论上没有明显调度空间。若一个机制在这里仍得到相同收益，更可能是对整个组合施加了优先级差异，而不是准确利用工具返回关系。

6.4.4 评价指标

本文使用两项用户可直接感知的完成时间指标。**最后一个工作流完成时间**也常称为总完成时间，从测试组合启动开始，直到最后一个会话结束。**平均工作流完成时间**是组合中所有会话耗时的平均值，能够反映部分会话是否更早完成。两项指标都是越低越好。本文还分别报告工具返回关系中的进程识别率、事件丢失、处理器开销、内存使用和状态恢复测试，不把完成时间、关系识别和实现成本合并为一个指标。

6.4.5 重放一致性与可复现性

调度器基准不能只比较命令名称是否相同。HarnessReplayBench 从三个层面核对条件。执行一致性要求可执行命令、工作目录、解释器环境、代码库状态和容器镜像一致；进程一致性核对进程创建与执行链、最大存活进程数量和命令结果；组合一致性固定会话归属、启动方式、允许的处理器和清理方法。这样，完成时间变化才能归因于处理器调度，而不是程序走了不同路径。

不同公开来源记录环境的详细程度不同。重新采集的任务恢复记录中的解释器、环境变量和命令解释器设置；导入记录缺少这些字段时，使用配套固定版本镜像中的测试环境。该规则写入重放记录并对所有调度器保持相同。来源记录中允许失败的探索命令可以返回非零值，但任何意外失败都会使该次运行结果失效，调度器不能依靠少执行命令或提前失败表现得更快。

全部容器到达统一的就绪点后才开始计时。成功结果保留事件、命令结果、调度器计数和摘要；失败结果保留完整工作区和输出。版本 1.0 的重放记录保持不变，测试组合另行指定任务、启动时间和处理器。增加新组合不会改变已有结果。

6.5 系统总览与设计依据

图 6.2 展示信息如何从沙箱运行时进入处理器选择。系统采用两项划分：在识别方面，运行时提供稳定的会话和工具调用编号，内核提供不断变化的进程与等待事件；在调度方面，会话之间的

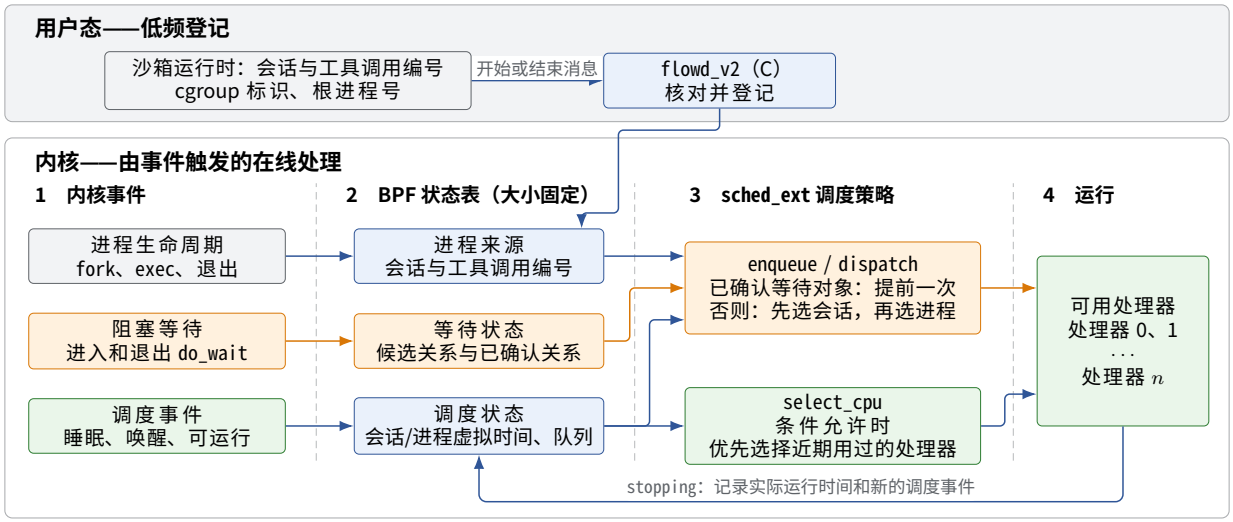


图 6.2: FlowCausal 的系统结构和反馈过程。C 程序登记会话、工具调用和 cgroup 标识。内核事件更新大小固定的 BPF 状态表；`sched_ext` 回调根据这些状态选择进程和可用处理器；随后记录实际运行时间，供下一次调度使用。

处理器时间分配与会话内部可运行进程的短期次序分别处理。这样可以让工具调用更早返回，又不会把一次短暂关系变成永久优先级。

运行时登记。每个会话位于独立 cgroup，同一工作流的进程由此归为一组。`flowd_v2` 核对 cgroup 标识、根进程号及会话和工具调用的起止消息，再更新 BPF 映射。cgroup 确定进程所属会话，工具调用编号与实际阻塞事件进一步确定当前等待关系。

内核观测。eBPF 让登记编号随 `fork` 与 `exec` 传递。进入 `do_wait` 时只建立候选关系，随后观察到进程睡眠才确认阻塞；生命周期事件负责删除数量受限的状态。

处理器调度。普通路径按照加权虚拟运行时间选择会话，再选择会话内进程。已经确认的工具返回关系可以让一个进程提前获得短时间片，其实际运行时间仍计入所属会话。与当前等待竞争的先前工作也可以短暂延后。这两种动作都不改变会话权重。

处理器选择。进程运行后 250 毫秒内，调度器优先尝试它近期使用且仍可用的处理器；负载、亲和性和可用状态仍具有更高优先级，因此不会形成固定绑核。

6.6 核心机制

6.6.1 记录不同工具调用产生的进程

设计理由。 运行时启动工具时知道根进程，但后来真正影响返回的进程可能是经过多次 `fork`/`exec` 产生的后代。周期性扫描整棵进程树的工作量会随历史进程数量增长，还可能错过寿命很短的进程。因此，FlowCausal 在创建关系的事件发生时增量维护来源信息。

具体做法。 C 程序登记会话编号、递增的工具调用序号、根进程号和 cgroup 标识。创建新进程时，子进程继承父进程的会话编号和工具调用序号；`exec` 改变程序，但不改变来源。进程退出后删除存活记录。映射键同时保存进程号和启动时间，防止进程号复用把新进程错误连接到旧会话。工具调用结束后，本次序号关闭；仍存活的后代保留原来源，因而能与下一次工具调用产生的进程区分。

这里记录的只是进程来自哪次工具调用，不代表它应当更早运行。只有通过下文的阻塞等待确

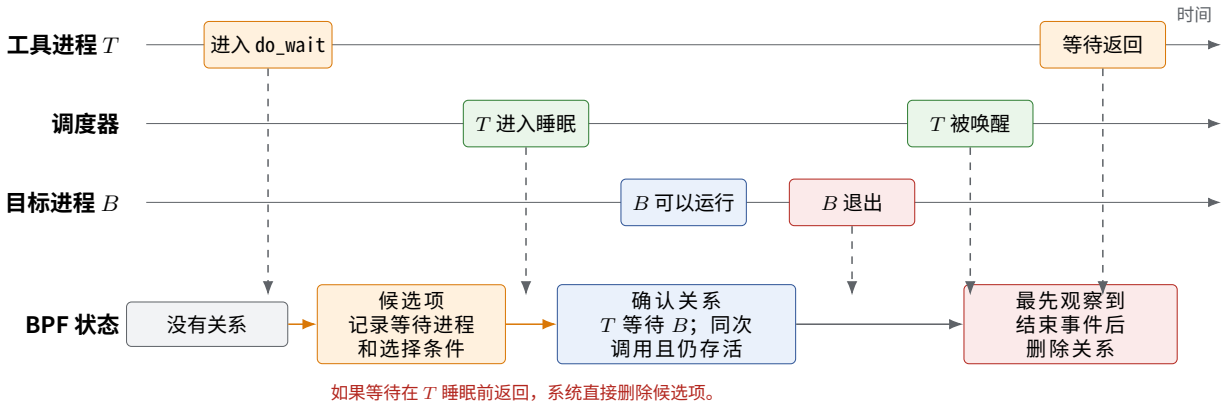


图 6.3: 根据内核事件的先后顺序确认工具返回关系。上方三行是内核事件，下方一行是 BPF 状态。进入等待时只产生候选项，随后观察到进程睡眠才确认关系；最先出现的结束事件会删除关系。若等待在进程睡眠前返回，则不会建立关系。

认后，后代才会进入工具返回关系。这一区分避免把“所有后代都重要”变成长期提权策略。

6.6.2 确认真正的阻塞等待

设计理由。 内核等待路径能够直接说明工具正在等待进程结束，但只看到函数入口仍不够。WNOHANG、已经结束的目标、信号和重试都可能使等待立即返回。因此，FlowCausal 将函数事件与调度器事件结合。

候选与确认。 do_wait 的 fentry 程序记录等待进程和选择条件，带 WNOHANG 的调用直接忽略。如果调度器随后发现该任务在候选项仍有效时离开处理器并进入睡眠，关系才被确认；fexit 程序删除立即返回的候选项。这样就不会把一次没有改变可运行状态的函数调用误认为进程依赖。本文把候选等待产生后、直到等待函数返回或等待进程被唤醒的这段过程称为“一轮等待”；后文只用它限制同一次等待中可以重复执行的动作。

选择相关进程。 若等待指定了明确的进程号，系统只在目标仍存活且属于同一会话和同一工具调用序号时使用它。对于等待任意子进程的 wait(-1)，系统增量查询近期仍存活的直接子进程，不扫描完整后代树。若选中的子进程本身也被确认处于阻塞等待，可以继续扩展一层。当前实现最多选择四个进程、最多跟踪两层等待。例如，工具进程等待构建进程，构建进程又等待编译器，这就是两层等待。这里的目标是识别少量能够解除当前等待的工作，而不是在 BPF 中建立没有上限的依赖图。

删除关系。 等待进程重新可运行、等待函数返回、目标退出、工具调用序号改变或会话结束时，关系都会被删除。序列号会拒绝先前一轮等待中迟到的更新。及时删除与建立关系同样重要，否则一次短暂观测会变成长期优先级。

图 6.3 概括了两个判断：只有等待进程确实进入睡眠，候选项才会变成正式关系；只有相关进程和工具调用仍然有效，关系才会保留。

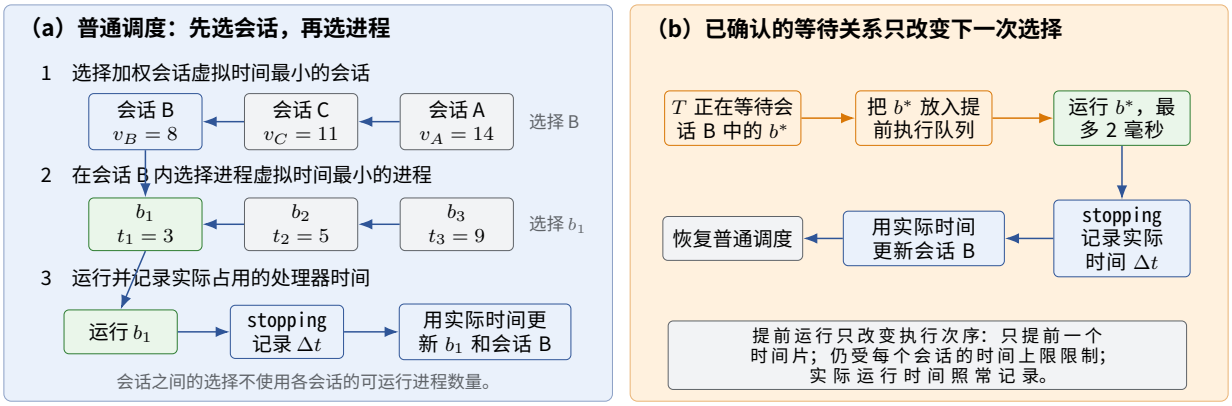


图 6.4: 进程选择和处理器时间记录示例。普通调度按照虚拟时间选择会话 B，再选择 b_1。已经确认的等待关系可以改为让 b^* 提前运行一次，最长 2 毫秒。两种方式都按实际运行时间更新所属会话，然后继续普通调度。

6.6.3 按会话调度与有限提前运行

为何普通任务按会话调度。 如果全部可运行进程在全局竞争，拥有大型测试进程池的会话会因为暴露更多任务而获得更多处理器时间；如果把工具返回关系中的所有子进程都排在其他会话之前，一次涉及大量子进程的等待又可能独占主机。FlowCausal 因此使用加权虚拟运行时间，从一个活动会话中选择普通任务。会话权重仍由系统管理者决定；仅仅创建更多进程不能增加该权重。

为何允许一个进程在普通顺序之前运行。 如果一定等到所属会话再次被选中，解除工具等待所需的进程仍可能排在其他会话之后。因此，工具返回关系中的一个进程可以进入全局“提前执行队列”，获得一个 2 毫秒时间片。这里的“提前”只表示在普通会话选择之前检查该队列。这不是不计成本的高优先级：进程停止运行时，调度器测量它实际使用的时间并计入所属会话的虚拟时间，下一次普通会话选择会反映这段提前使用的处理器时间。会话内部默认仍按每个进程的虚拟运行时间排序；先前工具调用产生且与本次返回无关的进程可以短暂延后，但不会被移除，也不会改变控制组权重。

限制与恢复。 每个会话都有提前执行的时间上限。默认设置只处理一个直接等待对象，同一进程在一轮等待中最多获得一个提前时间片。事件测试还覆盖两层嵌套等待和最多四个进程。只有父进程已经确认处于等待状态且下一层进程变为可运行时，下一层进程才会被选中。全部公开任务性能实验统一只处理直接等待对象，避免不同组合使用不同等待层数。被延后的任务必须在规定时间后重新得到普通调度。来源或等待状态缺失、矛盾或过期时，进程按照普通虚拟运行时间排序。因此，观测不完整只会失去优化机会，不会破坏程序正确性。

优先复用近期处理器。 进程刚运行过的一小段时间内，若上次使用的处理器空闲且符合亲和性设置，FlowCausal 会优先选择它，以尽量保留指令和数据缓存。该选择只是偏好：负载、亲和性和处理器可用状态具有更高优先级，因而不会产生固定绑核导致的计算能力损失。

图 6.4 给出了完整顺序。调度器先检查有明确限制的提前执行队列；若队列中没有可运行进程，再依次选择会话和会话内进程。两条路径使用同一套处理器时间核算。

表 6.2: 内核事实、调度动作及其设计理由。

观察到的事实	调度动作	理由
创建子进程	继承会话编号与工具调用序号	不扫描即可保存来源
进入等待后睡眠	确认进程关系	排除非阻塞查询
关系中的子进程可运行	一个提前时间片	尽快完成解除等待的工作
先前工具调用的进程可运行	暂时延后	减少与当前等待的竞争
近期使用的处理器可用	优先复用	不绑核地保持局部性
唤醒、退出或切换工具	删除状态	防止过期优先级

表 6.3: 实验采用的实现范围。

状态或动作	上限	作用
会话记录 / 恢复位置	4,096 / 64	限制状态和异常恢复遍历
存活任务记录	131,072	限制进程来源状态
工具调用记录	16,384	区分重叠的工具调用
近期直接子进程	每任务 8 个	无须扫描即可处理 `wait(-1)`
关系中的进程	1 个 / 4 个、2 层	性能实验 / 事件测试；限制事件处理
提前执行时间上限	每 100 毫秒 10 毫秒	限制每个会话提前使用的处理器时间
提前时间片 / 暂时延后	2 毫秒 / 20 毫秒	限制单次动作
会话登记超时 / 处理器复用	2 秒 / 250 毫秒	删除失效的登记和局部性状态

6.7 系统实现

6.7.1 调度器与状态组织

为何分别保存会话状态和任务状态。 FlowCausal 由 `sched_ext` 调度器、BPF 事件程序和 C 登记程序组成。调度器把所有活动会话 cgroup 放在一棵按加权虚拟运行时间排序的树中。每个 cgroup 状态表示一个会话；每个任务状态保存虚拟运行时间、会话编号、工具调用序号、近期处理器和等待轮次信息。每个会话的调度队列保存普通任务，后备队列保存未登记任务；全局提前执行队列只保存由已确认工具返回关系选出的进程。后者实际消耗的运行时间由停止运行回调记录，并计入对应会话。单独的会话状态保存已经确认的等待，以及普通会话选择使用的设置权重。

为何只在等权条件下省略重复核算。 普通会话选择同时使用虚拟运行时间和周期处理器时间核算。会话权重不同或会在线改变时，后者不可缺少；但所有已登记会话使用同一个固定权重时，两者表达的是同一个等权约束。C 程序因此维护当前基础权重是否全部相同。只有等权且固定权重时，eBPF 才跳过重复的周期核算；一旦权重不同或在运行期间改变，就继续使用完整核算。该优化不改变队列次序和资源分配规则。

为何所有状态都有明确上限。 表 6.3 列出实验采用的固定范围。它们限制两类成本：映射容量限制常驻状态，即使运行时没有报告结束也不会无限增长；更小的工具返回关系范围限制一次内核事件的处理量和能够提前运行的进程数。64 个活动会话位置只在虚拟时间树状态不一致、需要恢复某个会话队列时检查。普通会话选择直接使用虚拟时间树，不遍历该数组。

6.7.2 事件处理与调度路径

为何在事件发生处更新状态。 事件程序连接进程创建、执行、退出跟踪点，调度状态变化事件，以及 do_wait 周围的 fentry/fexit。BPF 映射把进程号和启动时间组成的身份连接到会话编号与工具调用序号。固定大小的近期子进程记录用于处理 wait(-1)。每个会话的计数器记录候选等待、确认睡眠、建立关系、提前时间片、暂时延后、近期处理器选择、后备调度和过期状态清理。基准测试只在一轮结束后读取这些计数，不会把它们反馈给调度决策。

BPF 程序把事实收集与高频调度分开。进程创建、程序替换、等待、切换和退出事件只在事实变化时更新任务记录。runnable 回调把新近可运行的子进程与当前睡眠的等待进程对应；enqueue 在所属会话队列、提前执行队列和后备队列之间作出选择；select_cpu 尝试复用近期处理器；dispatch 最多先取一个提前执行队列中的进程，随后继续按权重选择会话。最后，stopping 记录实际运行时间，退回没有使用的提前执行时间，并删除单时间片标记。调度回调不会遍历进程树，也不会遍历会话中的全部任务。

为何正常路径不进行详细测量。 实现还把论文机制与详细测量分开。正常版本只更新下一次调度所需的状态；单独的诊断选项才会记录每类进程的运行和排队次数。论文中的主要性能结果使用正常版本。开销实验使用两个构建方式相同的版本：一个关闭工具返回关系及其调度动作，另一个启用全部机制。因此，报告的差值表示维护工具返回关系的新增成本，不包含实验结束后的离线统计程序。

为何调度状态保留在 BPF 中。 调度器与观测程序共享同一个有界任务记录，不在用户态重复建立运行中的状态模型。这样做有两点实际意义。第一，进程入队时能够直接读取刚由内核事件产生的关系，不受轮询周期和事件复制往返影响。第二，BPF 校验器能够依据活动会话扫描上限和工具返回关系上限确定循环次数及内存范围。C 程序负责更适合普通错误处理的工作，例如核对新会话并打开控制组；高频调度判断仍留在 BPF 中。

6.7.3 C 程序登记与异常恢复

为何保留一个小型 C 程序。 flowd_v2 使用 libbpf，以本地 Unix 套接字接收登记消息，核对控制组和进程身份后更新映射。它不判断进程类别，不遍历 /proc，不轮询调度状态，也不选择处理器。若该程序退出，会话登记的定期更新时间随之停止；两秒超时后，相关任务自动使用后备调度路径。Python 只用于实验程序启动容器和离线汇总，不属于所部署的调度器或运行期间的用户态程序。

为何把事件测试与性能实验分开。 实现包含三层测试。事件级测试覆盖 fork 继承、exec 保持、非阻塞等待排除和状态清理；状态恢复测试主动结束 C 程序、遗漏一次工具调用结束消息以及强制目标退出；端到端重放要求命令结果符合预期且内核事件丢失为零。这种结构也体现 kSTEP 关于调度器开发需要确定性事件测试、不能只看吞吐数字的经验 [108]。

6.7.4 CPU 服务核算与重新分派

对于同进程 futex 唤醒，短计算线程获得的提前运行额度受实际睡眠时间限制，避免每次唤醒重复获得完整的 20 毫秒额度。临时会话权重的 CPU 服务核算从任务开始执行时刻与权重生效时刻的较晚者计算，避免将权重生效前的服务按新权重计入。最后一个可运行任务主动让出 CPU 后重新请求分派，仍经过会话份额和硬配额检查。动态并行运行限制只约束同时在 CPU 上运行的任务数量，不修改应用创建的进程数量。

6.8 实验评价

实验将引言中的三项贡献逐一对应到证据。**研究问题一（会话调度）：**把智能体会话作为处理器时间的长期分配单位，能否比 Linux 和按进程调度的 sched_ext 调度器更早完成工作流？**研究问题二（工具返回关系）：**依据工具返回关系安排进程次序，能否在会话调度之上继续产生收益，什么条件下存在这种收益？**研究问题三（适用范围与实现成本）：**同一套固定策略能否适用于不同处理器数量、会话数量和全部任务，并把处理器开销、状态数量和异常情况下的退回处理控制在实际可用范围内？以下每项结果只回答对应问题，消融实验不被当作外部调度器对照。

6.8.1 实验方法

实验机器。 实验主机配备两路 Intel Xeon Platinum 8375C，共 128 个逻辑处理器，可用内存 256 GiB，运行 Linux 6.17，处理器频率策略设为 performance。除规模实验外，全部工作流和调度器都限制在 0–7 号处理器上，即同一 NUMA 节点上的八个物理核；这些核的超线程兄弟不在所选集合中。同一组合的全部会话共享这八个处理器，不对各会话分别绑核。

对比调度器。 本文选择三种主要对照：（1）Linux 默认公平调度；（2）上游 scx_simple，本实验采用其默认的全局加权虚拟运行时间模式，它是 sched_ext 的基础示例；（3）Linux 下每个工作流处于独立同权 cgroup。Linux 代表生产环境默认设置；scx_simple 用来在相同调度框架中评估 FlowCausal 加入的运行信息是否有效；同权控制组用于区分“建立控制组”和 FlowCausal 进程排序的作用。“仅会话调度版本”的 FlowCausal 使用相同代码和按会话加权调度，但关闭工具返回关系、提前执行、暂时延后以及近期处理器复用。该版本只用于衡量这些机制带来的额外作用，不把它当成外部基线。

我们也尝试了上游 scx_flatcg。限制处理器范围后，两批各五轮实验都出现系统任务超过 30 秒没有运行，内核随后卸载该调度器。后续工作实际由 Linux 完成，因而这些结果不能组成 scx_flatcg 的性能均值。Linux 同权控制组提供稳定的控制组对照，仅会话调度版本则单独衡量 FlowCausal 中对应机制的作用。

运行方式。 不同方法使用完全相同的清单、容器镜像、控制组、可用处理器、启动时间和超时设置。两组四会话完整系统比较以及工具返回关系机制实验中，每种方法运行五次；较大的规模测试运行三次。每轮开始前清理页缓存；在同一系列中轮换方法顺序；失败结果保留现场，成功结果提取数据后删除可重建工作目录。表中报告算术平均值与样本标准差。所有进入论文的运行均符合任务预期返回类型、完成清理并且没有丢失内核事件。

每个会话位于独立 cgroup 且权重均为 100，全部会话共享同一个可用处理器集合。scx_simple 不使用控制组身份，但执行相同任务并使用相同处理器集合。所有方法都运行相同的 C/eBPF 记录程序。只有 FlowCausal 还运行 C 登记程序，其处理器时间计入开销结果。scx_simple 是使用相同框架的主要对照，Linux 默认调度是部署参考。

6.8.2 研究问题一（a）：完整系统与调度基线

主要结论：在“混合工作流组合”中，FlowCausal 比所有外部对照更早完成最后一个工作流，平均工作流完成时间也最短。图 6.5 展示四会话组合中每种方法的五轮交错实验。相对 scx_simple，完整 FlowCausal 将最后一个工作流完成时间从 101.872 秒降至 90.857 秒（10.80%），平均工作流

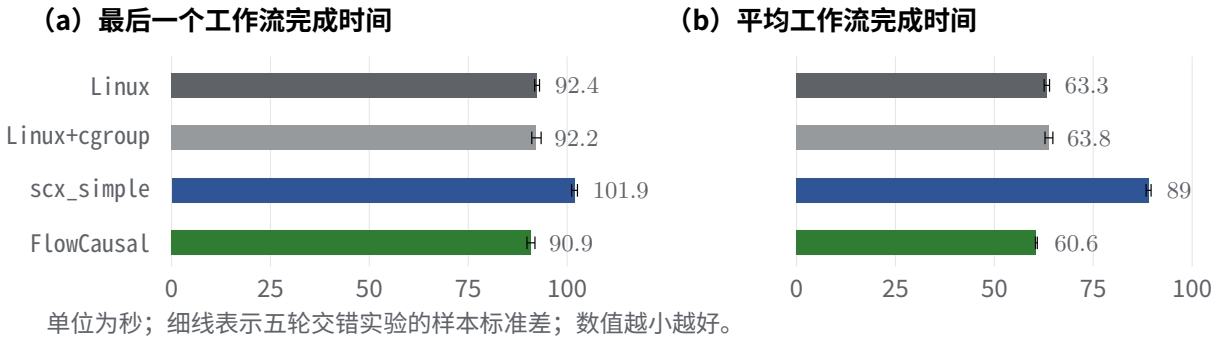


图 6.5: “混合工作流组合”的完整结果。标签给出五轮绝对平均时间，细线表示样本标准差，数值越低越好。

表 6.4: 混合工作流组合（平均值 ± 样本标准差，五轮）。

调度器	最后完成（秒）	平均工作流（秒）
Linux	92.411 ± 0.613	63.269 ± 0.682
Linux+cgroup	92.245 ± 1.181	63.797 ± 0.993
scx_simple	101.872 ± 0.740	88.990 ± 0.633
FlowCausal	**90.857** ± 1.011	**60.621** ± 0.236

完成时间从 88.990 秒降至 60.621 秒（31.88%）。两项逐轮配对变化的 95% 置信区间分别为 [8.95%, 12.66%] 和 [31.04%, 32.71%]。

Linux 默认调度的最后一个工作流和平均工作流完成时间分别为 92.411 秒和 63.269 秒；FlowCausal 分别缩短 1.68% 和 4.17%，逐轮配对区间为 [0.56%, 2.81%] 和 [2.51%, 5.84%]。Linux 使用同权控制组时的两项结果为 92.245 秒和 63.797 秒，与默认调度接近。FlowCausal 相对它分别缩短 1.48% 和 4.96%。由此可见，FlowCausal 的作用不只是建立控制组，还包括在会话之间分配处理器时间，以及在会话内部安排进程次序。

第二组公开任务。 “工具返回组合”保留 Dask、DVC、同时启动和 0–7 号处理器，用第二个 Numba 任务替换 SQLFluff。FlowCausal 相对 Linux 将最后一个工作流和平均工作流完成时间分别缩短 2.58%（区间为 [1.28%, 3.87%]）和 5.23%（区间为 [3.04%, 7.42%]）；相对 scx_simple 的平均工作流完成时间缩短 27.86%。相对 scx_simple 的最后完成时间差异为 0.54%，其区间跨过零。因此，两种任务组成相对 Linux 的两项指标都得到改善，而“混合工作流组合”给出了相同调度框架下更明确的最后完成时间收益。

6.8.3 研究问题一（b）：会话调度的作用

主要结论：把智能体会话作为处理器时间的分配单位，可以避免进程数量多的工作流仅凭任务数量占用更多处理器。一组独立的处理器时间实验解释了为什么需要按会话分组。任务组 A 保持 2 个可运行进程，任务组 B 保持 16 个可运行进程。两组位于同一控制组时，Linux 为 A 分配 11.14% 的处理器时间；把两组放入两个同权控制组后，A 的占比提高到 21.83%。在 scx_simple 下，两种设置均约为 11.13%，与按进程数量分配的结果一致。因此，全局进程队列会让进程多的工作流仅凭进程数量取得更多运行机会；先选择会话可以消除这一影响。主要组合中，scx_simple 与 FlowCausal 的平均工作流完成时间相差 28.72 秒，而 Linux 已经取得按控制组分配带来的大部分作用。下一组实验再单独衡量工具返回关系在会话调度之上增加的作用。

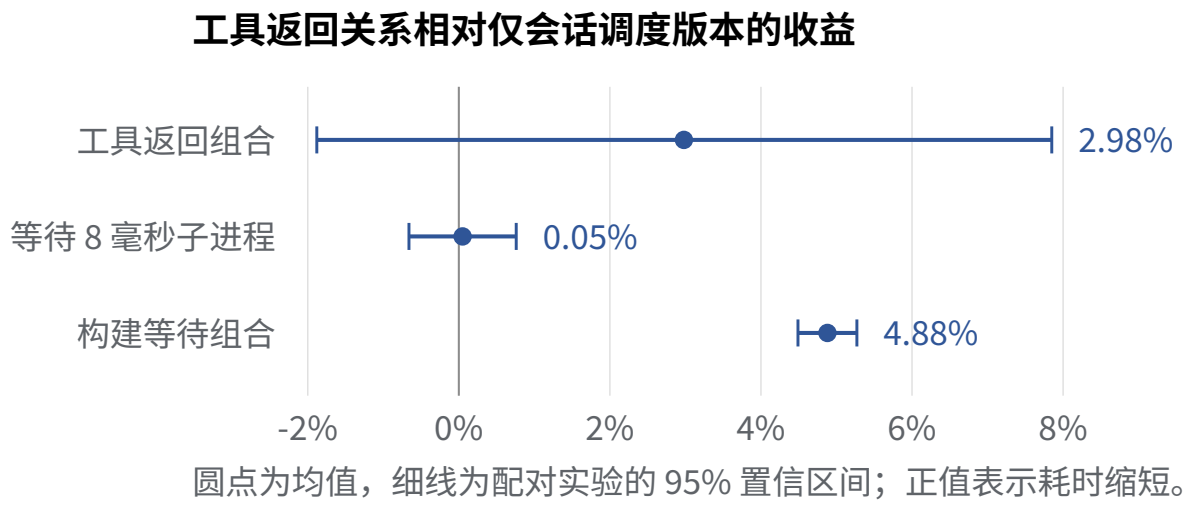


图 6.6: 工具返回关系相对仅会话调度版本的平均工作流完成时间收益，每种方法五轮。“构建等待组合”稳定改善，被等待子进程只运行约 8 毫秒的组合接近不变。

6.8.4 研究问题二：工具返回关系的额外收益

主要结论：当解除工具等待所需的进程仍在竞争处理器时，依据工具返回关系调整进程次序能够加快完成；当这段工作极短时，结果接近不变。图 6.6 使用同一调度器实现，对比完整 FlowCausal 与仅会话调度版本。在“工具返回组合”中，加入工具返回关系后平均工作流完成时间缩短 2.98%，95% 置信区间为 $[-1.88\%, 7.85\%]$。在“构建等待组合”中，工作流等待处理器密集的构建子进程结束后还要继续运行，工具返回关系使平均工作流完成时间缩短 4.88%，区间为 $[4.49\%, 5.27\%]$。这些任务都在查看候选调度器耗时之前，依据 `scx_simple` 的内核事件记录选出。

短子进程组合提供了有区分力的对照。选中的子进程只运行 7.75 毫秒，关系中只有一个任务。完整 FlowCausal 相对仅会话调度版本的平均工作流完成时间变化为 0.05%，区间为 $[-0.66\%, 0.76\%]$，接近预期的无明显变化。由此可见，该机制并非简单地提高当前工具调用的整体优先级；只有能够解除等待的子进程还需要足够多的处理器时间时，重新排序才会产生收益。

调度器计数进一步说明动作确实来自这一路径。在混合组合中，每轮约记录 10,983 个带有会话编号和工具调用序号的进程事件，以及 1,236 次确认睡眠等待，却只实际提供 31.7 个提前时间片，并暂时延后先前工具调用产生的进程 2.7 次。观测数量与动作数量之间的差距是有意设计的：来源和睡眠确认先缩小候选范围，每轮和每会话限制再使最终调度调整保持稀疏。

6.8.5 研究问题三（a）：处理器数量与会话数量

主要结论：FlowCausal 在标准和较多处理器设置下都能缩短相对 Linux 的平均工作流完成时间；八个会话同时运行时收益仍然存在。图 6.7 使用“工具返回组合”。四个会话使用 4、8 和 16 个处理器时，完整 FlowCausal 相对 Linux 将平均工作流完成时间分别缩短 3.22%、5.23% 和 5.66%。三轮 4 处理器实验的区间跨过零，8 和 16 处理器实验的区间保持为正。所有点都直接以 Linux 为参照，避免把 `sched_ext` 框架本身与 Linux 的差异误当成 FlowCausal 算法收益。

在八个处理器上把并发会话增至八个后，FlowCausal 相对 Linux 将平均工作流完成时间缩短 3.45%，区间为 $[1.95\%, 4.96\%]$。相对仅会话调度版本，完整 FlowCausal 又将最后一个工作流和平均工作流完成时间分别缩短 1.67% 和 1.61%，两项区间都保持为正。这正是 FlowCausal 面向的混

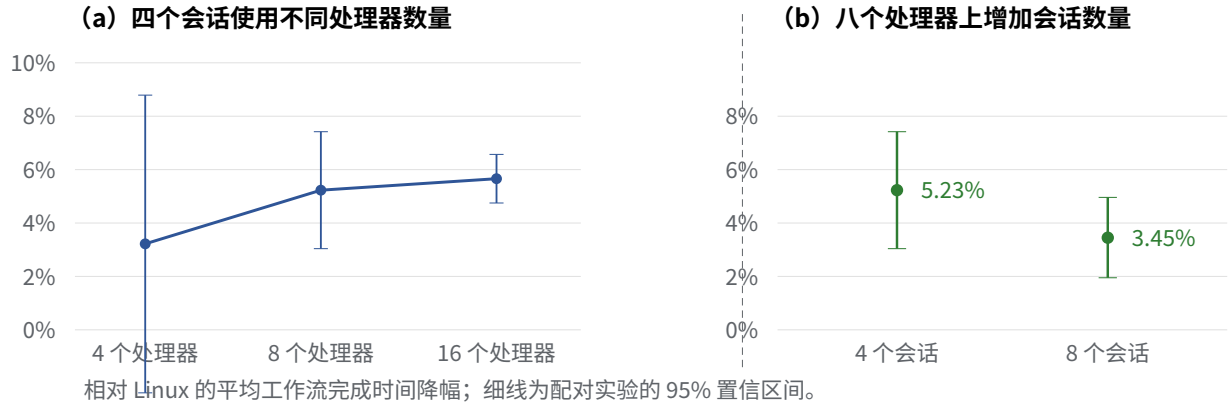


图 6.7: 工具返回组合的规模变化。所有数值均为相对 Linux 的平均工作流完成时间降幅，细线为逐轮配对的 95% 置信区间。处理器规模测试使用四个会话，会话规模测试使用八个处理器。

合运行情况：多个进程树同时可运行，但每次工具等待只有一组受限进程决定返回。全部八个会话共享同一处理器集合，没有为任何工作流分配专用处理器。

6.8.6 研究问题三（b）：全部任务的覆盖检查

主要结论：同一套固定策略能够覆盖任务库中的全部工作流，不需要根据代码库或命令名称编写专用规则。我们把 50 个任务分为 10 个五会话组合，每组在 0–7 号处理器上同时启动，每个任务恰好出现一次。当前 FlowCausal 完成了全部 10 个组合和 2,940 条命令。汇总十组结果，最后一个工作流完成时间相对 Linux 和 `scx_simple` 分别缩短 0.20% 和 0.31%，平均工作流完成时间分别缩短 0.26% 和 1.66%。这是覆盖范围检查而不是显著性结论：策略没有使用代码库名称，主要实验和机制实验中的多轮结果仍负责证明性能收益。

6.8.7 研究问题三（c）：成本、关系识别率与恢复

主要结论：完整在线路径约占所测八个处理器总计算能力的 1.2%，状态缺失时会自然退回普通排序。在“工具返回组合”中，完整 BPF 路径累计消耗 7.929 秒处理器时间，占八个处理器总计算能力的 1.173%。仅会话调度版本占 0.727%，其余机制新增 0.445 个百分点。917 万次 BPF 调用的平均耗时为 864 纳秒；四个 C 登记程序占总计算能力的 0.0292%，合计最大常驻内存为 8.8 MiB。

会话等权时，跳过重复的周期核算使 `dispatch` 平均耗时降低 15.5%，BPF 总处理器时间降低 10.1%。权重不同或运行期间发生变化时仍使用完整核算；重点任务和全部任务复验确认该处理没有改变调度结果。

离线分析在实验结束后重建每次阻塞工具调用可以由哪些进程解除等待；这些信息不会交给在线调度器。在线机制识别出其中 99.04% 的进程。漏掉一个进程只会减少优化机会，不会使它失去普通虚拟运行时间调度。恢复测试覆盖非阻塞等待、目标退出、工具调用结束消息遗漏、C 程序退出以及延后进程恢复普通调度。所有性能实验的任务结果均符合预期，内核事件丢失数均为零。

表 6.5: 工具返回组合中的机制证据与开销。

测量项目	结果
工具返回关系中的进程识别率	99.04%
丢失内核事件	0
新增 BPF 处理器开销	0.445%
完整 BPF 处理器开销	1.173%
BPF 单次调用平均耗时	864 纳秒
C 登记程序处理器开销	0.0292%
C 登记程序合计最大常驻内存	8.8 MiB
任务结果检查	全部通过
状态恢复测试	全部通过

6.9 跨命令依赖的调度与评估

6.9.1 旧命令进程成为当前等待对象

先前命令留下的后台进程可能被当前命令通过 `waitpid` 等待。如果调度器仍按旧后台身份安排该进程，它就可能延迟已经依赖于它的当前工具调用。为研究这种依赖变化，将跨命令处理作为单独的实验原型，与 FlowCausal 的默认实现进行对照。

原型对后台成员合计分配份额；确认某个成员成为当前工具的等待对象后，在其下一次入队时消除旧后台身份积累的组内排队惩罚。实际 CPU 服务仍计入所属会话，原有工作流份额、并行运行限制、硬配额和提前运行预算继续生效。这一处理改变会话内的执行次序，不增加该会话的总资源份额。

6.9.2 固定工作量消融

机制基准创建 2 个工作流，每个工作流先等待新命令子进程，再等待旧命令子进程。测试不向调度器预告后续等待对象；完成时间包含所有后台工作，计算结果通过校验值核对。计时从进程登记后的统一放行开始。主配置包含 2 秒模拟非 CPU 阶段，不包含真实 I/O 或完整应用启动成本。

在 4 个 CPU、8 个后台进程的同批消融中，每种方案运行 3 轮。原生 Linux 的平均完成时间为 4.828 ± 0.116 秒，完整原型为 4.065 ± 0.009 秒，降低 15.79%。关闭跨命令关联后为 4.925 ± 0.007 秒，关闭后台合计份额后为 4.907 ± 0.015 秒。后台进程数为 16 时降低 28.66%，为 2 时基本持平；移除模拟等待的对照降低 7.63%。结果说明，该机制的收益与依赖转换及后台竞争程度有关。

6.9.3 八工作流对照

在 4 个 CPU 上并发运行 8 个工作流，以交错顺序比较原生 Linux、FlowCausal 和跨命令依赖原型，每种方案运行 3 轮。四会话实验与本组实验采用各自固定的实现及运行配置；本组实现包含上述唤醒额度、CPU 服务核算与重新分派处理。表 6.6 报告轮间均值和样本标准差。

FlowCausal 相对原生 Linux 将平均工作流完成时间降低 7.32%，全部完成时间增加 2.65%。跨命令依赖原型的平均完成时间与默认实现接近；后台合计份额与依赖转换修正的关键计数均为零。因此，这组真实工具回放未显示该原型的额外收益，回放中的总体改善不能归因于跨命令处理。

实验仅统计实际调度器与计划一致的样本；原生 Linux 对照禁止配置调度器二进制或工具控制器，并在运行时检查调度器状态。机制实验共包含 66 次运行，八工作流对照包含 9 次运行，各组

表 6.6: 八工作流完成时间，单位为秒；均值 ± 样本标准差，3 轮。

方案	平均工作流完成	全部完成
原生 Linux	25.986 ± 0.100	52.201 ± 0.497
FlowCausal	24.084 ± 0.016	53.584 ± 0.057
跨命令依赖原型	24.103 ± 0.078	53.587 ± 0.053

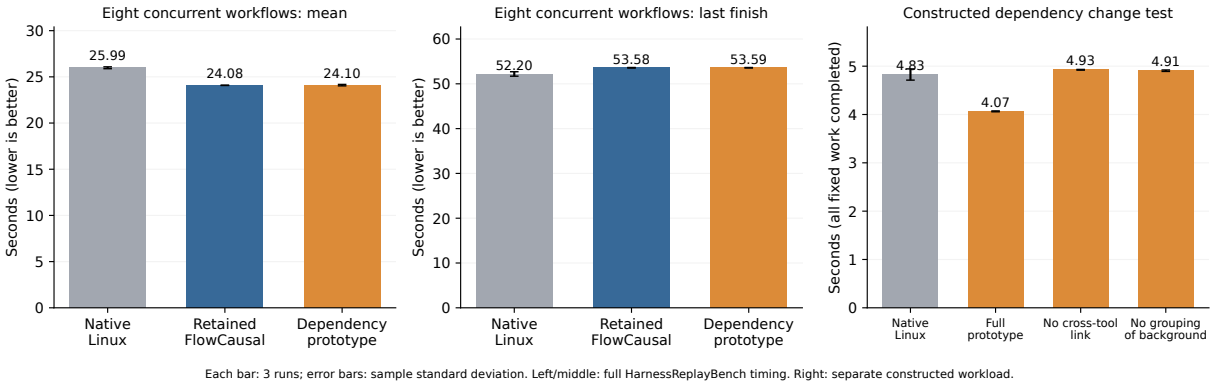


图 6.8: 跨命令依赖的固定工作量消融与八工作流对照。两类实验分别测量依赖转换的作用与完整工具执行的完成时间。

分别统计。标签与实际调度器不一致的试验不进入性能结果。

6.9.4 作用条件

固定工作量实验直接构造从后台进程到当前等待对象的依赖转换，能够触发对应调度动作；真实工具回放中未触发这些关键计数。两类结果表明，该原型仅在特定依赖和竞争条件下产生收益。I/O 优待筛选的对照改变了完成顺序，但未改善平均完成时间，因此默认实现不启用该规则。

6.10 讨论

FlowCausal 优化什么。 FlowCausal 面向这样一种处理器执行：工具进程已经阻塞，而某些子进程结束后才能解除等待。它不判断软件修改是否正确，也不向某个会话增加处理器。它识别能够解除已观察等待的存活进程，并把这些进程提前运行的时间计入所属会话。

与应用内部并行的关系。 构建系统和测试框架已经会决定自身进程数量与并行方式。FlowCausal 不修改其工作进程数，也不覆盖应用设置的亲和性。调度器只处理已经可以运行的进程；没有确认等待时，仍采用普通虚拟运行时间排序。

适用范围。 该机制依赖标准进程创建和等待行为，不依赖特定语言或智能体框架，因而能够覆盖 HarnessReplayBench 中的命令解释器、测试框架、编译器和工作进程池。eventfd、futex 或远程服务等其他完成方式需要分别验证，不能直接视为相同的等待关系。

部署方式。 `sched_ext` 允许在不修改内核核心调度代码的情况下加载策略，BPF 调度器退出后还会回到普通调度器 [104]。C 程序登记可以接入容器或微虚拟机的启动流程；会话权重、处理器掩码和普通控制组设置仍由系统管理者决定。

6.11 相关工作

相关系统分别提供可编程处理器调度、预先给出的工作流或者智能体编号。下面比较哪些系统能够给出会话边界，以及哪些系统能够观察本文调度所需的实时进程关系。

Linux 与应用处理器调度。 已有研究通过缺陷分析和策略重建考察 Linux 调度器 [109]。Nest 通过任务放置保持局部性 [110]；Caladan 将处理器分配与低时延运行时调度结合 [111]；Demeter 针对黑盒负载自适应调整处理器服务 [112]；其他工作分别研究虚拟处理器抽象、分组调度、考虑任务相互影响的放置，以及高性能计算任务的系统级调度 [113–116]。早期依赖驱动调度根据系统调用历史和反馈结果推测资源请求者与提供者之间的关系 [117]。FlowCausal 则把运行时提供的工具调用编号与内核确认的进程等待结合，建立只属于当前工具调用的工具返回关系，并把调整次序所用的处理器时间计入所属会话。

可编程调度与内核自适应。 Arachne 面向应用线程进行处理器感知管理 [118]。ghOSt、Skyloft 和用户态进程调度器把策略判断移出传统内核代码 [103, 119, 120]；LibPreemptible 加快用户态抢占 [121]；Enoki 支持快速开发 Linux 调度器 [122]；`sched_ext` 则提供本文使用的 BPF 调度接口 [104]。Xkernel 和 AutoOS 调整或生成范围更广的内核策略 [123, 124]。调度器协作锁和应用提供信息的内核同步机制说明，应用结构可以支撑有针对性的内核动作 [107, 125]。FlowCausal 把运行时提供的工具调用编号与内核确认的等待事实结合，并把提前使用的处理器时间计入所属会话。

集群与工作流调度。 Decima 为数据处理依赖图学习调度策略 [100]。Twine 和 Gödel 分别在作业、容器和资源池层面管理共享基础设施 [101, 102]；Autopilot 调整工作负载容量 [126]；Singularity 支持抢占与迁移 [127]；MLaaS 和 Beehive 分别处理加速器作业和高频小任务 [128, 129]。这些系统接收显式作业、依赖图或资源请求；FlowCausal 处理的是已经运行的沙箱会话内部不断出现的进程。

模型与加速器调度。 Orca 调度自回归生成的迭代 [130]；vLLM 通过 PagedAttention 管理键值缓存 [131]；Sarathi-Serve、DistServe 和 Llumnix 分别协调批处理、分离式服务和请求迁移 [132–134]。REEF 和 XSched 实现加速器抢占 [135, 136]；PowerInfer 在处理器与图形处理器之间划分推理计算 [137]；SAGA 与 Cortex 调度智能体的加速器工作流 [138, 139]。FlowCausal 调度的是执行代码库工具的主机进程。

智能体基准与运行时。 SWE-bench、SWE-agent、SWE-rebench 和 Terminal-Bench 使代码库与终端任务能够重复执行 [140–143]。Firecracker 提供高效微虚拟机基础 [105]。AgentCgroup 管理工具调用资源 [99]，Crab 根据智能体进展安排检查点保存 [106]。HarnessReplayBench 使用这些公开任务环境在受控条件下比较主机调度器；FlowCausal 在登记后的会话内部确认这些接口没有表达的工具返回关系。

6.12 结论

本文提出 FlowCausal，将运行时提供的工具调用标识与内核确认的阻塞事件结合，恢复工具返回所依赖的进程。系统以 cgroup 组织会话，在会话份额内提供有上限的提前运行，并按实际 CPU 使用记账。HarnessReplayBench 的 50 个公开任务用于评价这一设计：两组四会话实验中，相对

Linux 的平均完成时间降低 4.2%–5.2%；构建等待消融中，工具返回关系相对仅会话调度进一步降低完成时间 4.88%。

在 4 个 CPU、8 个工作流、每种方案交错运行 3 轮的对照中，FlowCausal 相对原生 Linux 将平均完成时间降低 7.32%，全部完成时间增加 2.65%。跨命令依赖原型在固定工作量消融中降低平均完成时间 15.79%，但在完整工具回放中未取得额外收益。这些结果说明，实际等待关系可用于改善会话内的执行次序，其作用取决于等待进程的计算量、后台竞争及依赖转换是否发生；平均完成时间与全部完成时间仍需分别评估。

第七章　专用机制的适用边界与跨场景分析

7.1　锁交接的两个调度机会

LOCKCOOP 以同一份逐锁状态连接解锁前的持锁者推进和唤醒后的后继放置。二者并不在每个场景中贡献相同：纯 OLTP 的主要收益来自同锁放置，CPU 干扰实验则识别持锁者推进的增量作用。资源域锚定进一步减少首次放置与不绑定拓扑造成的波动。

实验覆盖 openEuler 24.03-LTS ARM64、x86、RISC-V 和 VMware。在 256 线程 MariaDB OLTP 实验中，LOCKCOOP 相对同机 `scx_simple` 在 ARM64 节点 0 达到 5.30 倍吞吐、p95 延迟降低 84.8%；x86 全机不绑定的吞吐为 4.14 倍、p95 降低 82.2%；RISC-V 的吞吐为 2.15 倍。加入 CPU 干扰后，组合机制相对仅启用 LockNest 的吞吐增加 9.6%–36.1%，批处理完成工作量差异不超过 0.53%。这组消融区分了同锁放置与持锁者推进的作用。

7.2　工具返回与会话公平的关系

FlowCausal 将会话间的 CPU 分配与会话内的进程次序分别处理。cgroup 确定会话归属，工具调用标识与实际阻塞事件确定等待关系；已确认的等待对象可以短暂提前运行，实际 CPU 使用仍计入会话份额。

在四 CPU、八工作流、每种方案交错运行三轮的对照中，FlowCausal 相对原生 Linux 将平均完成时间降低 7.32%，全部完成时间增加 2.65%。两项指标的不同变化表明，部分工作流提前完成，并不意味着最后一个工作流也更早结束。

跨命令依赖原型在固定工作量消融中降低平均完成时间 15.79%，但在完整工具回放中未触发依赖转换修正的关键计数，也未取得额外收益。其作用条件是后台进程成为当前等待对象并受到组内排队影响，不能将默认调度器的总体收益归因于未触发的机制。

7.3　证据对应与机制选择

KernAdapt 的候选策略应由当前依赖关系及直接指标支持。锁竞争任务的 waiter、verified owner 与逐锁 CPU 历史支持释放前后动作；工作流的实际阻塞、工具代际和会话服务支持返回依赖调度。CPU 利用率、进程数和应用名称不能单独替代这些证据。

各组实验分别记录平台、CPU 范围、缓存与数据库配置，并按固定协议重复测量。性能对照使用同机同批次基线；原生 Linux 对照在运行前独立检查实际调度器身份。

机制状态失效、预算用尽或目标 CPU 不可用时，数据面沿用正常调度路径。对控制面而言，新增机制证据需要与适用条件、端到端效果和共置任务服务同时关联，才能形成可重复使用的策略经验。

7.4　适用范围

KernAdapt 的两组实验分别测量组合预测与实际执行效果。560 个实验组中，完整动作留出的平均 regret 为 0.04357；600 次决策中实际执行 147 次，114 次取得正收益。前者比较方案选择的质

量，后者测量运行收益；两组实验均固定模型与参数，范围限定于已有任务分布和动作参数。

跨 ISA 与更多应用实验扩展了 LockCoop 的证据范围，但并不保证对所有锁 ABI、CPU 拓扑或低竞争应用均有收益。FlowCausal 的评估聚焦 CPU 可见工具执行，固定工作量机制基准还具有明确的计时边界；模型推理、远程服务、动态工具选择与真实任务正确性需要独立评价。

第八章　总结与展望

8.1　研究工作总结

共享基础设施使数据库、持续集成、代码智能体沙箱与异构计算负载在同一 Linux 节点上并发运行。集群层已经完成节点选择之后，内核仍需决定任务何时运行、运行在哪个 CPU 以及如何共享缓存、NUMA、内存带宽和 I/O 等资源。现有通用调度器能够维持任务级公平，但缺少锁依赖、工作流阶段和设备供给链等上层进展关系；另一方面，`sched_ext` 、eBPF、cgroup 和 resctrl 降低了部署专用机制的成本，却没有自动解决机制何时适用的问题。本文围绕“如何识别决定应用进展的执行关系，并据此安全使用专用内核机制”开展研究。

首先, 本文从用户可见进展出发建立混合负载分析模型。该模型区分任务集合、进展关系、资源路径与性能目标, 并说明运行队列、CPU 利用率和缓存未命中等资源现象到性能根因存在多对一映射。同步密集服务由等待者–锁–持锁者–后继者关系决定局部进展，沙箱工作流由会话–阶段–进程组–前沿关系决定阶段推进, 批处理与异构推理还可能受到 barrier、内存带宽、文件系统和 CPU–GPU 供给链限制。由此得到的基本结论是：专用机制的选择依据必须包含可识别的进展关系和可干预瓶颈，不能由应用名称或单个资源计数器直接决定。

其次，本文研究了资源配置的选择方法。系统检查机制的运行条件，比较其对目标负载和共存任务的影响，并计入观测开销与切换成本。内核完成逐任务调度，用户态按较低频率选择方案、安排试运行并分析结果。执行范围、资源预算、超时和恢复检查限制错误决策的影响。KernAdapt 实现这一过程，模型负责提出方案，独立执行模块负责权限检查与资源调整。

再次，本文针对同步密集负载提出 LOCKCOOP 。该机制把原本对调度器不可见的逐锁因果关系转化为调度状态，并以两个相互衔接的动作处理锁释放前后的退化路径：LockBoost 根据持锁者信息的可靠程度决定是否延长时间片，并限制扩展上限，使等待者所依赖的持锁者及时运行；LockNest 根据每把热点锁的近期交接位置放置后继任务，使同锁连续持有者在整个缓存一致性拓扑中保持接近。两者共同构成 LOCKCOOP ，而不是并列的负载类别调度器。

FlowCausal 通过工具调用标识与实际阻塞事件恢复工具返回关系，以 cgroup 组织会话并按份额分配 CPU。调度器在会话内部为已确认的等待对象提供短时间片提前运行，实际服务计入所属会话。跨命令依赖原型进一步处理先前工具的后台进程成为当前等待对象时的排队延迟，并通过独立消融实验分析该机制的作用条件。

8.2　主要研究结论

8.2.1　应用进展与任务公平具有不同语义

实验和机制分析共同表明，决定应用短期进展的任务可能只是可运行任务中的少量动态子集。一个锁持有者获得短暂连续服务可以同时释放多个等待者，一个工作流前沿的尾部进程完成可以使整个会话进入下一阶段。通用任务公平仍是长期资源分配的必要约束，但仅以任务虚拟时间或平均 runnable wait 为目标，不能完整表达锁释放时间、阶段屏障和用户可见完成时间。专用机制的价值正在于恢复这些被任务抽象隐藏的进展语义。

8.2.2 锁竞争需要同时处理时间与空间退化

锁交接前后的动作需要分别验证。在 256 线程 OLTP 实验中，LOCKCOOP 相对 `scx_simple` 在 openEuler ARM64 节点 0 达到 5.30 倍吞吐并降低 p95 84.8%，在 x86 不绑定达到 4.14 倍、在 RISC-V 达到 2.15 倍。纯 OLTP 的收益主要来自同锁放置；加入 CPU 干扰后，组合机制相对 LockNest 额外提高吞吐 9.6%–36.1%，从而直接验证持锁者推进的增量作用。

跨应用结果显示，不同数据库对两个动作的敏感程度不同：部分负载主要受持锁者延迟影响，部分负载更依赖交接局部性，部分负载则只在尾延迟上体现收益。因此，确认存在锁竞争仍不是选择完整组合机制的充分条件，还需要结合直接指标判断时间和空间路径各自的贡献。

8.2.3 工具返回关系决定会话内的调度次序

代码智能体工作流需要区分工具进程的来源与当前真正的返回依赖。FlowCausal 从实际阻塞确认关系，并在会话公平服务内施加有界提前运行。在四 CPU、八工作流、每种方案三轮交错运行的对照中，平均完成时间降低 7.32%，全部完成时间增加 2.65%。独立依赖变化原型在固定工作量消融中降低平均完成时间 15.79%，但在上述完整回放中未触发关键机制计数，也未带来额外收益。

8.2.4 异构机制应统一选择原则而非数据面状态

LockCoop 与 FlowCausal 表明，异构机制能够统一的是选择与验证原则，而不是数据面内部状态。逐锁关系和工具返回关系具有不同的建立事件、生命周期、直接指标和调度动作，若把二者压缩为统一“关键任务分数”，就会丢失机制可解释性。KernAdapt 统一表达其适用条件、动作边界、评价目标与恢复语义，各机制仍保留独立快速路径，从而兼顾专用调度能力与统一在线管控。

8.3 研究局限

本文仍有以下限制。第一，LockCoop 已经覆盖 ARM64、x86 双路裸机与 RISC-V、有无 SMT、socket 内与双 socket 范围以及 VMware guest，但尚未覆盖 AMD 裸机、四路及以上 NUMA 服务器、共享 futex、PI 和 requeue 路径上的时间片扩展和长期多租户部署。第二，FlowCausal 主要评估工作流回放中的工具执行，真实代码智能体还包含模型服务、网络、工具选择、失败重试和任务正确性，工具执行提速不一定等比例缩短完整会话。第三，KernAdapt 已完成 E1 任务分布内的组合预测与执行实验；模型和策略在评测期间保持不变，跨主机、未知工作负载和长期在线更新仍需验证。第四，现有数据面工作主要围绕 CPU 调度与局部性，缓存容量、内存带宽、I/O 和 GPU runtime 尚缺少同等完整的专用机制和实验。

这些边界不改变已验证机制在目标实验中的结论，但限制其外推范围。后续工作需要继续遵循本文的归因标准：先确认进展关系和可干预瓶颈，再验证动作与直接指标，最后评价端到端收益、安全性和共存影响。

8.4 未来研究方向

8.4.1 缓存容量与内存带宽管控

LLC miss 同时包含容量驱逐、一致性迁移、远端 NUMA 和流式访存等不同来源。未来可结合 LLC occupancy、MPKI、HITM、uncore 带宽和页面位置区分这些路径，并研究 resctrl CAT/MBA、并

行宽度与任务/内存共置之间的协同。该方向不能直接复用 LockNest 的逐锁锚点，因为工作集容量竞争与锁保护缓存行交接具有不同状态和时间尺度。

8.4.2 异构推理的主机关键路径

在 GPU 主导的推理服务中，分词、采样、算子启动、主机内存和数据传输可能使设备等待数据，此时 CPU 调度才可能影响整体完成时间。仅使用 CPU 的推理或混合推理还可能受到工作线程同步、末级缓存、NUMA 和内存带宽限制。后续需要结合运行时事件、CPU 调度事件和 GPU 执行记录定位主机瓶颈，再决定如何调整数据准备线程、CPU 放置、线程数、内存带宽或设备调度。

8.4.3 容器、虚拟机与跨层协同

在容器环境中，主机能够直接观察进程；在 microVM 中，主机通常只看到 vCPU 和 VMM 线程。未来可研究 guest 内最小进展接口与 host 侧资源管控的组合，使进展语义跨越隔离边界而不泄露完整应用状态。集群调度器还可以向节点传递服务目标和资源预算，节点则返回机制适用性与效果证据，从而形成跨层但不混淆状态所有权的协同控制。

8.4.4 可验证在线专家

在线专家的进一步研究重点不应是生成更复杂的调度器代码，而是提高瓶颈假设的新颖性、证据引用的完整性和未知场景下的拒绝能力。可以结合结构化检索、反事实重放、形式化 schema 和确定性 verifier，量化模型相对规则基线在新负载识别、状态冲突消解和失败归因上的增量。对于高风险动作，候选生成与执行授权应继续分离，使模型能力提升不会扩大单次错误的作用域。

8.5 本章小结

本文从混合负载的进展关系出发，研究了专用内核机制的设计、验证与在线选择。LockCoop 证明锁持有者推进和同锁交接局部性可以共同缓解同步扩展性退化，FlowCausal 表明实际工具返回关系可以成为会话内 CPU 调度的依据，KernAdapt 则给出在不完全观测与安全约束下管理异构机制的方法。全文结果说明，自适应内核资源管控的基础是建立进展关系、资源瓶颈、机制动作和用户可见目标之间能够通过实验解释的联系。

参考文献

[1] Tang C, et al. Twine: A unified cluster management system for shared infrastructure [C]//OSDI. 2020: 787-803.

[2] Xiang W, et al. Gödel: Unified large-scale resource management and scheduling at bytedance [C]// SoCC. 2023: 308-323.

[3] Choudhury A, et al. MAST: Global scheduling of ML training across Geo-Distributed datacenters at hyperscale [C]//OSDI. 2024: 563-580.

[4] Lozi J P, Lepers B, Funston J, et al. The linux scheduler: A decade of wasted cores [C/OL]// Proceedings of the 11th European Conference on Computer Systems (EuroSys). 2016. DOI: 10.1145/2901318.2901326.

[5] Lawall J, et al. Os scheduling with nest: Keeping tasks close together on warm cores [C]//EuroSys. 2022: 368-383.

[6] Fried J, Ruan Z, Ousterhout A, et al. Caladan: Mitigating interference at microsecond timescales [C/OL]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2020. https://www.usenix.org/conference/osdi20/presentation/fried.

[7] Bhatt N, Parihar A, Prabhu M. Lock-contention-aware scheduler: A scalable and energy-efficient method for addressing scalability collapse on multicore systems [J]. ACM Transactions on Architecture and Code Optimization, 2013, 10(4): 1-23.

[8] Patel Y, et al. Avoiding scheduler subversion using scheduler-cooperative locks [C]//EuroSys. 2020: 1-17.

[9] Kashyap S, et al. Scalable and practical locking with shuffling [C]//SOSP. 2019: 586-599.

[10] Mao H, Schwarzkopf M, Venkatakrishnan S B, et al. Learning scheduling algorithms for data processing clusters [C/OL]//Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM). 2019. DOI: 10.1145/3341302.3342080.

[11] Zheng Y, Fan J, Fu Q, et al. Agentcgroup: Understanding and controlling os resources of ai agents [J/OL]. arXiv preprint arXiv:2602.09345, 2026. https://arxiv.org/abs/2602.09345.

[12] Anonymous. Towards agentic OS: An LLM agent framework for Linux schedulers [Z]. 2025.

[13] Wang X, Jia S, Huang Z, et al. Mixture-of-schedulers: An adaptive scheduling agent as a learned router for expert policies [EB/OL]. 2025. https://arxiv.org/abs/2511.11628.

[14] Humphries J T, Natu N, Chaugule A, et al. ghOSt: Fast and flexible user-space delegation of linux scheduling [C/OL]//Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP). 2021. DOI: 10.1145/3477132.3483542.

[15] Qin H, et al. Arachne:Core-Aware thread management [C]//OSDI. 2018: 145-160.

[16] Miller S, et al. Enoki: High velocity linux kernel scheduler development [C]//EuroSys. 2024: 962-980.

[17] Cao T, Zhong S W, Whitaker C, et al. kSTEP: Characterization and deterministic testing of linux cpu scheduler bugs [C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2026. https://www.usenix.org/conference/osdi26/presentation/cao.

[18] Guo E, et al. Optimizing task scheduling in cloud vms with accurate vcpu abstraction [C]//EuroSys. 2025: 753-768.

[19] Li Y, et al. Libpreemptible: Enabling fast, adaptive, and hardware-assisted user-space scheduling [C]//HPCA. 2024: 922-936.

[20] Gu R, et al. Fluid: Dataset abstraction and elastic acceleration for cloud-native deep learning training jobs [C]//ICDE. 2022: 2182-2195.

[21] Duraisamy P, et al. Towards an adaptable systems architecture for memory tiering at warehouse-scale [C]//ASPLOS. 2023: 727-741.

[22] Park J, et al. Locks as a resource: Fairly scheduling lock occupation with cfl [C]//PPOPP. 2024: 17-29.

[23] Park S, et al. Application-Informed kernel synchronization primitives [C]//OSDI. 2022: 667-682.

[24] Tian B, et al. Contention-aware lock scheduling for transactional databases [J]. PVLDB, 2018, 11 (5).

[25] Bilodeau-Savaria C E, Nidzwetzki J K, Scherzinger S, et al. Unfair by design: eBPF-based scheduling of mixed database workloads [EB/OL]. 2026. https://arxiv.org/abs/2605.02377.

[26] Agache A, Brooker M, Florescu A, et al. Firecracker: Lightweight virtualization for serverless applications [C/OL]//17th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2020. https://www.usenix.org/conference/nsdi20/presentation/agache.

[27] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world github issues? [C/OL]//International Conference on Learning Representations (ICLR). 2024. https://openreview.net/forum?id=VTF8yNQM66.

[28] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering [C/OL]//Advances in Neural Information Processing Systems (NeurIPS). 2024. https://arxiv.org/abs/2405.15793.

[29] Merrill M A, Shaw A G, Carlini N, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces [J/OL]. arXiv preprint arXiv:2601.11868, 2026. https://arxiv.org/abs/2601.11868.

[30] Wu T, Chang C, Cao L, et al. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes [J/OL]. arXiv preprint arXiv:2604.28138, 2026. https://arxiv.org/abs/2604.28138.

[31] Pagonas N, Chung Y, Kaffes K, et al. Cortex: Workflow-aware resource pooling and scheduling for agentic serving [J/OL]. arXiv preprint arXiv:2510.14126, 2025. https://arxiv.org/abs/2510.14126.

[32] Guo D, Wu J, Yiu S M. SAGA: Workflow-atomic scheduling for ai agent inference on gpu clusters [C/OL]//Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC). 2026. https://arxiv.org/abs/2605.00528.

[33] Xiao W, Bhardwaj R, Ramjee R, et al. Gandiva: Introspective cluster scheduling for deep learning [C/OL]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2018. https://www.usenix.org/conference/osdi18/presentation/xiao.

[34] Gu J, Chowdhury M, Shin K G, et al. Tiresias: A gpu cluster manager for distributed deep learning [C/OL]//16th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2019. https://www.usenix.org/conference/nsdi19/presentation/gu.

[35] Mahajan K, Balasubramanian A, Singhvi A, et al. Themis: Fair and efficient gpu cluster scheduling [C/OL]//17th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2020. https://www.usenix.org/conference/nsdi20/presentation/mahajan.

[36] Xiao W, Ren S, Li Y, et al. Antman: Dynamic scaling on gpu clusters for deep learning [C/OL]// 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2020. https: //www.usenix.org/conference/osdi20/presentation/xiao.

[37] Qiao A, Choe S K, Subramanya S J, et al. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning [C/OL]//15th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2021. https://www.usenix.org/conference/osdi21/presentation/qiao.

[38] Han M, Zhang H, Chen R, et al. Microsecond-scale preemption for concurrent gpu-accelerated dnn inferences [C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022. https://www.usenix.org/conference/osdi22/presentation/han.

[39] Shen W, Han M, Liu J, et al. Xsched: Preemptive scheduling for diverse xpus [C/OL]//19th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2025. https://www.usenix.org/conference/osdi25/presentation/shen-weihang.

[40] Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for transformer-based generative models [C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022. https://www.usenix.org/conference/osdi22/presentation/yu.

[41] Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with pagedattention [C/OL]//Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). 2023. DOI: 10.1145/3600006.3613165.

[42] Agrawal A, Kedia N, Panwar A, et al. Taming throughput-latency tradeoff in llm inference with sarathi-serve [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024. https://www.usenix.org/conference/osdi24/presentation/agrawal.

[43] Zhong Y, Liu S, Chen J, et al. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving [C/OL]//18th USENIX Symposium on Operating Systems

Design and Implementation (OSDI). 2024. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[44] Sun B, Huang Z, Zhao H, et al. Llumnix: Dynamic scheduling for large language model serving [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024. https://www.usenix.org/conference/osdi24/presentation/sun-biao.

[45] Song Y, Mi Z, Xie H, et al. Powerinfer: Fast large language model serving with a consumer-grade gpu [C/OL]//Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP). 2024. DOI: 10.1145/3694715.3695964.

[46] Chen H, Wen Y, Cheng L, et al. AutoOS: Make your os more powerful by exploiting large language models [C/OL]//Proceedings of Machine Learning Research: volume 235 Proceedings of the 41st International Conference on Machine Learning (ICML). 2024. https://proceedings.mlr.press/v235/chen24at.html.

[47] Chen Z, Zhang W, Tang Y, et al. Xkernel: Principled performance tunability of operating system kernels [C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2026. https://www.usenix.org/conference/osdi26/presentation/chen-zhongjie.

[48] Liargkovas G, Jabrayilov V, Franke H, et al. Expert in residence: Towards self-optimizing operating systems [C]//NeurIPS Workshop on Machine Learning for Systems. 2025.

[49] Anonymous. KnowOS: Knowledge-grounded language models for operating-system configuration [Z]. 2026.

[50] Rzadca K, et al. Autopilot: workload autoscaling at google [C]//EuroSys. 2020: 1-16.

[51] Linux Kernel Developers. Extensible scheduler class [EB/OL]. 2026. https://docs.kernel.org/scheduler/sched-ext.html.

[52] Dice D, Kogan A. Avoiding scalability collapse by restricting concurrency [C/OL]//Lecture Notes in Computer Science: volume 11725 Proceedings of the 25th International European Conference on Parallel and Distributed Computing (Euro-Par). Springer, 2019: 363-376. DOI: 10.1007/978-3-030-29400-7_26.

[53] Mellor-Crummey J M, Scott M L. Algorithms for scalable synchronization on shared-memory multiprocessors [J/OL]. ACM Transactions on Computer Systems, 1991, 9(1): 21-65. DOI: 10.1145/103727.103729.

[54] Dice D, Marathe V J, Shavit N. Lock cohorting: A general technique for designing NUMA locks [C/OL]//Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP). ACM, 2012: 247-256. DOI: 10.1145/2145816.2145848.

[55] Kashyap S, Calciu I, Cheng X, et al. Scalable and practical locking with shuffling [C/OL]// Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2019: 586-599. DOI: 10.1145/3341301.3359629.

[56] Dice D, Kogan A. Compact NUMA-aware locks [C/OL]//Proceedings of the 14th European Conference on Computer Systems (EuroSys). ACM, 2019: 1-15. DOI: 10.1145/3302424.3303984.

[57] Cui Y, Wang Y, Chen Y, et al. Lock-contention-aware scheduler: A scalable and energy-efficient method for addressing scalability collapse on multicore systems [J/OL]. ACM Transactions on Architecture and Code Optimization, 2013, 9(4). DOI: 10.1145/2400682.2400703.

[58] Park J, Eom Y I. Locks as a resource: Fairly scheduling lock occupation with CFL [C/OL]// Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP). ACM, 2024: 17-29. DOI: 10.1145/3627535.3638477.

[59] Lawall J, Chhaya-Shailesh H, Lozi J P, et al. OS scheduling with nest: Keeping tasks close together on warm cores [C/OL]//Proceedings of the 17th European Conference on Computer Systems (EuroSys). ACM, 2022: 368-383. DOI: 10.1145/3492321.3519585.

[60] Patel Y, Yang L, Arulraj L, et al. Avoiding scheduler subversion using scheduler-cooperative locks [C/OL]//EuroSys '20: Proceedings of the Fifteenth European Conference on Computer Systems. ACM, 2020. DOI: 10.1145/3342195.3387521.

[61] Park S, Zhou D, Qian Y, et al. Application-informed kernel synchronization primitives [C/OL]// Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2022: 667-682. https://www.usenix.org/conference/osdi22/presentation/park.

[62] Stoica I, Abdel-Wahab H. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation: 95-22 [R/OL]. Old Dominion University, 1995. https://www.cs.cmu.edu/~istoica/pubs.html.

[63] Linux kernel documentation. Extensible scheduler class [EB/OL]. 2024. https://docs.kernel.org/scheduler/sched-ext.html.

[64] Franke H, Russell R, Kirkwood M. Fuss, futexes and furwocks: Fast userlevel locking in Linux [C/OL]//Proceedings of the Ottawa Linux Symposium. 2002: 479-495. https://www.kernel.org/doc/ols/2002/ols2002-pages-479-495.pdf.

[65] Kim S, Kim H, Lee J, et al. Enlightening the I/O path: A holistic approach for application performance [C/OL]//Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2017: 345-358. https://www.usenix.org/conference/fast17/technical-sessions/presentation/kim-sangwook.

[66] Kontothanassis L I, Wisniewski R W, Scott M L. Scheduler-conscious synchronization [J/OL]. ACM Transactions on Computer Systems, 1997, 15(1): 3-40. DOI: 10.1145/244764.244765.

[67] Xian F, Srisa-an W, Jiang H. Contention-aware scheduler: Unlocking execution parallelism in multithreaded Java programs [C/OL]//Proceedings of the 23rd ACM SIGPLAN Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA). ACM, 2008: 163-180. DOI: 10.1145/1449764.1449778.

[68] Teabe B, Nitu V, Tchana A, et al. The lock holder and the lock waiter pre-emption problems: Nip them in the bud using informed spinlocks (I-Spinlock) [C/OL]//Proceedings of the 12th European Conference on Computer Systems (EuroSys). ACM, 2017: 286-297. DOI: 10.1145/3064176.3064180.

[69] Laforet V, Kashyap S, Iorgulescu C, et al. FlexGuard: Fast mutual exclusion independent of subscription [C/OL]//Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP). ACM, 2025: 1115-1130. DOI: 10.1145/3731569.3764852.

[70] Linux kernel documentation. Restartable sequences [EB/OL]. 2026. https://docs.kernel.org/userspace-api/rseq.html#scheduler-time-slice-extensions.

[71] Lozi J P, Lepers B, Funston J, et al. The Linux scheduler: A decade of wasted cores [C/OL]//Proceedings of the 11th European Conference on Computer Systems (EuroSys). ACM, 2016: 1-16. DOI: 10.1145/2901318.2901326.

[72] Isstaif A A T, Kalyvianaki E, Mortier R. Mitigating context switching in densely packed Linux clusters with latency-aware group scheduling [EB/OL]. 2025. https://arxiv.org/abs/2508.15703.

[73] Cai M, Liu S, Huang H. tScale: A contention-aware multithreaded framework for multicore multiprocessor systems [C/OL]//Proceedings of the IEEE International Conference on Parallel and Distributed Systems (ICPADS). IEEE, 2017: 334-343. DOI: 10.1109/ICPADS.2017.00052.

[74] Chabbi M, Fagan M W, Mellor-Crummey J M. High performance locks for multi-level NUMA systems [C/OL]//Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP). ACM, 2015: 215-226. DOI: 10.1145/2688500.2688503.

[75] Kashyap S, Min C, Kim T. Scalable NUMA-aware blocking synchronization primitives [C/OL]//Proceedings of the 2017 USENIX Annual Technical Conference (ATC). USENIX Association, 2017: 603-615. https://www.usenix.org/conference/atc17/technical-sessions/presentation/kashyap.

[76] Lo S, Lin H T, Hsieh Y H, et al. RON: One-way circular shortest routing to achieve efficient and bounded-waiting spinlocks [C/OL]//Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2023: 17-31. https://www.usenix.org/conference/osdi23/presentation/lo.

[77] Gupta V, Dwivedi K K, Kothari Y, et al. Ship your critical section, not your data: Enabling transparent delegation with TCLOCKS [C/OL]//Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2023: 1-16. https://www.usenix.org/conference/osdi23/presentation/gupta.

[78] Corbet J. AutoNUMA: the other approach to NUMA scheduling [EB/OL]. 2012. https://lwn.net/Articles/488709/.

[79] Tian B, Huang J, Mozafari B, et al. Contention-aware lock scheduling for transactional databases [J/OL]. Proceedings of the VLDB Endowment, 2018, 11(5): 648-662. DOI: 10.1145/3187009.3177740.

[80] Kalikar S, Nasre R. Toggle: Contention-aware task scheduler for concurrent hierarchical operations [C/OL]//Lecture Notes in Computer Science: volume 11725 Proceedings of the 25th International European Conference on Parallel and Distributed Computing (Euro-Par). Springer, 2019: 142-155. DOI: 10.1007/978-3-030-29400-7_11.

[81] Cui Y, Wang Y, Chen Y, et al. Requester-based spin lock: A scalable and energy efficient locking scheme on multicore systems [J/OL]. IEEE Transactions on Computers, 2015, 64(1): 166-179. DOI: 10.1109/TC.2013.196.

[82] Ji J, Gu Y, Fu Y, et al. Beyond the bottleneck: Enhancing high-concurrency systems with lock tuning [C/OL]//Proceedings of the International Symposium on Formal Methods. Springer, 2024: 325-337. DOI: 10.1007/978-3-031-71177-0_20.

[83] Pusukuri K K, Gupta R, Bhuyan L N. Shuffling: A framework for lock contention aware thread scheduling for multicore multiprocessor systems [C/OL]//Proceedings of the 23rd International Conference on Parallel Architectures and Compilation Techniques (PACT). ACM, 2014: 289-300. DOI: 10.1145/2628071.2628074.

[84] Yu Z, Gu J, Wu Z, et al. HTLL: Latency-aware scalable blocking mutex [J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2025, 36(3): 471-486. DOI: 10.1109/TPDS.2025.3526859.

[85] Tam D, Azimi R, Stumm M. Thread clustering: Sharing-aware scheduling on SMP-CMP-SMT multiprocessors [C/OL]//Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys). ACM, 2007: 47-58. DOI: 10.1145/1272996.1273004.

[86] Lepers B, Quéma V, Fedorova A. Thread and memory placement on NUMA systems: Asymmetry matters [C/OL]//Proceedings of the 2015 USENIX Annual Technical Conference (ATC). USENIX Association, 2015: 277-289. https://www.usenix.org/conference/atc15/technical-session/presentation/lepers.

[87] Chen T, Chen Y. Cache aware scheduling [EB/OL]. 2026. https://lore.kernel.org/lkml/cover.1770760558.git.tim.c.chen@linux.intel.com/.

[88] Tang W, Ke Y, Fu S, et al. Demeter: QoS-aware CPU scheduling to reduce power consumption of multiple black-box workloads [C/OL]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). ACM, 2022: 31-46. DOI: 10.1145/3542929.3563476.

[89] Zheng Y, Li C, Xiong Y, et al. iAware: Interaction aware task scheduling for reducing resource contention in mobile systems [J/OL]. ACM Transactions on Embedded Computing Systems, 2023, 22(5s): 1-24. DOI: 10.1145/3609391.

[90] Guo E, Jia W, Ding X, et al. Optimizing task scheduling in cloud VMs with accurate vCPU abstraction [C/OL]//Proceedings of the European Conference on Computer Systems (EuroSys). ACM, 2025: 753-768. DOI: 10.1145/3689031.3696092.

[91] Álvarez D, Sala K, Beltran V. nOS-V: Co-executing HPC applications using system-wide task scheduling [C/OL]//Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, 2024: 312-324. DOI: 10.1109/IPDPS57955.2024.00035.

[92] Humphries J T, Natu N, Chaugule A, et al. ghOSt: Fast & flexible user-space delegation of linux scheduling [C/OL]//Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2021: 588-604. DOI: 10.1145/3477132.3483542.

[93] Fried J, Ruan Z, Ousterhout A, et al. Caladan: Mitigating interference at microsecond timescales [C/OL]//Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2020: 281-297. https://www.usenix.org/conference/osdi20/presentation/fried.

[94] Qin H, Li Q, Speiser J, et al. Arachne: Core-aware thread management [C/OL]//Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2018: 145-160. https://www.usenix.org/conference/osdi18/presentation/qin.

[95] Miller S, Kumar A, Vakharia T, et al. Enoki: High velocity Linux kernel scheduler development [C/OL]//Proceedings of the European Conference on Computer Systems (EuroSys). ACM, 2024: 962-980. DOI: 10.1145/3627703.3629569.

[96] Jia Y, Tian K, You Y, et al. Skyloft: A general high-efficient scheduling framework in user space [C/OL]//Proceedings of the ACM Symposium on Operating Systems Principles (SOSP). ACM, 2024: 265-279. DOI: 10.1145/3694715.3695973.

[97] Li Y, Lazarev N, Koufaty D, et al. LibPreemptible: Enabling fast, adaptive, and hardware-assisted user-space scheduling [C/OL]//Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2024: 922-936. DOI: 10.1109/HPCA57654.2024.00075.

[98] Chen Z, Zhang W, Tang Y, et al. Xkernel: Principled performance tunability of operating system kernels [C/OL]//Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2026: 547-565. https://www.usenix.org/conference/osdi26/presentation/chen-zhongjie.

[99] Zheng Y, Fan J, Fu Q, et al. AgentCgroup: Understanding and controlling OS resources of AI agents [EB/OL]. 2026. https://arxiv.org/abs/2602.09345.

[100] Mao H, Schwarzkopf M, Venkatakrishnan S B, et al. Learning scheduling algorithms for data processing clusters [C/OL]//Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM). 2019. DOI: 10.1145/3341302.3342080.

[101] Tang C, et al. Twine: A unified cluster management system for shared infrastructure [C]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2020: 787-803.

[102] Xiang W, et al. Gödel: Unified large-scale resource management and scheduling at ByteDance [C]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). 2023: 308-323.

[103] Humphries J T, Natu N, Chaugule A, et al. ghOSt: Fast and flexible user-space delegation of linux scheduling [C/OL]//Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP). 2021. DOI: 10.1145/3477132.3483542.

[104] Linux Kernel Developers. Extensible scheduler class [EB/OL]. 2026. https://docs.kernel.org/scheduler/sched-ext.html.

[105] Agache A, Brooker M, Florescu A, et al. Firecracker: Lightweight virtualization for serverless applications [C/OL]//17th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2020. https://www.usenix.org/conference/nsdi20/presentation/agache.

[106] Wu T, Chang C, Cao L, et al. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes [EB/OL]. 2026. https://arxiv.org/abs/2604.28138.

[107] Park S, et al. Application-informed kernel synchronization primitives [C]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022: 667-682.

[108] Cao T, Zhong S W, Whitaker C, et al. kSTEP: Characterization and deterministic testing of Linux CPU scheduler bugs [C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2026. https://www.usenix.org/conference/osdi26/presentation/cao.

[109] Lozi J P, Lepers B, Funston J, et al. The Linux scheduler: A decade of wasted cores [C/OL]// Proceedings of the 11th European Conference on Computer Systems (EuroSys). 2016. DOI: 10.1145/2901318.2901326.

[110] Lawall J, Chhaya-Shailesh H, Lozi J P, et al. OS scheduling with Nest: Keeping tasks close together on warm cores [C/OL]//Proceedings of the 17th European Conference on Computer Systems (EuroSys). 2022. DOI: 10.1145/3492321.3519585.

[111] Fried J, Ruan Z, Ousterhout A, et al. Caladan: Mitigating interference at microsecond timescales [C/OL]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2020. https://www.usenix.org/conference/osdi20/presentation/fried.

[112] Tang W, et al. Demeter: QoS-aware CPU scheduling to reduce power consumption of multiple black-box workloads [C]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). 2022: 31-46.

[113] Guo E, et al. Optimizing task scheduling in cloud VMs with accurate vCPU abstraction [C]// Proceedings of the European Conference on Computer Systems (EuroSys). 2025: 753-768.

[114] Isstaif A A T, Kalyvianaki E, Mortier R. Mitigating context switching in densely packed Linux clusters with latency-aware group scheduling [EB/OL]. 2025. https://arxiv.org/abs/2508.15703.

[115] Zheng Y, et al. iAware: Interaction-aware task scheduling for reducing resource contention in mobile systems [J]. ACM Transactions on Embedded Computing Systems, 2023.

[116] Alvarez D, et al. nOS-V: Co-executing HPC applications using system-wide task scheduling [C]// IEEE International Parallel and Distributed Processing Symposium (IPDPS). 2024: 312-324.

[117] Zheng H, Nieh J. SWAP: A scheduler with automatic process dependency detection [C/OL]//First Symposium on Networked Systems Design and Implementation (NSDI 04). San Francisco, CA: USENIX Association, 2004. https://www.usenix.org/conference/nsdi-04/swap-scheduler-automatic-process-dependency-detection.

[118] Qin H, et al. Arachne: Core-aware thread management [C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2018: 145-160.

[119] Jia Y, et al. Skyloft: A general high-efficient scheduling framework in user space [C]//Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP). 2024: 265-279.

[120] Yasukata K, et al. Developing process scheduling policies in user space with common OS features [C]//Asia-Pacific Workshop on Systems (APSys). 2024: 38-44.

[121] Li Y, et al. Libpreemptible: Enabling fast, adaptive, and hardware-assisted user-space scheduling [C]//IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2024: 922-936.

[122] Miller S, Kumar A, Vakharia T, et al. Enoki: High velocity Linux kernel scheduler development [C]// Proceedings of the European Conference on Computer Systems (EuroSys). 2024.

[123] Chen Z, Zhang W, Tang Y, et al. Xkernel: Principled performance tunability of operating system kernels [C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2026. https://www.usenix.org/conference/osdi26/presentation/chen-zhongjie.

[124] Chen H, Wen Y, Cheng L, et al. AutoOS: Make your OS more powerful by exploiting large language models [C/OL]//Proceedings of the 41st International Conference on Machine Learning (ICML). 2024. https://proceedings.mlr.press/v235/chen24at.html.

[125] Patel Y, et al. Avoiding scheduler subversion using scheduler-cooperative locks [C]//Proceedings of the European Conference on Computer Systems (EuroSys). 2020: 1-17.

[126] Rzadca K, et al. Autopilot: Workload autoscaling at Google [C]//Proceedings of the European Conference on Computer Systems (EuroSys). 2020: 1-16.

[127] Shukla D, et al. Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads [EB/OL]. 2022. https://arxiv.org/abs/2202.07848.

[128] Weng Q, et al. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters [C]//19th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2022: 945-960.

[129] Cheng Y, Xu L, Yang T, et al. Beehive: Decentralised high-frequency small tasks scheduling in large clusters [J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2025, 36(6): 1326-1337. DOI: 10.1109/TPDS.2025.3563457.

[130] Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for transformer-based generative models [C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022. https://www.usenix.org/conference/osdi22/presentation/yu.

[131] Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with PagedAttention [C/OL]//Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). 2023. DOI: 10.1145/3600006.3613165.

[132] Agrawal A, Kedia N, Panwar A, et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024. https://www.usenix.org/conference/osdi24/presentation/agrawal.

[133] Zhong Y, Liu S, Chen J, et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[134] Sun B, Huang Z, Zhao H, et al. Llumnix: Dynamic scheduling for large language model serving [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024. https://www.usenix.org/conference/osdi24/presentation/sun-biao.

[135] Han M, Zhang H, Chen R, et al. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences [C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022. https://www.usenix.org/conference/osdi22/presentation/han.

[136] Shen W, Han M, Liu J, et al. XSched: Preemptive scheduling for diverse XPUs [C/OL]//19th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2025. https://www.usenix.org/conference/osdi25/presentation/shen-weihang.

[137] Song Y, Mi Z, Xie H, et al. Powerinfer: Fast large language model serving with a consumer-grade GPU [C/OL]//Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP). 2024. DOI: 10.1145/3694715.3695964.

[138] Guo D, Wu J, Yiu S M. SAGA: Workflow-atomic scheduling for AI agent inference on GPU clusters [C/OL]//Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC). 2026. https://arxiv.org/abs/2605.00528.

[139] Pagonas N, Chung Y, Kaffes K, et al. Cortex: Workflow-aware resource pooling and scheduling for agentic serving [EB/OL]. 2025. https://arxiv.org/abs/2510.14126.

[140] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world github issues? [C/OL]//International Conference on Learning Representations (ICLR). 2024. https://openreview.net/forum?id=VTF8yNQM66.

[141] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering [C]//Advances in Neural Information Processing Systems (NeurIPS). 2024.

[142] Badertdinov I, Golubev A, Andryunin E, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents [C/OL]//Advances in Neural Information Processing Systems: Datasets and Benchmarks Track. 2025. https://arxiv.org/abs/2505.20411.

[143] Merrill M A, Shaw A G, Carlini N, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces [EB/OL]. 2026. https://arxiv.org/abs/2601.11868.

研究成果说明

本研究实现了 KernAdapt 控制面、LOCKCOOP 锁调度机制和 FlowCausal 工具工作流调度机制。KernAdapt 负责资源配置选择、执行检查与恢复；LockCoop 和 FlowCausal 分别处理锁竞争与工具等待。

LOCKCOOP 相关论文已投稿 EuroSys 2027；FlowCausal 已形成论文稿，计划投稿 OSDI 2027。KernAdapt 论文研究组合动作的效果预测、方案选择风险及实际执行效果。